

Unit 1 — Introducing Streaming Data & Data Ingestion

Stream Processing and Analytics (AI372IA) · RVCE VII Sem · Syllabus: real-time systems · real-time vs streaming · architectural blueprint · security · scaling · common interaction patterns · scaling the patterns · fault tolerance

Sources used: Psaltis *Streaming Data* Ch.1–2 (printed pp.3–37) + faculty PPT "Unit I" + all CIE/SEE/Model papers. **BOOK p.xx** ↗ chips open *Streaming-Data[1].pdf* at that printed page (PDF page = printed + 20). Blue **highlight** = must-remember. **IMPORTANT** = asked in a past paper.

0 · Exam pattern & what Unit 1 looks like in the paper

Part	Pattern (from SEE Feb'26, Model QP, syllabus rubric)
Part A (Q1)	10 short questions × 2 marks = 20 marks , whole syllabus (all units). Some are fill-ups / scenario one-liners.
Part B	5 full questions × 16 marks = 80 . Q2 (Unit 1) is COMPULSORY . Then one of Q3/Q4 (Unit 2), Q5/Q6 (Unit 3), Q7/Q8 (Unit 4), Q9/Q10 (Unit 5). Each question has max 3 sub-parts (e.g. 8+8, 10+6).
Unit 1 in SEE Feb'26	Q2a (8) "streaming pattern for collecting data with an example"; Q2b (8) "publish/subscribe pattern with neat diagram". Part A: 1.1 stateless design, 1.2 which interaction pattern, 1.3 which logging technique.

Topic hot-list (ranked by how often asked)

Rank	Topic	Where it appeared	Notes section
1	Interaction patterns (esp. Pub/Sub , Stream, compare all)	SEE Feb'26 Q2a,Q2b,1.2 · CIE-1 Dec'24 Q2a,Q3a · CIE-1 Nov'25 PartA-1, Q2b	\$4
2	Streaming data architecture (draw + explain) for a scenario	Model QP Q2a · CIE-1 Dec'24 Q1a (vehicle tracking) · CIE-1 Nov'25 Q1a (IoT)	\$2, \$8
3	RBML / SBML / HML, checkpointing	CIE-1 Dec'24 Q3b · CIE-1 Nov'25 PartA-3, Q1a · SEE Feb'26 1.3 · Model QP 1.2	\$6
4	Real-time classes (hard/soft/near) & streaming vs real-time	Model QP 1.1 · CIE-1 Dec'24 PartA-1 · CIE-1 Nov'25 Q2a	\$1
5	Scaling (vertical vs horizontal; importance)	Model QP Q2b · CIE-1 Dec'24 PartA-3 · CIE-1 Nov'25 Q1a	\$3, \$5
6	Needs for streaming data; stateless design	CIE-1 Dec'24 & Nov'25 PartA-2 · SEE Feb'26 1.1	\$1

1 · Real-time systems vs streaming systems

1.1 What is a real-time system? [BOOK p.4 ↗](#)

- A real-time system produces results within a **time limit**. Three classes: **Hard, Soft, Near** (hard/soft after Kopetz; near from Portland Pattern wiki).
- **Hard**: very strict deadline; a miss ⇒ **total system failure / loss of life**. Embedded systems. (Out of scope of the book.)
- Soft vs near: **the line is blurry, subjective and depends on the consumer** of the data.

Table 1.1 (textbook) — Classification of real-time systems [BOOK p.5 ↗](#)

Class	Examples	Latency measured in	Tolerance for delay
Hard	Pacemaker, anti-lock brakes	Microseconds–milliseconds	None — system failure, loss of life
Soft	Airline reservation system, online stock quotes, VoIP (Skype)	Milliseconds–seconds	Low — no system failure, no life at risk
Near	Skype video, home automation	Seconds–minutes	High — no system failure, no life at risk

Faculty PPT slides 7 & 9 — extra comparison (hard vs soft; soft vs near)

Hard vs Soft: response time **hard = perform predictably** / soft = degraded · error detection: system / user · safety: critical / non-critical · redundancy: active / standby · time granularity: millisecond / second · data files: small-medium / large.

Soft vs Near: timing precision — soft "deadlines important but flexible", near "processed as quickly as possible with very minimal delay" · tolerance — soft "some delay or missed deadlines OK", near "some lag (often in seconds) acceptable" · examples — soft: video streaming, VoIP, online gaming; near: stock trading, traffic monitoring, sensor data processing.

⚠ CONFLICT / TRAP — Soft vs Near examples differ between sources

Textbook Table 1.1 puts *online stock quotes* under **soft** and *Skype video / home automation* under **near**, and says near has **higher** delay tolerance (seconds–minutes). The **faculty slides (slide 9/12) and the CIE-1 Nov'25 solution** put *stock trading* under **near** and *video streaming* under **soft**, and describe near as "very low delay, almost instantaneous".

Which to write: the paper is set from the faculty slides, so for a 2-mark "classify soft and near with an example" write: *Soft — deadlines preferred, occasional delay tolerable, latency ms–s (e.g. video streaming/VoIP). Near — processed with minimal lag of seconds (e.g. stock trading ticker, traffic monitoring)*. Quote the Table 1.1 latency numbers; avoid the word "instantaneous" and avoid stating which has higher tolerance. In a Part B answer, give the Table 1.1 table *and* mention the slide examples.

💡 MNEMONIC — Hard–Soft–Near = H–S–N = "Microseconds, Milliseconds, Minutes"

Hard **µs–ms** (no tolerance) → Soft **ms–s** (low) → Near **s–min** (high). Latency grows, tolerance grows.

1.2 Real-time vs streaming BOOK p.7 ↗

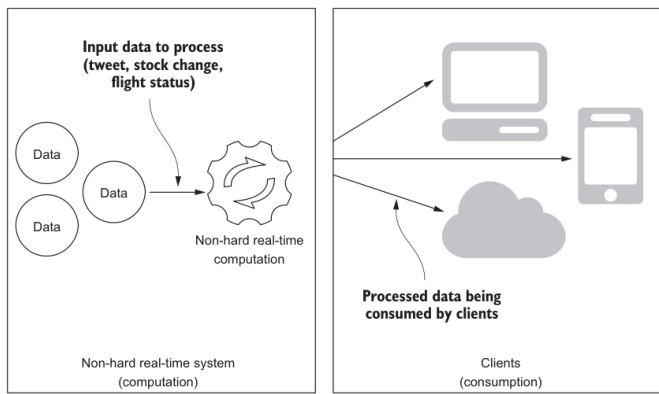


Figure 1.3 Real-time computation and consumption split apart

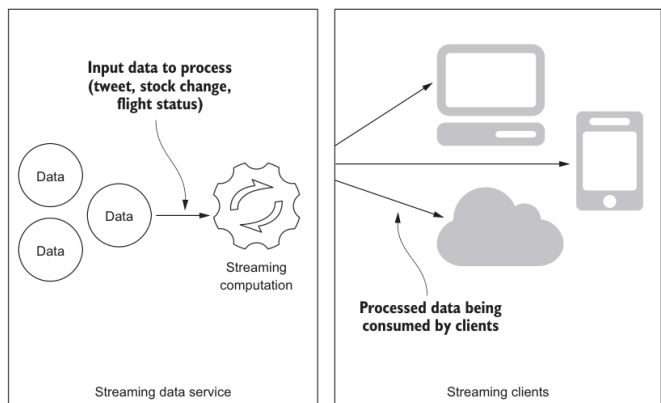


Figure 1.4 A first view of a streaming data system

Fig 1.3 Computation and consumption split apart (book p.7) Fig 1.4 A first view of a streaming data system (book p.8)

DEFINITION — Streaming data system
A **non-hard real-time service** whose computation runs continuously, with **clients that consume the data when they need it** (delays from network, app design, client not running). It is "neither soft nor near — it is **streaming**" / an **in-the-moment system**.

- Split the system in two: **computation** (non-hard real-time) and **consumption** (clients). Streaming = serve data at the moment the client needs it.
- Examples: **Twitter** (latest tweets on request, some seconds old, some hours old); **FlightAware** (latest status per airport/flight); **NASDAQ Real Time Quotes** (latest quote per stock).

Real-time system	Streaming data system
Judged by delay seen by consumer (soft/near/hard)	Judged by whether data is available when client asks
One word for both computation + consumption — ambiguous	Separates computation (non-hard RT) from consumption (clients pull/receive when ready)
Deadline-centric	Scalability, availability, continuous flow-centric

1.3 Need and characteristics of streaming data (faculty PPT slides 17–18)

IMPORTANT CIE-1 Dec'24 A2 · CIE-1 Nov'25 A2 (list any two)

- Some data **naturally comes as a never-ending stream** of events.
- Batch lets data pile up and processes at once; stream processes as it arrives, spreading the work over time.
- Data is so huge it may be **impossible to store**.
- Huge sources (transactions, activity, web visits) growing faster with IoT sensors.

Characteristics (slide 18): **Chronological significance** (order matters) · **Continuous flow** · **Uniqueness** · **Non-homogeneity** · **Imperfection**.

MNEMONIC — CCUNI — "Chrono, Continuous, Unique, Non-homogeneous, Imperfect"
Five characteristics of streaming data.

🔥 HOW TO WRITE IT — Part A "list two needs"

Write any two numbered points above in one line each — 1 mark each. Don't elaborate.

1.4 Stateless design (SEE Feb'26 Q1.1) **IMPORTANT** SEE Feb'26 1.1

- **Stateless** = the service keeps **no information about the client between requests**; every request carries everything needed. So a client can call **any** instance and instances can be added/removed freely. ([BOOK p.26 ↗](#) (book p.26: "we don't have any state information about the client... can easily add new instances"))
- *Live video analytics example*: each camera frame/chunk is sent to any analytics instance behind a load balancer, processed on its own and the result returned; no instance remembers earlier frames, so a crashed instance is simply replaced and auto-scaling works. (Example is ours; the idea is from p.26.)

2 · The architectural blueprint [BOOK p.9 ↗](#)

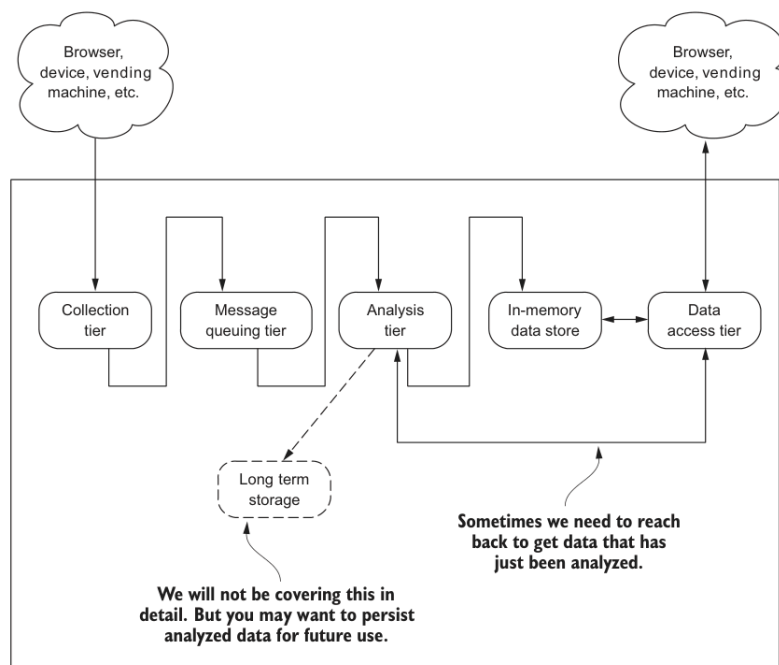


Figure 1.5 The streaming data architectural blueprint

Fig 1.5 The streaming data architectural blueprint (book p.9) — DRAW THIS

Tier	Job	Twitter example (book p.10)
Collection tier	Entry point: takes data from browsers/devices/vending machines etc.	User posts a tweet → collected by Twitter services
Message queuing tier	Decouples collection from analysis; buffers/transport	Collection and analysis may be in different data centres
Analysis tier	Stream processing / continuous queries on in-flight data	Identify the followers of a tweet
In-memory data store	Holds freshest results for fast access	Tweets a few seconds old
Data access tier	Delivers data to clients (API)	All Twitter clients connect to access the service
Long-term storage (optional, dashed)	Persist analysed data for later	Old tweets in a persistent store

- Tiers are **not rigid like LEGO pieces**, may be spread over several physical locations; they don't prescribe deployment. [BOOK p.10 ↗](#)
- Message queuing tier is not stressed in Fig 1.7 but still plays a collection role in some cases.

🧠 MNEMONIC — C-M-A-I-D (+L)

Collection → Message queue → Analysis → In-memory store → Data access; Long-term storage hangs off the analysis tier.

Security for streaming systems [BOOK p.10 ↗](#)

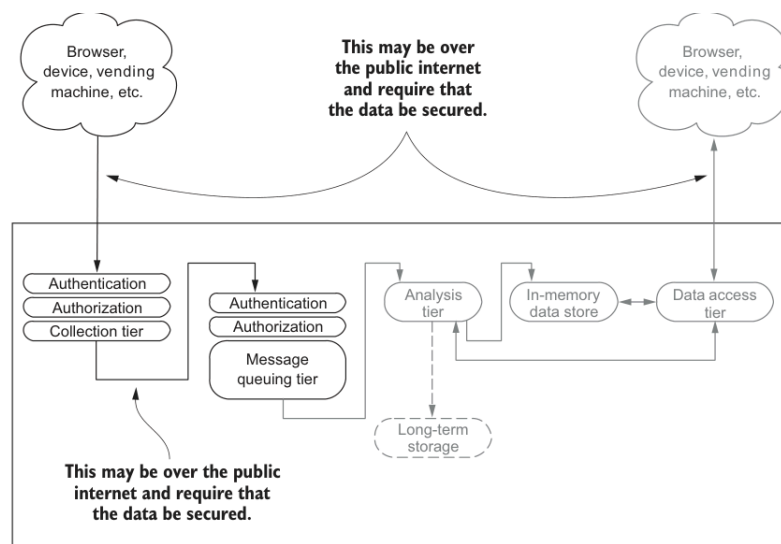


Figure 1.6 The architectural blueprint with security identified

Fig 1.6 Blueprint with security identified (book p.11)

- Security is **overlaid** on the blueprint, not a separate tier. Data travels over the **public internet** at the collection side and the data-access side ⇒ must be secured.
- Apply **Authentication** (who are you) and **Authorization** (what may you do) at the collection tier, the message queuing tier and the data access side. (Also encryption in transit/at rest — see Unit 2 Fig 3.14.)

3 · How do we scale? [BOOK p.11 ↗](#) **IMPORTANT** CIE-1 Dec'24 A3 · Model QP 2b

	Vertical scaling (scale up)	Horizontal scaling (scale out)
Meaning	Add resources (CPU, RAM, disk) to the existing server	Add more servers and spread the load
Limit	Capped by maximum capacity of one machine	Practically unlimited; can shrink too (elasticity / auto-scaling)
Analogy (book)	Restaurant: more tables/chairs until max occupancy	Highway: add lanes; airport: open/close security lines
Example	Add memory to a collection node	Add nodes behind a load balancer

- Goal for every tier of a streaming system = **horizontal scaling**; vertical is used only to find an ideal resource config, then you scale out. [BOOK p.12 ↗](#)
- **Importance of scaling** (Model QP 2b, 6M): handle growing volume/velocity (millions of vehicles, 46,000 searches/s), avoid single-machine ceiling, keep latency low during spikes, elasticity saves cost, supports fault tolerance via redundancy.

4 · Common interaction patterns (collection tier) [BOOK p.15 ↗](#)

Five patterns, whatever the protocol: **Request/response · Request/acknowledge · Publish/subscribe · One-way · Stream**. **IMPORTANT** SEE Feb'26 Q2a,Q2b,1.2 · CIE-1 Nov'25 A1,Q2b · CIE-1 Dec'24 Q2a,Q3a

 **MNEMONIC — R-R-A-P-O-S → "Really Rapid Acknowledgement, Pub, One, Stream"**
Request/Response · Request/Acknowledge · Pub/sub · One-way · Stream.

4.1 Request/response [BOOK p.16 ↗](#)

- Client sends request; service sends response over the **same connection**. Simplest, **synchronous**; client must wait. Use when an **immediate response** is needed (web browsing, Google search, buy a ticket).
- Three fixes for waiting: **half-async client side** (client sends async and does other work — all browsers), **half-async service side** (service delegates work then replies; more scalable), **full-async** (both).
- Example in book: real-time traffic & routing service for all vehicles — full-async variant chosen. [BOOK p.18 ↗](#)

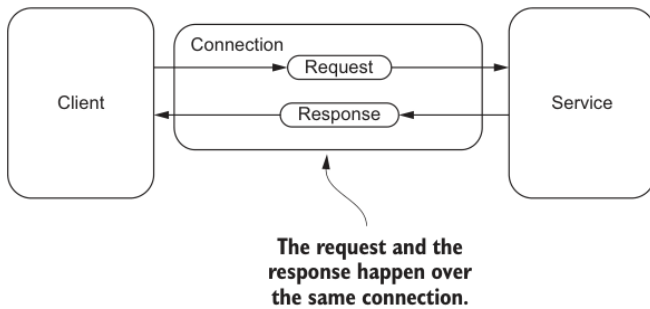


Figure 2.2 Basic request/response pattern

Fig 2.2 Basic request/response (p.16)

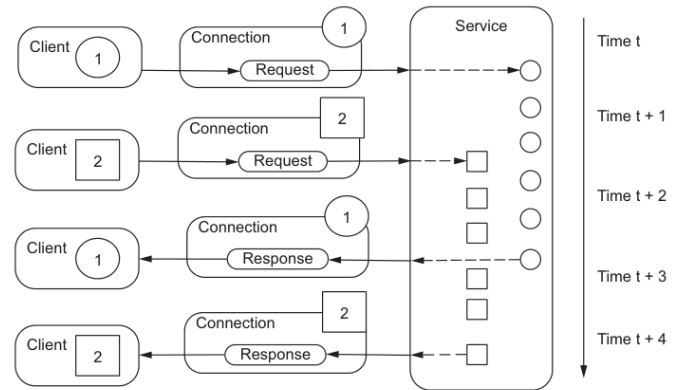


Figure 2.4 Service async request/response pattern

Fig 2.4 Service async request/response (p.17)

4.2 Request/acknowledge [BOOK p.19 ↗](#) **IMPORTANT** SEE Feb'26 1.2

- Like request/response but you only need **confirmation the request was received**, not the processed result. The acknowledgement carries a **token/ID** used in later requests (check status, get final result). Real life: order confirmation number.
- Book example: e-commerce **propensity-to-buy score** — first page visit returns a unique ID (ACK); later pages pass that ID to the propensity service. [BOOK p.19 ↗](#)

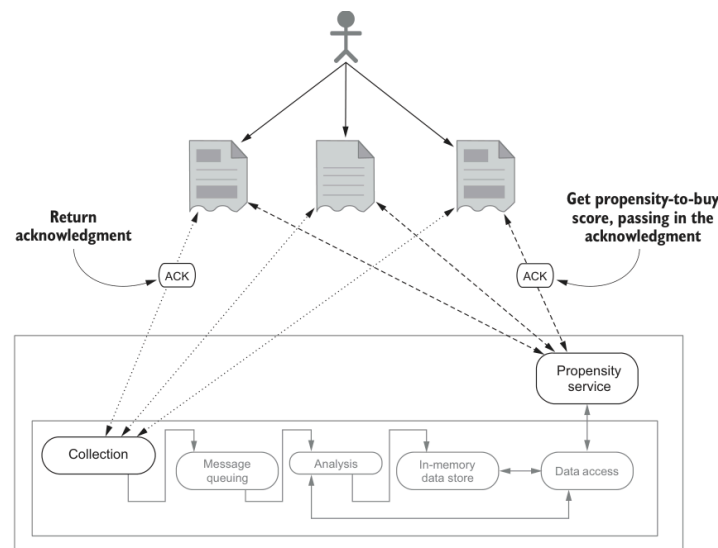


Figure 2.6 Visitor browsing while data is collected and their propensity-to-buy score is updated

Fig 2.6 Visitor browsing; ACK ID used to fetch propensity score (p.20)

🔥 HOW TO WRITE IT — SEE Feb'26 Q1.2 — warehouse sends an order update, only needs confirmation it was received

Request/acknowledge pattern. The warehouse system gets an acknowledgement token as soon as the message is received (no processed response needed), so it knows the message was not lost, can retry if no ACK arrives, and can use the token later to check status ⇒ improves reliability and keeps the sender non-blocking.

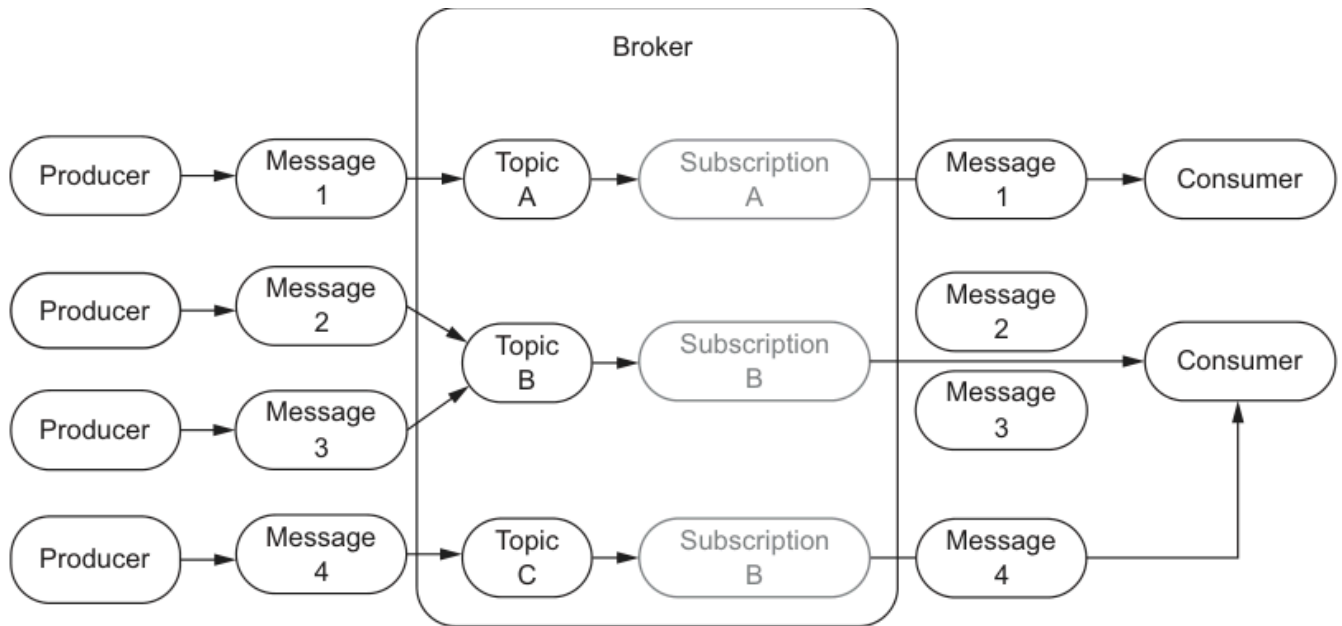


Figure 2.7 General data flow for the publish/subscribe message pattern

Fig 2.7 General data flow for the pub/sub pattern (book p.21) — DRAW THIS for Q2b

1. **Producer** publishes a message to a **broker**, usually into a **topic** (logical grouping).
2. Broker keeps topics and **subscriptions**; delivers the message to **all consumers subscribed** to that topic (some products push, others let the consumer pull).
3. Consumer reads it asynchronously. A producer need not subscribe; a subscriber need not produce.

Component (PPT slides 35–38)	Role
Publisher	Creates messages, classifies them by topic; doesn't know subscribers
Subscriber	Declares interest in topics; doesn't know publishers
Topic	Named channel/category of messages
Message broker	Intermediary: routes, can add persistence, scalability, reliability
Message	Unit of data (text, JSON, binary)
Subscription	Link subscriber→topic; may carry delivery guarantees (at-most/at-least-once) and ack rules

- **Benefit:** **decouples sender from consumer** (time, space, identity) — vital when scaled from 4 cars to all cars in the US. Fully **asynchronous**.
- **Example (book Fig 2.8):** cars publish "current traffic" to a *Traffic topic*; a car with a traffic subscription consumes it. [BOOK p.22](#)

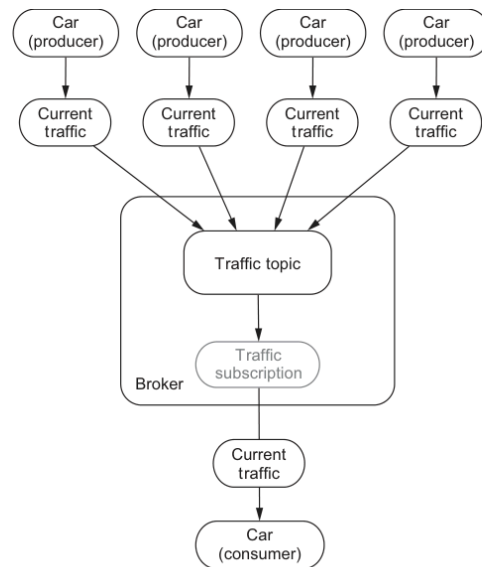


Figure 2.8 Current traffic publish/subscribe message pattern

Fig 2.8 Current traffic pub/sub (p.22)

🔥 HOW TO WRITE IT — Q2b 8 marks

Definition (1) → labelled Fig 2.7 (3) → 6 flow steps (2) → components table + benefit (1) → example (1).

4.4 One-way pattern ("fire and forget") [BOOK p.22 ↗](#)

- Client sends, service sends **no response**; client doesn't even know whether it arrived. Suits clients with no resources/need to process a reply, and where **losing some data is tolerable** for simplicity, low resource use and speed.
- Examples: server sending CPU/memory stats every 10 s; **NFL RFID tags** sending 25 samples/s — losing 5 doesn't change the analysis; weather collector sending periodic updates (CIE-1 Nov'25 A1b).

4.5 Stream pattern [BOOK p.23 ↗](#) **IMPORTANT** SEE Feb'26 Q2a (8M) · CIE-1 Dec'24 Q3a "ingesting a stream and producing another stream" (5M) · CIE-1 Nov'25 A1a

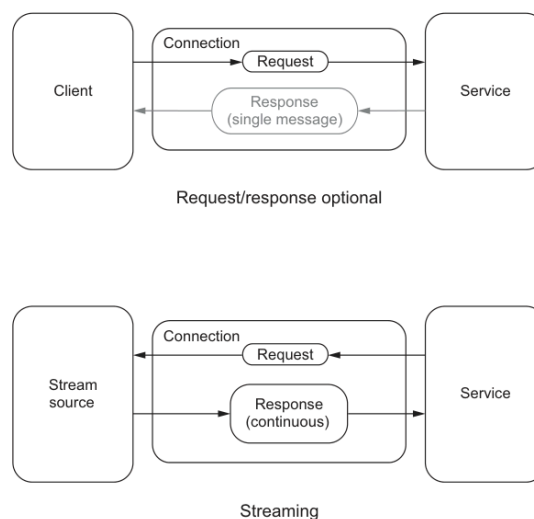


Figure 2.9 Comparing the request/response patterns to the stream pattern

Fig 2.9 Request/response optional vs streaming (p.23)

- The pattern is **flipped**: the service becomes the client. The collection tier **connects to a stream source and pulls data in**; one request → **continuous flow** of responses (vs 0 or 1 response).
- Request/response optional: an *external client pushes* messages to you. Stream: *you connect and consume* (e.g. connect to Twitter for sentiment analysis; Meetup RSVP JSON stream over one long-lived HTTP connection).
- **Ingest a stream, produce another stream**: build an analysis system that consumes a public stream and publishes a new stream derived from your analysis.
- Characteristics (PPT 43): asynchronous, multiple data chunks, non-blocking. Use for live updates, chat, video streaming; request/response for one-off API calls.

 HOW TO WRITE IT — SEE Feb'26 Q2a "streaming pattern with example" (8M)

Definition + flip-of-roles diagram (Fig 2.9 bottom) + 4 differences vs request/response + one example (Meetup RSVP stream / Twitter sentiment: collection tier connects, consumes JSON events, pushes to message queue) + scaling note (buffering layer, §5).

4.6 Compare patterns (Part B "describe and compare") **IMPORTANT** CIE-1 Nov'25 Q2b (5M)

Pattern	Response?	Sync?	Typical use / example	Reliability
Request/response	Yes, full	Sync (or half/full-async)	Web, REST, search	Client knows result
Request/acknowledge	ACK token only	Async	Order/job submit, propensity score	Receipt confirmed
Pub/sub	None to publisher	Async	Traffic topic, Kafka, Slack channels	Via broker guarantees
One-way	None	Async	Telemetry, RFID, logs	Lossy by design
Stream	Continuous	Async, non-blocking	Tweets, clickstream, live video	Needs buffering + logging

Choose (CIE-1 Nov'25 A1, solution): e-commerce clickstream → **Stream pattern** (continuous events); weather collector sending periodic updates with no reply → **One-way pattern**.

5 · Scaling the interaction patterns **BOOK p.25 ↗**

5.1 Request/response-optional patterns **BOOK p.26 ↗**

- Scale **horizontally** because the pattern is **stateless**: (1) a client can connect to **any** instance, (2) new instances can be added **without changing** running ones. Cloud **auto-scaling** adds/removes instances with demand.
- Scale of example: ~253 million US vehicles, 2.966 trillion miles (2012) ⇒ thousands–millions of simultaneous requests. A **load balancer** routes to collection nodes, which pass data to the message queuing tier.

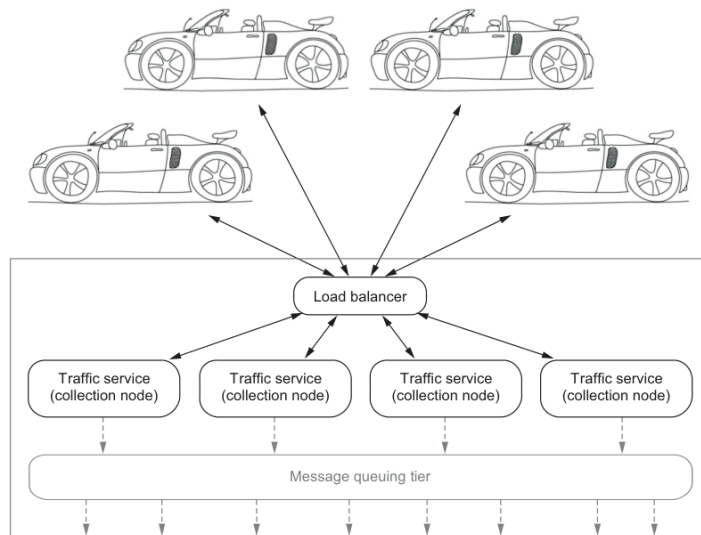


Figure 2.10 Vehicle and traffic service with a load balancer

Fig 2.10 Vehicle and traffic service with a load balancer (p.26)

5.2 Stream pattern [BOOK p.27](#)

- Problem: a stream connection is **direct and persistent** to *one* collection node $\Rightarrow$ the other nodes **sit idle** (46,000 Google search events/s example).
- Solutions (not exclusive): **(a) scale up** the consuming node (limited by hardware) and **(b) add a buffering layer**. Key rule: **no business logic** on messages read from the stream; push them to the buffer fast; a second set of collection nodes (that do the business logic) reads from the buffer and is **horizontally scalable**. [BOOK p.28](#)

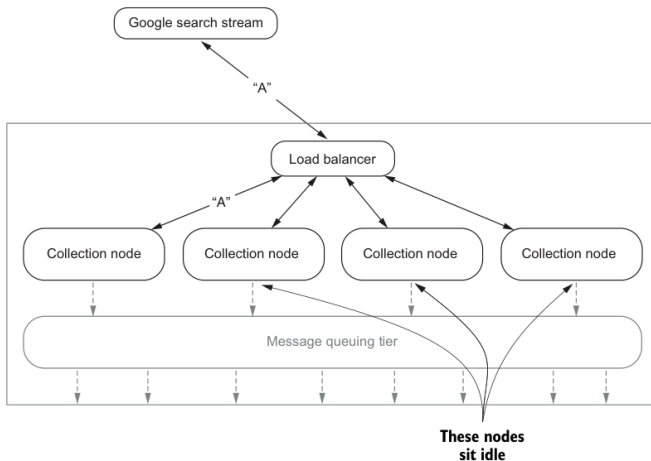


Figure 2.11 Search stream with direct connection to single collection node

Fig 2.11 Direct connection – idle nodes (p.27)

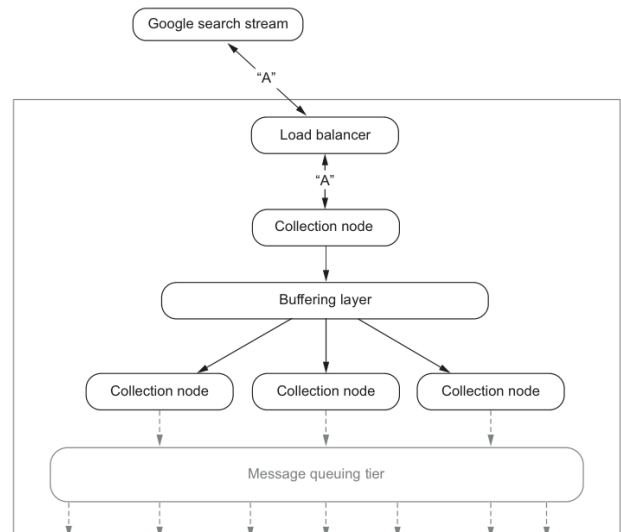


Figure 2.12 Collection tier with buffering layer in place

Fig 2.12 Buffering layer in place (p.28)

6 · Fault tolerance in the collection tier [BOOK p.28](#)

- Collection nodes **will** fail (bug, 3rd-party software, hardware). Redundancy alone is not enough because the **message from a client may not be reproducible** — the client can't resend. Goal: **mask failures, don't lose data, recover as if the crash never happened**.
- Two approaches: **Checkpointing** and **Logging**. [BOOK p.29](#)

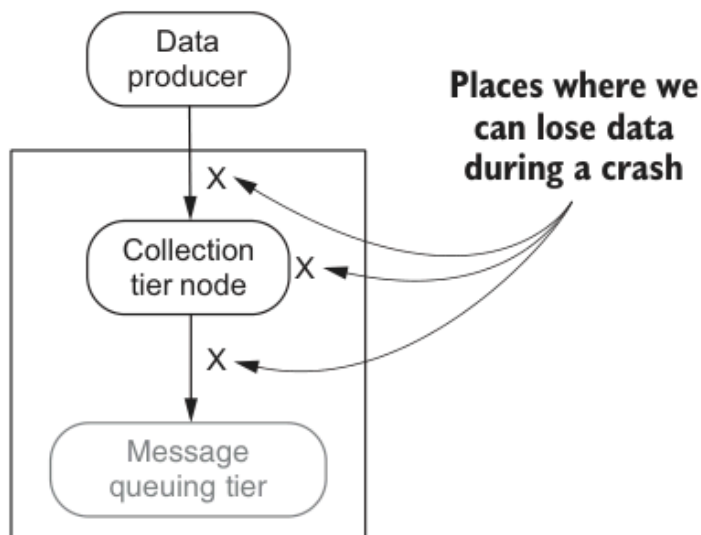


Figure 2.13 Collection scenario with our data-loss potential identified

Fig 2.13 Places where data can be lost during a crash (p.29)

6.1 Checkpointing **IMPORTANT** Model QP 1.2 · CIE-1 Nov'25 Q1a

- Regularly save a **global snapshot** of the **whole system** (collection → data access), not just one tier, to durable storage. On recovery, restore the last snapshot.
- Potential for data loss:** only recoverable up to the last snapshot; messages processed afterwards are lost. Analogy: MS Word/Google Docs autosave.
- Hard to take a consistent global snapshot across many layers/technologies ⇒ book calls it **a bad choice for a streaming system** (OK for building HDFS/NoSQL). [BOOK p.30 ↗](#)

6.2 Logging protocols

- Aim: avoid checkpoint expense/complexity and **recover up to the last message received**. Idea: **if a message can be replayed, the system reaches a consistent state without a global snapshot**. Each tier independently logs the messages it receives and replays after a crash. [BOOK p.30 ↗](#)

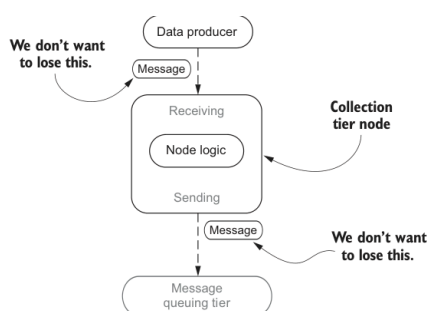


Figure 2.14 High-level overview of receiver-based and sender-based message logging

Fig 2.14 RBML protects incoming, SBML protects outgoing (p.31)

	RBML — receiver-based BOOK p.31 ↗	SBML — sender-based BOOK p.33 ↗	HML — hybrid BOOK p.34 ↗
What is logged	Every received message , synchronously to stable storage, before any processing (raw input)	Every outgoing message after node logic, before sending to next tier	RBML + SBML combined
Protects against	Crash of this node while handling the message ("protect ourselves from ourselves")	Next tier crashing / network interruption	Both
Recovery	Take node out of load-balancer rotation → RBML logger replays unprocessed messages through node logic → put back in rotation	Replay logged messages; need to know if next tier already got them ⇒ ACK from queuing tier, then delete/mark the logged message	Same protection, less overhead
Cost	Extra write per message (throughput hit)	Extra write + ACK handling	One consolidated stable storage + asynchronous writes ; reported 43% overhead reduction vs RBML (Meyer et al.)

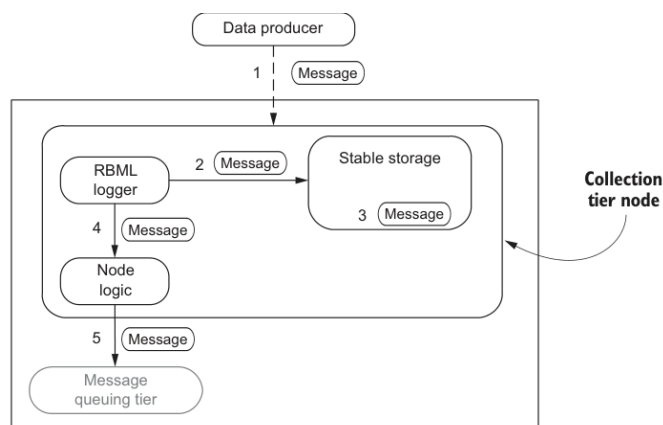


Figure 2.15 RBML implemented for the simple collection node showing the happy path

Fig 2.15 RBML happy path: 1 message → 2 logger → 3 stable storage → 4 node logic → 5 queue (p.32)

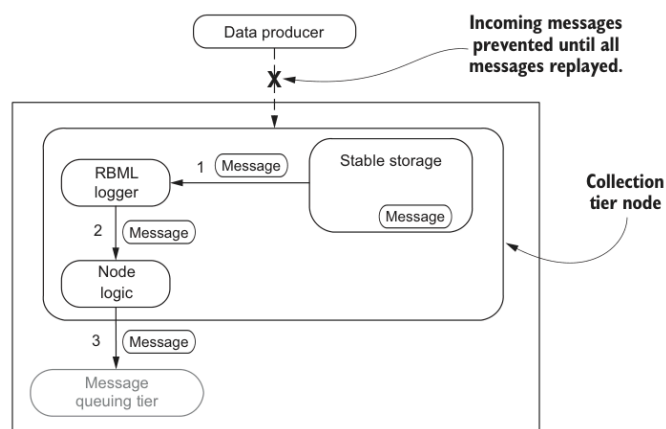


Figure 2.16 The recovery data flow for RBML

Fig 2.16 RBML recovery (p.32)

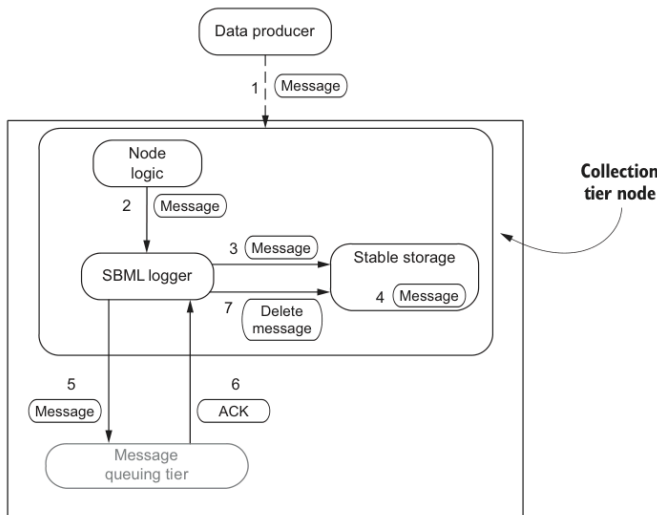


Figure 2.17 The normal execution data flow for SBML

Fig 2.17 SBML normal flow, steps 1–7 incl. ACK + delete (p.33)

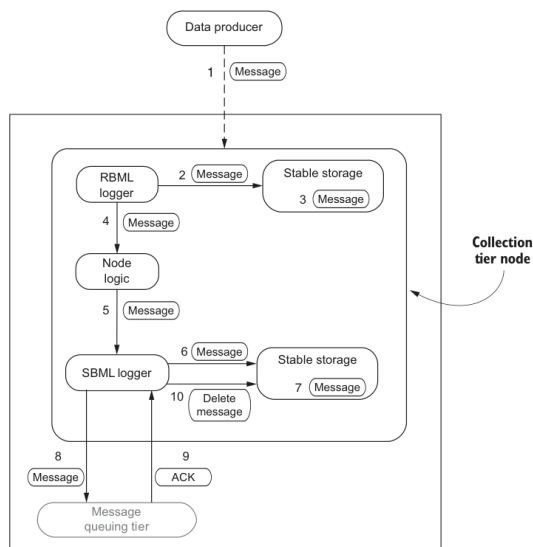


Figure 2.19 RBML and SBML together in the collection tier node

Fig 2.19 RBML + SBML together – 10 steps (p.35)

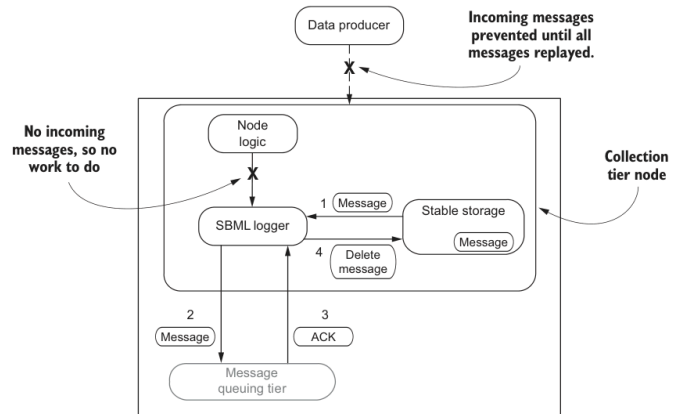


Figure 2.18 Recovery data flow for SBML

Fig 2.18 SBML recovery (p.34)

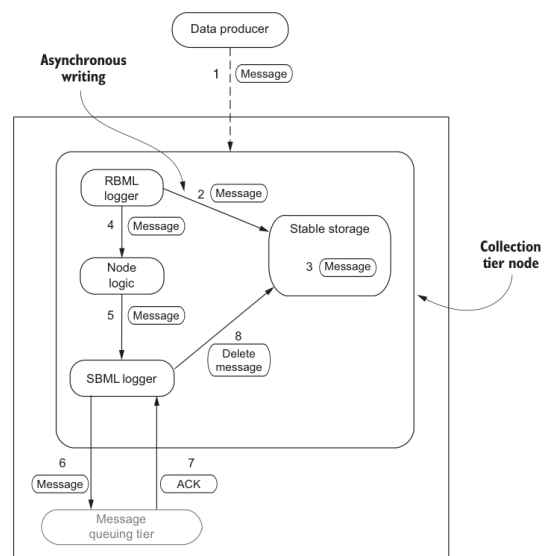


Figure 2.20 HML sample data flow

Fig 2.20 HML sample flow, asynchronous writing (p.36)

🧠 **MNEMONIC** — RBML = "Received → Back it up first"; SBML = "Sent → Back it up first"

RBML logs at the **front door**, SBML logs at the **back door**.

⚠️ **CONFLICT / TRAP** — Which logging technique? — read the scenario carefully

CIE-1 Nov'25 A3 (node loses messages after a *power failure*) — official solution: **RBML**: it saves every received message before processing, so a crashing collector can replay all of them.

SEE Feb'26 1.3 (a *sensor gateway crashes* but all upstream sensor nodes stay up **and can resend**) — no scheme seen. The upstream nodes hold the messages, so the matching technique is **SBML** (senders log before sending and replay to the crashed receiver). This is our reading of the question; if you prefer to hedge, add "RBML would also work at the gateway if senders could not resend".

Rule: **the node itself may lose input** → **RBML**; **the next tier/receiver crashes and the sender holds copies** → **SBML**; **want both with low overhead** → **HML**.

⚠ CONFLICT / TRAP — Checkpointing — book vs solution scheme

The textbook (p.30) says checkpointing is a **bad fit** for streaming systems (global snapshot is hard, data loss after last snapshot). The CIE-1 Nov'25 solution (from the faculty slides) says "*checkpointing and logging jointly ensure reliable recovery: checkpoint restores state, log replays lost data*" and cites Flink checkpoints to S3. **Write both:** state the two characteristics (global snapshot, possible data loss) and then, if the question says "jointly", describe *restore last checkpoint + replay logged messages after it*. Slide 54 also lists these two characteristics.

6.3 A dose of reality (BOOK p.37 ↗)

Book anecdote: dashboards over WebSockets kept up with only ~60% of the stream and users still loved them — don't over-engineer; reflect on "must-have" vs reality.

7 · Quick revision — Unit 1

Three classes of real-time system?	Hard, soft, near.
Hard real-time example?	Pacemaker, anti-lock brakes (μ s–ms, failure = disaster).
Streaming system in one line?	Non-hard real-time service whose clients consume data when needed.
Five interaction patterns?	Req/resp, req/ack, pub/sub, one-way, stream.
Which pattern is "fire and forget"?	One-way.
Which pattern returns a token?	Request/acknowledge.
Stream pattern flips what?	Service becomes the client — collection tier pulls from the stream source.
Why is a buffering layer needed for stream pattern?	Direct connection leaves other collection nodes idle; buffer decouples and lets nodes scale out.
Why scale horizontally?	Stateless, no ceiling, auto-scaling.
Checkpoint = ?	Periodic global snapshot; loses data after last snapshot.
RBML / SBML logs when?	RBML: on receipt, before processing. SBML: before sending to next tier.
HML advantage?	Same safety, less overhead (one storage, async writes, –43%).

8 · Worked answer — streaming architecture for a scenario

IMPORTANT CIE-1 Dec'24 Q1a (10M) · CIE-1 Nov'25 Q1a (10M) · Model QP Q2a (10M)

Scenario (CIE-1 Dec'24): Vehicle tracking — monitor location, speed, fuel of a fleet, alert on anomalies.

1. **Collection tier:** on-board GPS/OBD units send location, speed, fuel every few seconds (**one-way/stream pattern**, behind a **load balancer**, stateless nodes; RBML logging so no reading is lost). Authenticated (TLS + device credentials).
2. **Message queuing tier:** Kafka-like broker with a *vehicle-events* topic, **durable**, partitioned by vehicle ID; decouples spikes from analysis.

3. **Analysis tier:** stream processor (Storm/Flink/Spark) runs **continuous queries**: speed > limit, fuel drop rate, geofence exit → raises anomaly alerts; sliding window for averages.
4. **In-memory data store:** latest position/status per vehicle (Redis/IMDB) for the live map.
5. **Data access tier:** API/WebSocket/pub-sub pushing live location + alerts to dashboards and the driver app.
6. **Long-term storage:** trips and alert history in HDFS/S3 for reports.
7. **Cross-cutting:** security (authN/authZ each tier), horizontal scaling per tier, fault tolerance (replicated brokers, logging).

For the IoT environmental network (CIE-1 Nov'25 Q1a, solution): Sensors → edge gateways → broker (Kafka/Pulsar) → Flink/Spark → hot store (Redis)/cold store (S3/HDFS) → dashboards/alerts; pattern = **pub/sub + stream**; scaling = **horizontal** (vertical only for stateful nodes); fault tolerance = replication + checkpoints + message logging; security = TLS/mTLS, device auth, RBAC.

Unit 2 — Data Transportation & Analysing Streaming Data

Syllabus: message queue · core concepts · security · applying core concepts to business logic · in-flight data analysis · distributed stream-processing architecture · key features of stream-processing frameworks

Sources: Psaltis *Streaming Data* Ch.3 (printed pp.38–57) and Ch.4 (pp.58–76) + all question papers. **BOOK p.xx** ↗ opens *Streaming-Data[1].pdf* at printed page xx (PDF page = printed + 20). Blue **highlight** = must-remember. **IMPORTANT** = asked before.

0 · Hot-list for Unit 2 (Q3/Q4 — choose one, 16 marks)

Rank	Topic	Asked in
1	Need for message queuing tier	Model QP 1.3 & 3a · CIE-1 Dec'24 2b · CIE-2 Dec'25 1a
2	Producer–broker–consumer diagram & working	SEE Feb'26 4b (6M) · CIE-2 Dec'25 2a (10M)
3	Durable messaging / offline consumers / replay scenario	CIE-2 Dec'25 1b · CIE-1 Dec'24 5b · Model QP 3b
4	Delivery semantics (+ exactly-once)	CIE-1 Dec'24 A5 · CIE-2 Dec'25 A1 · Model QP 4b · SEE Feb'26 3a(iv), 1.5
5	Broker crash / fault tolerance in MQ tier	CIE-1 Dec'24 A4 · CIE-2 Dec'25 A2 · CIE-1 Nov'25 3a · SEE Feb'26 3b
6	Points of failure in stream processing (7 points)	Model QP 4a · CIE-2 Jan'25 3a · SEE Feb'26 8b · CIE-1 Dec'24 4a · CIE-1 Nov'25 3a
7	Stream-processing frameworks (Storm, Samza, Flink, Spark) & architecture	Model QP 1.4 · CIE-2 Jan'25 A1,A2,2a · CIE-3 Jan'26 A1,1a,4 · SEE Feb'26 1.4 (Flink), 4a (Samza)
8	State management; state-machine vs rollback recovery	CIE-3 Jan'26 2a,3b · CIE-2 Jan'25 A3
9	Traditional DBMS vs streaming (continuous query)	CIE-2 Jan'25 1a · CIE-3 Jan'26 A2
10	Scenario tables (Coke machines, traffic replay, fraud)	SEE Feb'26 3a · Model QP 3b

1 · Why we need a message queuing tier BOOK p.39 ↗ IMPORTANT Model QP

1.3, 3a (10M) · CIE-1 Dec'24 2b (4M) · CIE-2 Dec'25 1a (6M)

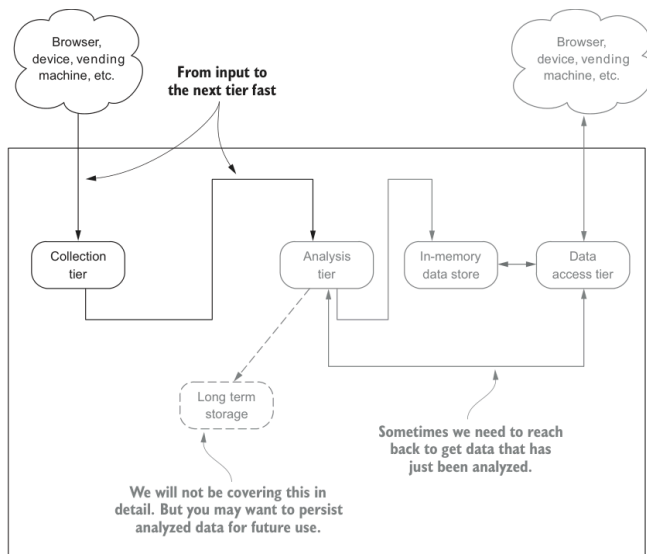


Figure 3.2 From the collection tier straight to the analysis tier

Fig 3.2 Architecture WITHOUT the queue tier (p.40)

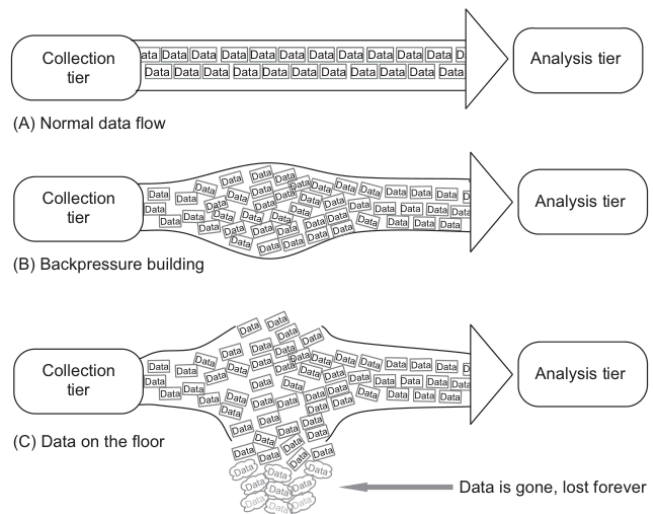


Figure 3.6 The three stages of data flowing without a message queue. We don't want step C.

Fig 3.6 (A) normal → (B) back-pressure builds → (C) data on the floor (p.43)

- Without it, collection calls analysis directly. A single machine may work, but real systems span machines: we want to **decouple the tiers** (and components in each tier). The queue lets tiers work at a higher level of abstraction — pass messages, no explicit calls to the next layer. BOOK p.40 ↗
- Needs / benefits (write 5 points):**
 - Decoupling** of collection from analysis (independent scale, deploy, fail).
 - Buffering / absorbing surges** — producer faster than consumer causes back-pressure; without a queue data is dropped "on the floor" (Fig 3.6C). BOOK p.43 ↗
 - Offline / slow consumers** (e.g. hourly batch readers) — needs durable messaging. BOOK p.44 ↗
 - Durability & replay** / disaster recovery (data-centre link cut).
 - Delivery guarantees, ordering, flow control, fault tolerance** (CIE-2 Dec'25 solution adds FIFO ordering, back-pressure management).
- Example: collection in San Diego, analysis in Amsterdam; fibre cut for 2–3 days — a durable queue keeps the messages (Figs 3.7–3.9). BOOK p.44 ↗

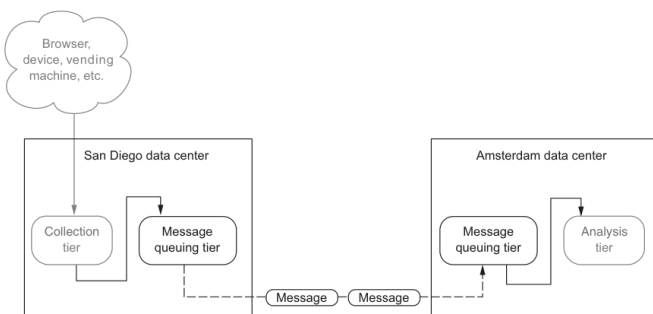


Figure 3.7 Two data centres with data flowing between them

Fig 3.7 Two data centres (p.44)

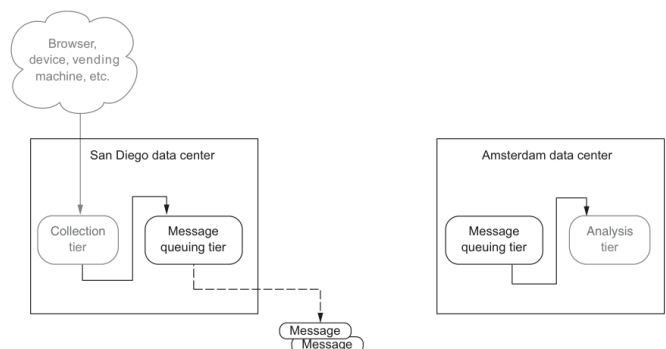


Figure 3.8 Two data centers with data flowing into the ocean

Fig 3.8 Link cut – data flows into the ocean (p.45)

🔥 HOW TO WRITE IT — Model QP 3a (10M) "elaborate the need for message queuing tier with example"

Intro (Fig 3.1/3.2 without queue) → Fig 3.6 back-pressure story → 5 needs above → San Diego/Amsterdam example → conclusion: "the queue tier lets the collection tier and analysis tier run at their own speed and fail independently".

2 · Core concepts [BOOK p.40 ↗](#)

"Message queuing" is used broadly: RabbitMQ, ActiveMQ, HornetQ (traditional) to NSQ, ZeroMQ and [Apache Kafka](#) (publish/subscribe to streams of records).

2.1 Producer, broker, consumer [BOOK p.41 ↗](#) **IMPORTANT** SEE Feb'26 4b (6M) · CIE-2 Dec'25 2a (10M)

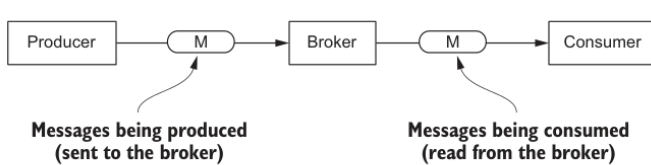


Figure 3.3 The three core parts to a message queuing system

Fig 3.3 The three parts (p.41)

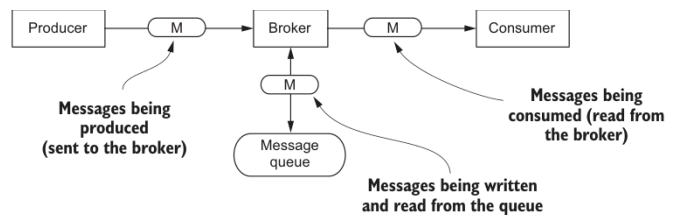


Figure 3.4 The broker with the message queue being shown

Fig 3.4 Broker with the queue shown (p.42)

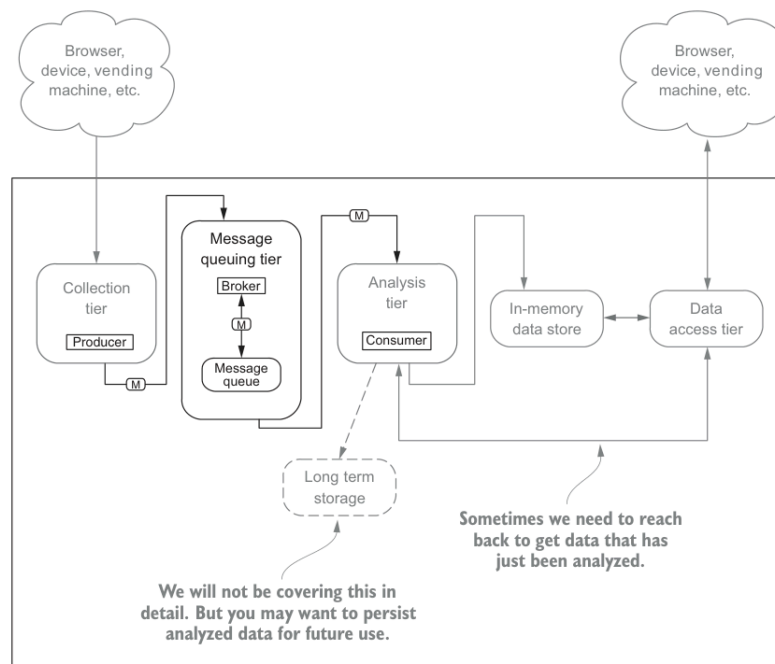


Figure 3.5 Streaming architecture with message queuing tier components shown in context

Fig 3.5 Queue tier components mapped onto the streaming architecture (p.42) — DRAW THIS

1. **Producer** (usually the collection tier) **sends a message to a broker**.
2. **Broker** **puts the message into a queue**. "Broker" not "queue" because the broker is an **abstraction** that may manage **multiple queues**.
3. **Consumer** (usually the analysis tier) **reads the message from the broker**.

🔥 HOW TO WRITE IT — 6M / 10M "working of producer, broker and consumer"

Fig 3.5 + the 3 steps + role of each + then add: durability (queue persists), delivery semantics (ack), decoupling and where each maps (producer = collection tier, consumer = analysis tier). For 10 marks add the failure points of Fig 3.12 (§4) and security of Fig 3.14.

2.2 Isolating producers from consumers [BOOK p.43 ↗](#)

- If the producer outruns the consumer (analysis is heavier than collection) pressure builds "like a plugged hose" → data lost (Fig 3.6). It is **not a consumer problem**: consumers may legitimately read slowly or be offline (hourly batch).
- Not all queue products offer producer flow control — then you must throttle the collection tier yourself. Durable messaging is what supports slow/offline consumers.

2.3 Durable messaging [BOOK p.44 ↗](#) **IMPORTANT** CIE-2 Dec'25 1b (4M) · CIE-1 Dec'24 5b · Model QP 3b

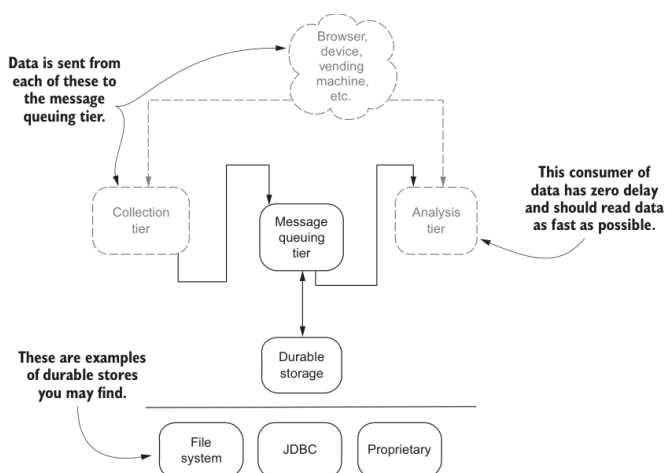


Figure 3.9 Durable messages—where they fit and how they may be stored

Fig 3.9 Durable messages and how they may be stored: JDBC, file system, proprietary (p.46)

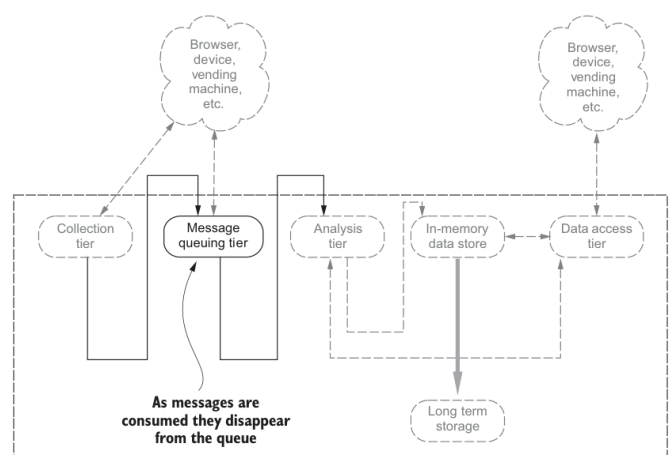


Figure 3.10 Transient messages get discarded after the analysis tier consumes them.

Fig 3.10 Transient messages vanish after consumption (p.46)

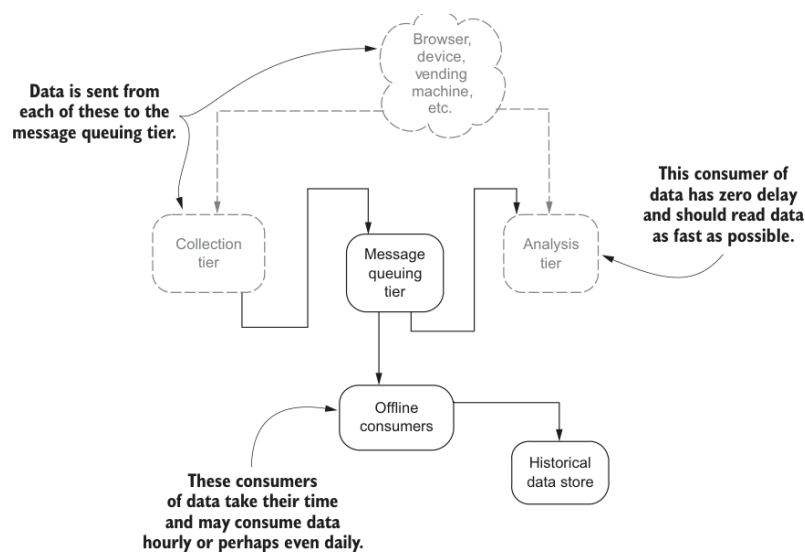


Figure 3.11 Offline consumers persisting data for historical reporting/analysis

Fig 3.11 Offline consumers persist data for historical reporting (p.47)

- **Durable** = messages are **persisted** (database via JDBC, file system, proprietary store) and retained until consumed (or longer), so they survive broker crash, outage and slow/offline consumers. **Transient** = deleted once consumed.

- **Key aspects (CIE-2 Dec'25 scheme):** persistence · message queues marked durable (RabbitMQ, Kafka, JMS) · fault tolerance/resilience · acknowledgements before deletion.
- **Traffic-replay scenario (Model QP 3b / CIE-1 Dec'24 5b style):** app re-routes drivers in real time; 3 months later business wants a *historical traffic-replay product*. With transient queue (Fig 3.10) consumed messages are gone ⇒ impossible. **Solution = Fig 3.11:** durable queue + **offline consumer** that reads at its own pace and writes to a **historical data store**; replay by re-reading retained messages for city/day/week/month. [BOOK p.45 ↗](#)

🗨️ **MNEMONIC — "DOR" — Durable = Outage-proof, Offline consumers, Replay**

2.4 Message delivery semantics [BOOK p.47 ↗](#) **IMPORTANT** CIE-1 Dec'24 A5 · CIE-2 Dec'25 A1 · Model

QP 4b · SEE Feb'26 3a(iv)

Semantics	Meaning (queue tier, Ch.3)	Complexity / speed
At-most-once	May get lost , never re-read	Least complexity, fastest, weakest
At-least-once	Never lost, may be re-read (duplicates)	Medium; consumer must tolerate duplicates
Exactly-once	Never lost, read once and only once	Most complexity, performance hit, strongest (Table 3.1)

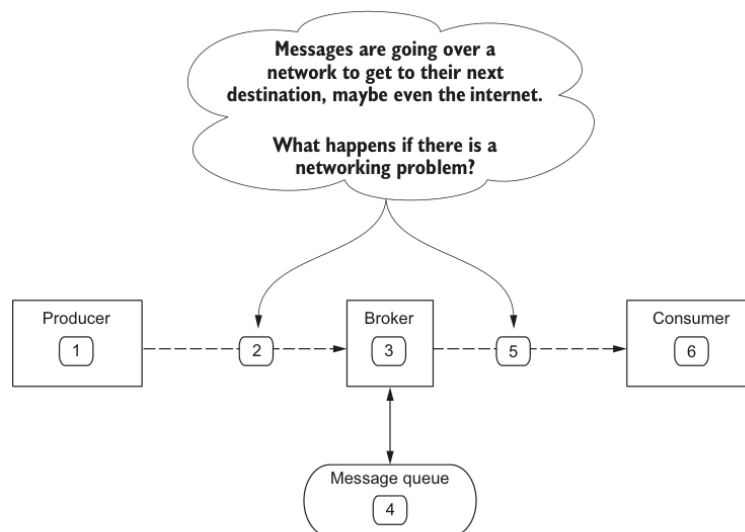


Figure 3.12 Possible points of failure that need to be considered

Fig 3.12 Possible points of failure (p.48)

Six failure points in the queue path (Fig 3.12) [BOOK p.48 ↗](#)

1. **Producer** fails after producing but before sending ⇒ message lost; fails waiting for ack ⇒ resends ⇒ duplicate.
2. **Network producer→broker**: message or ack lost ⇒ producer may resend ⇒ duplicate.
3. **Broker**: crash with messages only in memory ⇒ loss; fails before ack ⇒ resend; fails before committing consumer-read info ⇒ consumer re-reads.
4. **Message queue** (persistent store) fails to write to disk ⇒ loss.
5. **Network broker→consumer**: message sent but not received, or ack lost ⇒ message sent again.

6. **Consumer** fails before recording it processed $\Rightarrow$ re-requests the same message (also duplicates with multiple consumers).

Getting exactly-once when the product doesn't give it [BOOK p.49](#)

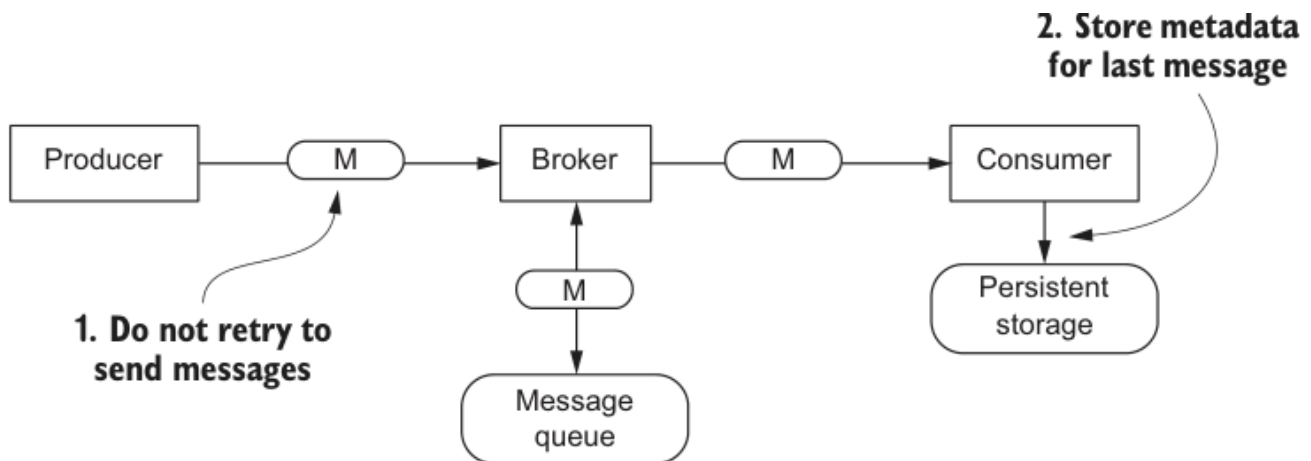


Figure 3.13 The two ways to have exactly-once semantics if the messaging system doesn't provide it

Fig 3.13 Two ways to have exactly-once semantics (p.50)

1. **Do not retry to send messages** — track what the producer sent; if no response, **read the broker to verify** whether the message arrived instead of blindly resending.
2. **Store metadata for the last message** read (JMS: *JMSMessageID*; Kafka: the **offset**) in persistent storage so the consumer never reprocesses.

Bonuses [BOOK p.50](#): **message auditing** (track sent/arrived counts, max/min/avg processing time, find slow producer/consumer) and **duplicate detection** (check metadata/payload identity). **IMPORTANT** SEE Feb'26 1.5: "How do message auditing and duplicate detection improve data quality?" $\rightarrow$ auditing shows lost/late/slow messages; duplicate detection removes repeats $\Rightarrow$ complete, unique, trustworthy data.

⚠ **CONFLICT / TRAP — Does Kafka give exactly-once?**

The textbook (Ch.3, p.49) says products like **Apache Kafka and ActiveMQ do not provide exactly-once** (written 2017). The Kafka book used for Unit 5 (2nd ed., 2021, Ch.8/14) says Kafka supports exactly-once with an **idempotent producer + transactions**, and Kafka Streams enables it with `processing.guarantee=exactly_once`. **Write:** for a generic queue question use the two-technique method above; if the question is about Kafka, say "at-least-once by default; exactly-once via idempotence + transactions".

⚠ **CONFLICT / TRAP — "Six" vs "seven" points of failure**

Ch.3 / Fig 3.12 lists **6** failure points in *message queuing* (producer, 2 networks, broker, queue, consumer). **Ch.4 / Fig 4.15** lists **7** points in *stream processing* (§7). Model QP 4a says "seven points of failure in stream processing data flow" $\Rightarrow$ use the Ch.4 seven. If the question says "queue/delivery semantics failure scenarios" use the six.

🔥 HOW TO WRITE IT — Model QP 4b — "apply delivery semantics to the traffic-replay system, with a neat diagram"

Navigation app + historical replay ⇒ no message may be lost (history must be complete) but duplicates could skew counts ⇒ choose **exactly-once** (or at-least-once + de-duplication). Draw Fig 3.13: producer (no blind retry, verify with broker) → broker/queue → consumer storing offset/metadata in persistent store; mention auditing + duplicate detection bonuses and the trade-off (complexity vs guarantee, Table 3.1).

3 · Security of the queue tier [BOOK p.51](#)

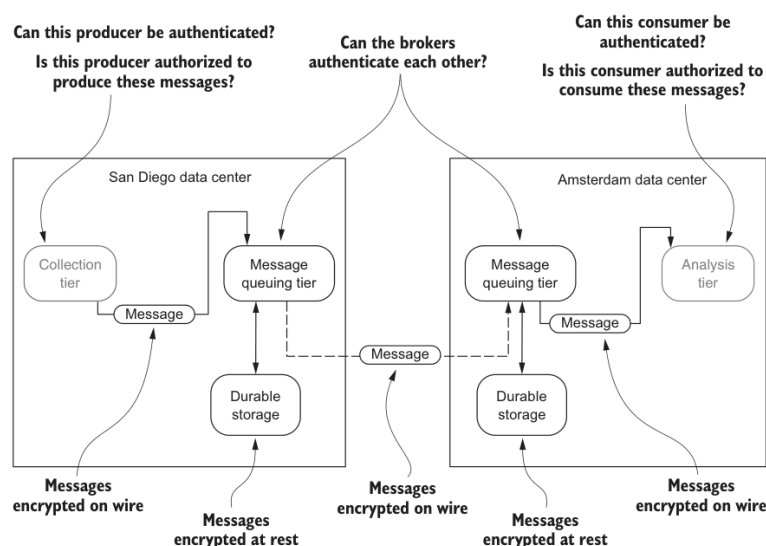


Figure 3.14 Security points that need to be considered for the message queuing tier

Fig 3.14 Security points for the message queuing tier (p.51)

- Messages **encrypted on the wire** (between tiers, brokers, data centres) and **encrypted at rest** (durable storage).
- **Authenticate + authorize producers** ("can this producer be authenticated? may it produce these messages?"), **authenticate + authorize consumers**, and **brokers must authenticate each other**.

4 · Fault tolerance of the queue tier

BOOK p.52

IMPORTANT

SEE Feb'26 3b (8M)

CIE-2 Dec'25 A2 · CIE-1 Dec'24 A4 · CIE-1 Nov'25 3a

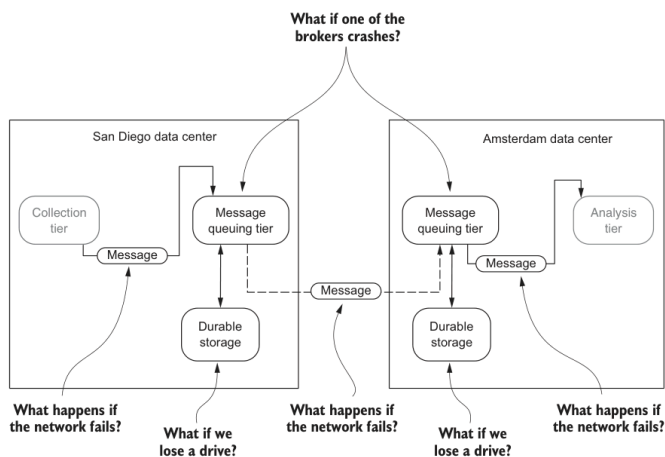


Figure 3.15 A high-level view of where things can go wrong

Fig 3.15 High-level where things can go wrong (p.52)

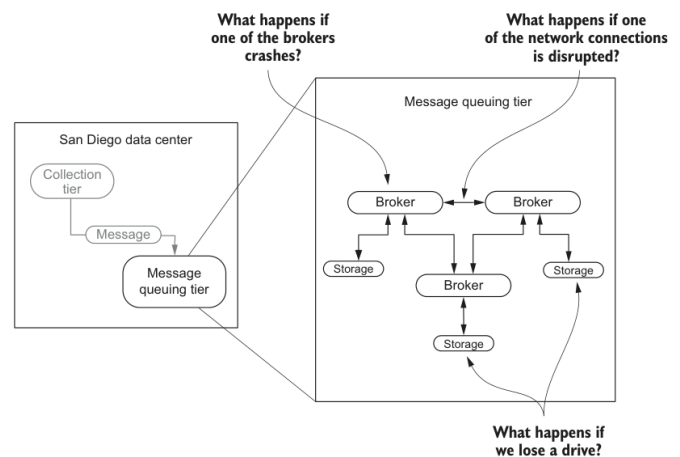


Figure 3.16 Zoomed in view of brokers and what can go wrong

Fig 3.16 Zoom on brokers (p.53)

Failure	Impact on reliability	Mitigation (book p.53–54)
Broker crashes	With durable storage only in-memory messages at risk; messages unacknowledged are lost / re-sent	(1) producer waits for ack that data was written to disk; (2) replicate to > 1 broker (two must crash together); (3) keep as little in memory as possible (trade performance for durability)
Network between brokers is interrupted	With replication data is safe on other brokers; broker later rejoins and synchronises missed messages . Risk: producer mid-send to the severed broker	Ask: is a new replica chosen? what if connection restored? configurable delay before "down"? what if never restored?
Storage drive fails	Potentially drastic; data in flight/not yet on disk lost	Replicas of the lost data? is un-flushed replicating data lost? how to recover the broker?

Also covered by chapter 2 lessons: producer handles network failures; durable store mitigates inter-data-centre link loss.

5 · Applying the concepts to business problems

BOOK p.54 ↗

IMPORTANT

SEE Feb'26 3a (8M)

Question (Tables 3.2–3.4)	Fraud detection (Paul)	Tweeting Coke machines (Frank) — SEE Feb'26 3a	E-commerce recommendations (Rex)
Impact if collection↔analysis link down for long?	Catastrophic — service can't run	Lose money/offers; more if used for inventory management	Lost up-sell/cross-sell, lost sales
Days of data tolerable to lose?	Zero	Depends on inventory handling & machine popularity (locale-specific)	Maybe batch recommender as backup; freshness issue
Need historical data?	Yes — performance reports	Perhaps — reporting on machine performance	Yes — reporting, algorithm refinement
Delivery semantics?	Exactly-once (at-least-once possible, more consumer work)	At-least-once suffices (at-most-once arguable)	At-least-once (exactly-once overkill)

🔥 HOW TO WRITE IT — SEE Feb'26 3a — answer the 4 sub-questions in 2 marks each

Use the Frank column: (i) revenue lost / wrong offers / stale stock recommendation, (ii) tolerance small-to-moderate and locale-specific, (iii) yes for reporting/inventory trends, (iv) **at-least-once** with idempotent consumers, justified because a duplicate tweet/offer is harmless but a missed one costs a sale; use durable, replicated broker.

6 · Analysing streaming data — in-flight analysis

BOOK p.59 ↗

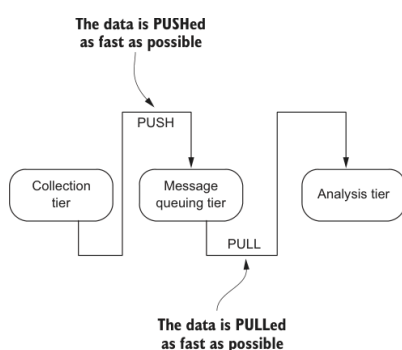


Figure 4.2 Data being pushed from the collection tier and pulled from the analysis tier

Fig 4.2 Push into queue, pull by analysis (p.60)

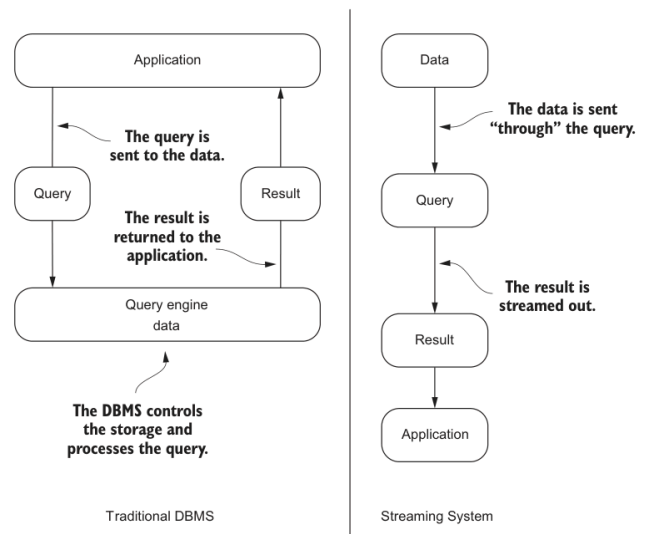


Figure 4.3 Turning things on their head: non-streaming versus streaming

Fig 4.3 Traditional DBMS vs streaming system (p.61)

- **In-flight data** = all tuples in the system from input (queue tier) to output (client): **always in motion, never at rest** (never persisted to durable storage).
- **Continuous query model**: instead of querying data at rest, **data is moved through the query**; a user registers a query once, it runs each time data arrives or at an interval, results are **pushed**. Example: trending article / broken link — 3 steps (collect, start query, act) vs 5 in a DBMS (gather, load, query, act, repeat every x hours). ([BOOK p.60 ↗](#))
- Inverts the model: DBMS user = **active**, system **passive**; streaming system = **active**, user **passive**.

Table 4.1 — DBMS vs streaming system ([BOOK p.62 ↗](#)) **IMPORTANT** CIE-2 Jan'25 1a (6M) · CIE-3 Jan'26

A2

	Traditional DBMS	Streaming system
Query model	One-time query on consistent state; answer returned, query forgotten; pull	Continuous query on flowing data; registered once, results regularly pushed
Changing data	During downtime data can't change	Data keeps arriving while analysis tier is down ⇒ catch-up after crash
Query state	Crash ⇒ query forgotten; user re-issues it	Registered queries may or may not resume where they left off — often as if never stopped
Data set	Static, loaded before query	Unbounded, never ends

Use-case categories (p.62): tracking behaviour (McDonald's NL: +700% offer redemptions), traffic safety/efficiency (EU congestion ~€100 bn/yr), real-time fraud analytics (US card fraud losses down 70% as a % of sales).

7 · Distributed stream-processing architecture ([BOOK p.63 ↗](#)) **IMPORTANT**

CIE-3 Jan'26 1a (10M) · Model QP 1.4 · CIE-2 Jan'25 A1

Need: volume/velocity exceeds one machine (a GE gas turbine makes ~1 TB/hour) ⇒ distributed analysis tier. Popular open-source engines: **Spark Streaming, Storm, Flink, Samza** (all Apache). All share three parts: a component the application is **submitted** to, **separate nodes** that run the algorithm, and **data sources** as input.

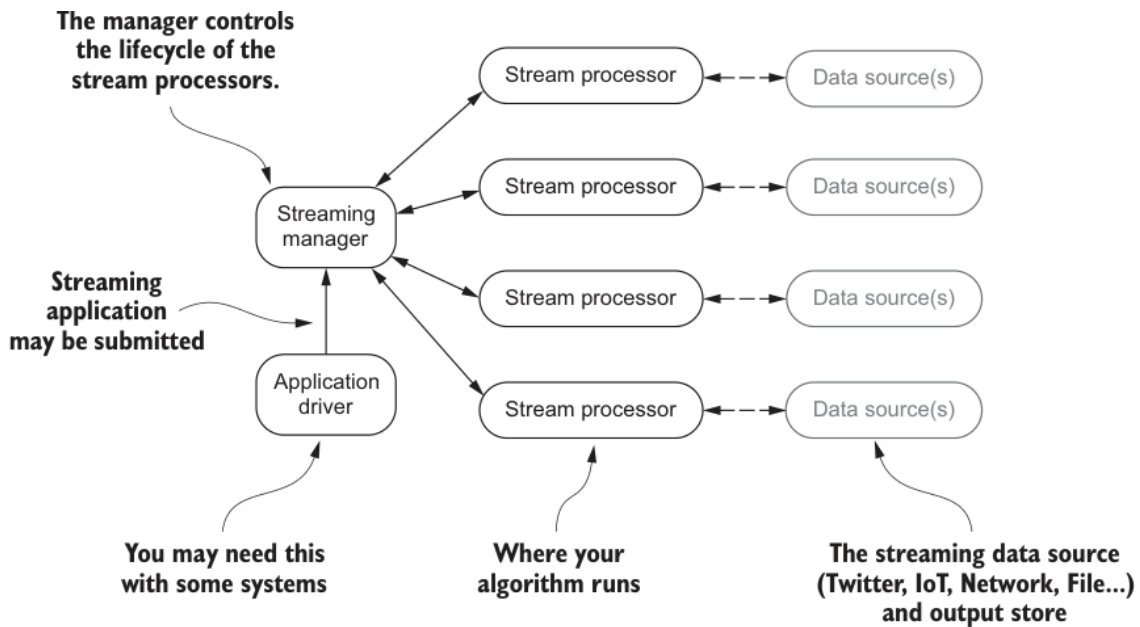


Figure 4.4 Generic streaming analysis architecture you will find with many products on the market

Fig 4.4 Generic streaming analysis architecture (p.64) — DRAW THIS

Component	Role
Application driver	Client code that defines the streaming program and talks to the manager (Spark: driver submits the job, may collect results, controls job lifetime). Needed in some systems only.
Streaming manager	Gets the job to the stream processors; controls their lifecycle; may request resources.
Stream processor	Where the job runs ("rubber meets the road").
Data source(s)	Input (and possibly output store) — Twitter, IoT, network, files. Some platforms ingest many sources per job, others one.

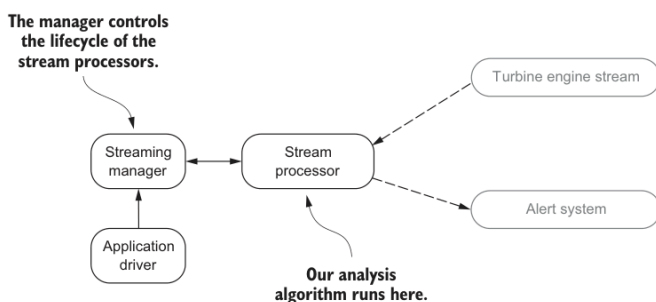


Figure 4.5 Turbine engine monitoring on our common architecture

Fig 4.5 Turbine monitoring mapped to the common architecture (p.65)

7.1 The four frameworks BOOK p.65 ↗

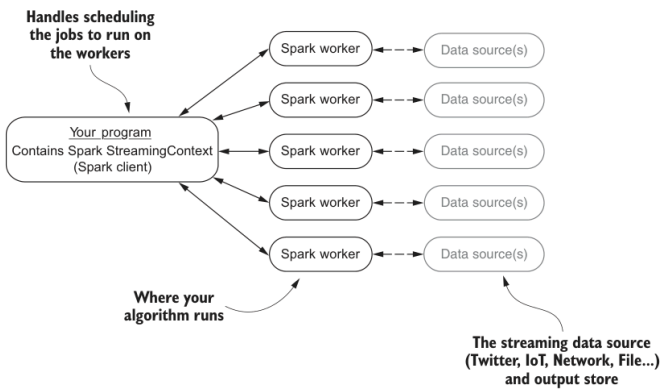


Figure 4.7 Spark Streaming's high-level architecture

Fig 4.7 Spark Streaming (p.66)

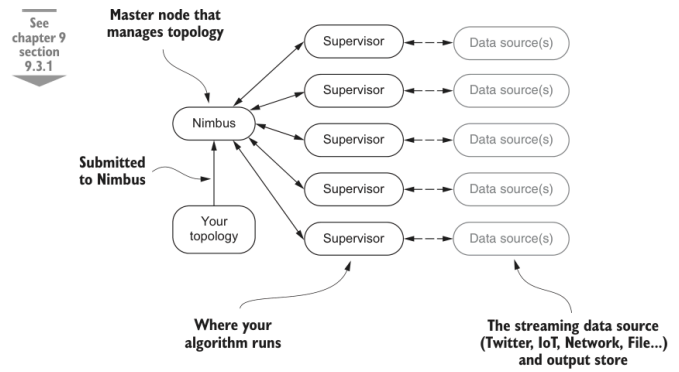


Figure 4.8 High-level overview of Storm architecture showing Nimbus, supervisors, and a data source

Fig 4.8 Storm (p.67) – SEE Q1.4 asks Flink, Model QP Q1.4 asks Storm

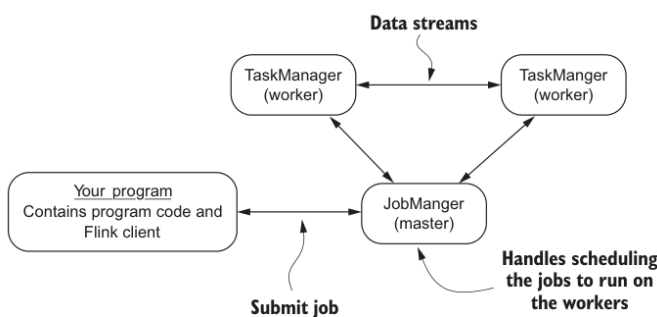


Figure 4.9 High-level Apache Flink architecture

Fig 4.9 Flink (p.68) — SEE Feb'26 Q1.4 "diagrammatically represent Flink"

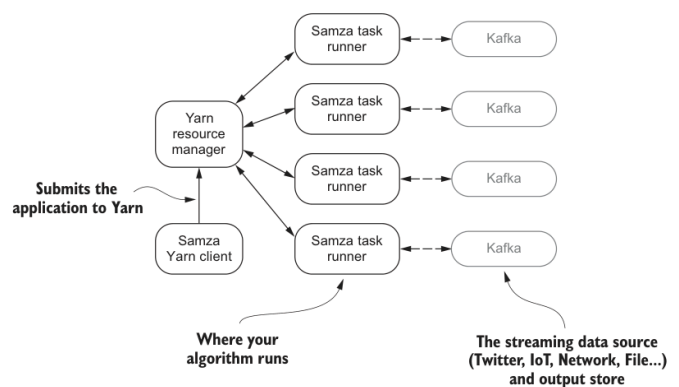


Figure 4.10 High-level Apache Samza architecture

Fig 4.10 Samza (p.68)

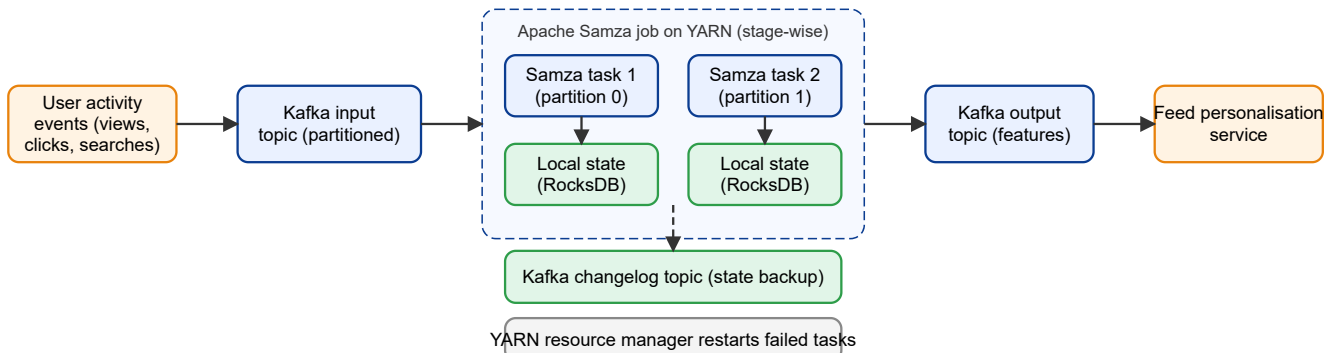
	Spark Streaming	Storm	Flink	Samza
Model	Micro-batches (jobs over short time slices, even <0.5 s)	Tuple-at-a-time	Stream-first : everything is a stream; streams + transformations; dataflow source→transform→sink	Stage-wise stream processing
Unit of work	Job	Topology (runs forever)	Streaming dataflow / job	Samza job/tasks
Master / worker	Driver with StreamingContext → Spark workers	Nimbus (master: deploys topology, assigns tasks, monitors failures) → Supervisors	JobManager (master) → TaskManagers (workers); Flink client submits job	YARN resource manager + Samza task runners ; Samza YARN client submits
Built on / needs	Apache Spark (MLlib, GraphX, SparkSQL; Java/Scala/Python/R)	—	—	Apache YARN (resources, scheduling) + Apache Kafka (all input/output)

🧠 **MNEMONIC — "S-S-F-S": Spark = Small batches, Storm = Single tuple, Flink = Stream-first, Samza = Stage-wise (YARN+Kafka)**

7.2 Samza scenario — professional networking platform (SEE Feb'26 4a, 10M)

IMPORTANT

Question: process millions of user-activity events/sec (profile views, clicks, job searches, feed interactions) in near real time; goals low-latency personalisation, high throughput + scalability, strong fault tolerance + state consistency. "Implement Apache Samza architecture, draw diagram, explain steps." (Answer built from Ch.4 Samza text + Ch.4 state/fault-tolerance features; the LinkedIn scenario itself is not in the textbook.)



Samza architecture for the activity-feed scenario (redrawn — our diagram)

1. Activity events are written to **Kafka** (input topics partitioned by member ID) — Samza reads **and writes all data through Kafka**.
2. Samza job submitted by the **Samza YARN client** to the **YARN resource manager**, which allocates containers and starts **Samza task runners** (one task per input partition) ⇒ **horizontal scalability** and high throughput.
3. Each task keeps **local state** (e.g. member's recent views/clicks) for low-latency personalisation features; state change log is written to a Kafka changelog topic ⇒ **replicated, recoverable state** (state-management continuum, §8).
4. Results published to an **output Kafka topic** (chaining stream jobs — Samza's core pattern) consumed by the feed-personalisation service.
5. **Fault tolerance**: a failed task is restarted by YARN and rebuilds state by replaying the changelog from its last checkpointed Kafka offset (rollback recovery, §9); Kafka replication protects the data.
6. **Delivery semantics**: at-least-once with idempotent updates (or exactly-once if the framework supports it) so duplicates do not corrupt counts ⇒ consistent state.

8 · Key features of stream-processing frameworks [BOOK p.69 ↗](#)

8.1 Message delivery semantics in the analysis tier [BOOK p.69 ↗](#)

- **At-most-once** — a message may be lost but never processed twice; simplest, no special logic. **At-least-once** — never lost, may be processed more than once: the manager tracks

sends + acks and **re-sends** on timeout $\Rightarrow$ the job must be **idempotent** (same message $\Rightarrow$ same result). **Exactly-once** — also detects and ignores duplicates; most complex.

- **Choosing:** turbine-monitoring prediction may tolerate missing data $\Rightarrow$ least complex guarantee first; **ad network billing** $\Rightarrow$ exactly-once. Always design jobs to be idempotent.

[BOOK p.71 ↗](#)

- CIE-2 Jan'25 A2 "Storm vs Samza message semantics" — see Storm/Samza text above: Storm tuple-at-a-time, Samza stage-wise via Kafka; the book does not tabulate their guarantees, so answer with the three definitions + say Samza relies on Kafka's log for replay.

⚠ CONFLICT / TRAP — Figure 4.11 caption error

The book's Fig 4.11 (p.70) is captioned "At-least-once" but it illustrates **at-most-once** (messages dropped / processor crashed, "two messages went in and only one came out"). In the exam, label that diagram at-most-once.

8.2 State management [BOOK p.71 ↗](#) **IMPORTANT** CIE-3 Jan'26 2a (4M) · Model QP 1.10 (internal vs external state, Unit 5 too)

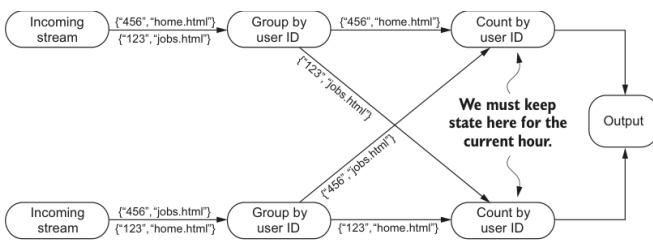


Figure 4.12 Simple example of counting page views per user over an hour

Fig 4.12 Page views per user per hour: state kept in the counter (p.72)

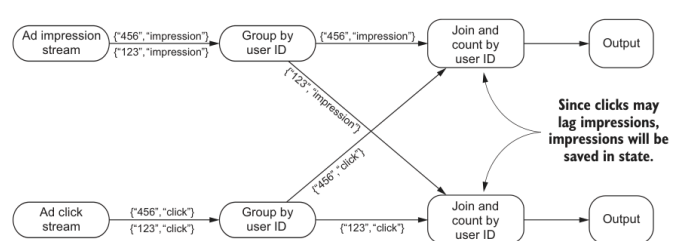


Figure 4.14 Handling ad impression and ad click streams that use stream state

Fig 4.14 Ad impression + ad click streams joined using state (p.73)

- State is needed once an algorithm depends on **previous messages or external data** (counts, joins, late events). Without state management, a failure loses intermediate results.
- **Continuum** (Fig 4.13): **in-memory only** (low complexity; lose everything on failure — keep in memory and flush each hour) $\rightarrow$... $\rightarrow$ **replicated, queryable, persistent store** (high complexity; allows joins of streams). Low end only keeps a computation alive through failures; high end answers complex questions.
- **Example (CIE-3 scheme):** ads — clicks lag impressions, so impressions are **retained in state keyed by user/ad ID** until the matching click arrives; persistent + replicated state survives failures and enables the join and click-through counts.

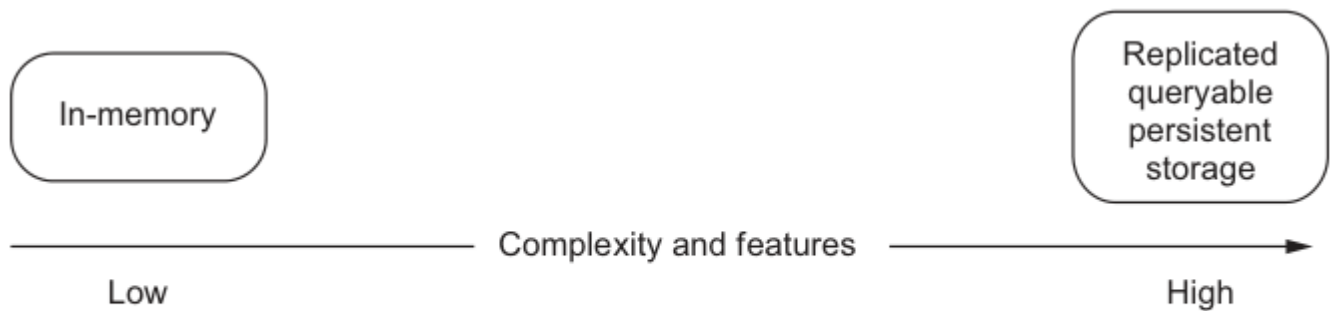


Figure 4.13 State management complexity continuum for stream-processing tools

Fig 4.13 State-management complexity continuum (p.72)

8.3 Fault tolerance — the SEVEN points of failure [BOOK p.73 ↗](#) **IMPORTANT** Model QP 4a (10M) · CIE-2 Jan'25 3a (10M) · SEE Feb'26 8b

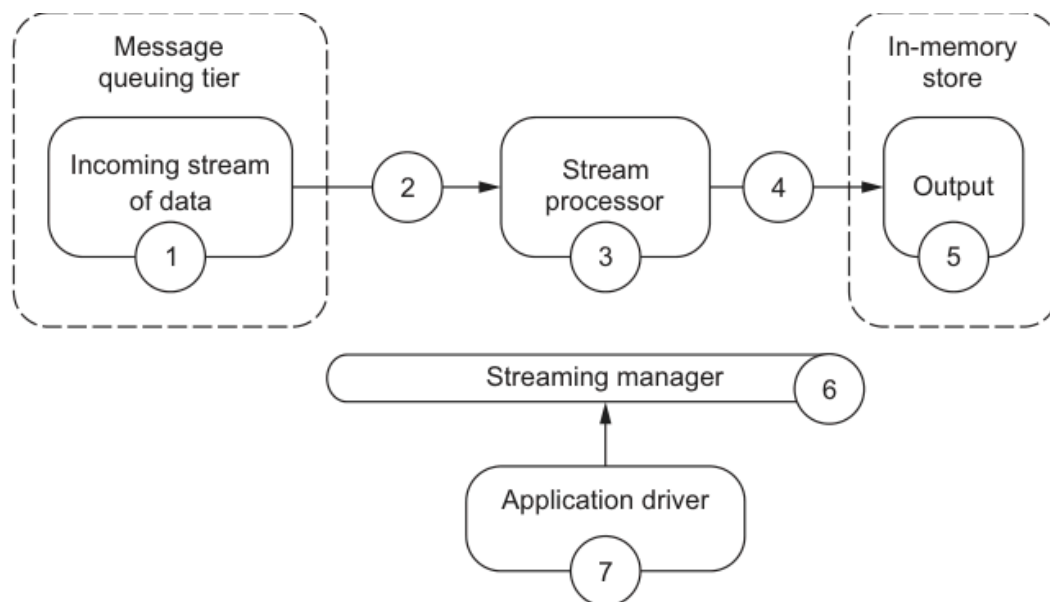


Figure 4.15 The points of failure with stream processing in the context of the streaming architecture

Fig 4.15 Points of failure with stream processing in the streaming architecture (p.74) — DRAW THIS

#	Failure point	What the framework must do
1	Incoming stream of data (queue tier may fail)	Respond gracefully; don't fail if data/resource unavailable
2	Network carrying the input stream	Handle disruption gracefully (can't control it)
3	Stream processor (job software or machine fails)	Streaming manager restarts it or moves it to another machine
4	Connection to output destination	Control flow from last processor; avoid network back-pressure overload
5	Output destination	Not supervised, but its failure impacts processing
6	Streaming manager	If it fails the processors run headless — no supervision, no new/recovered processors
7	Application driver	Only a risk if it contains the manager (second flavour)

- Consolidated: **data loss** (network, processor crash with in-memory data) and **loss of resource management** (manager, driver). Techniques are all **replication + coordination**; **k-fault tolerant** = survives k simultaneous failures.

🗨️ MNEMONIC — I-N-P-C-O-M-D

Incoming stream · Network · Processor · Connection to output · Output destination · Manager · Driver.

State-machine vs rollback recovery BOOK p.75 > IMPORTANT CIE-3 Jan'26 3b (4M) · CIE-2 Jan'25 A3

	State-machine approach	Rollback recovery
Idea	Manager replicates the job on independent nodes , sends the same input in the same order to all	Processor periodically checkpoints state (to another node/disk, often with in-memory logs); on failure rebuild from last checkpoint and replay the log
Cost	k + 1 times the resources of one replica (high)	Lower overhead
Recovery	Quick failover , little disruption	Slower recovery (checkpoint + replay); disks add latency
Best for	Low-latency always (e.g. intrusion detection)	Fault tolerance important, rare moderate latency acceptable

Both assume the job is **idempotent and consistent**: two non-faulty processors with the same input in the same order give the same output in the same order. Fault tolerance via state machine in CIE-2 Jan'25 A3 (2M): replicate the job on k+1 nodes with identical ordered input, so another replica takes over instantly.

9 · Quick revision — Unit 2

Three parts of a message queuing system?	Producer, broker, consumer.
Why "broker" not "queue"?	Broker abstracts over multiple queues.
Two things the queue tier gives (Part A)?	Decoupling, buffering of surges, durability, delivery guarantees.
Durable vs transient?	Durable persisted (JDBC/file/proprietary); transient deleted after consumption.
Three semantics?	At-most, at-least, exactly-once.
Two techniques for exactly-once without product support?	Don't blindly retry (verify with broker); store last-message metadata (Kafka offset).
Two bonuses of those techniques?	Message auditing, duplicate detection.
Broker crash mitigations?	Producer ack after disk write; replicate to >1 broker; minimal in-memory data.
In-flight data means?	Tuples between input and output, never at rest.
Continuous query = ?	Registered once, evaluated on arriving data, results pushed.
Storm vs Samza?	Storm tuple-at-a-time (Nimbus/supervisors); Samza stage-wise on YARN + Kafka.
Spark Streaming processes?	Micro-batches (<0.5 s).
Flink master and workers?	JobManager, TaskManagers.
Idempotent job?	Same input $\Rightarrow$ same output; needed for at-least-once.
Points of failure in stream processing?	7 (incoming stream, network, processor, output connection, output, manager, driver).
"Running headless"?	Streaming manager down; processors keep running unsupervised.
State-machine cost?	k+1 times resources.

Unit 3 — Algorithms for Data Analysis & Storing the Data

Syllabus: accepting constraints and relaxing · thinking about time · summarization techniques · long-term storage · keeping it in memory

Sources: Psaltis *Streaming Data* Ch.5 (printed pp.77–94) and Ch.6 (pp.95–112) + all question papers. **BOOK p.xx** ↗ opens *Streaming-Data[1].pdf* at printed page xx (PDF = printed + 20). Blue **highlight** = must-remember. **IMPORTANT** = asked before.

0 · Hot-list for Unit 3 (Q5/Q6 — choose one, 16 marks)

Rank	Topic	Asked in
1	Windows : sliding / tumbling, choose & justify, compare	SEE Feb'26 5a (10M treadmill), 6a (6M compare) · Model QP 5a (Fitbit) · CIE-2 Jan'25 4a · CIE-3 Jan'26 A4
2	Summarization techniques (sampling, distinct, frequency, membership)	SEE Feb'26 5b (6M), 6b (10M) · CIE-3 Jan'26 A5 & Q5 (10M) · Model QP 5b (reservoir) · CIE-2 Jan'25 1b (Bloom filter)
3	Caching strategies	Model QP 6a · CIE-2 Jan'25 5a (3 strategies, 10M)
4	Writing to long-term storage (3 ways) / batch loader	CIE-3 Jan'26 2b, 3a · Model QP 1.6
5	Stream time vs event time; trigger & eviction policy	Model QP 1.5 · CIE-2 Jan'25 A4, A5
6	Sliding-window support in frameworks	Model QP 6b · CIE-2 Jan'25 4a(ii)
7	In-memory database (IMDB/IMDG), constraints, concept drift	SEE Feb'26 1.6 · CIE-3 Jan'26 A3

1 · Querying a stream and accepting constraints **BOOK p.77** ↗

- **Ad-hoc query**: asked once about the stream (max value so far) — like an RDBMS query.
Continuous query: asked at all times (max value every 5 minutes, alert if above threshold).
BOOK p.77 ↗
- No common SQL-like language: Storm SQL (experimental), Samza none, Flink Table API (SQL-like), Spark SparkSQL/Hive. **BOOK p.78** ↗

Constraints you must accept [BOOK p.78 ↗](#)

Constraint	Meaning
One-pass	Data is not archived — you touch each item once ; multi-pass data-mining algorithms must be redesigned
Concept drift IMPORTANT CIE-3 Jan'26 A3	Statistical properties of the data change over time , hurting predictive models (e.g. a stock-price model trained pre-pandemic fails after it)
Resource constraints / load shedding	Arrival rate is not controllable; when overwhelmed the algorithm drops tuples (random or semantic load shedding)
Domain constraints	Business data itself is huge (100 M users' emails ⇒ petabytes) so even distinct counts are hard

Because of these, nearly every streaming method uses a **synopsis** (online summary) — accept **approximate** answers with high confidence. [BOOK p.79 ↗](#)

MNEMONIC — "One Cat Really Dislikes" = **One-pass · Concept drift · Resource limits · Domain constraints**

2 · Thinking about time [BOOK p.79 ↗](#)

2.1 Stream time vs event time [BOOK p.80 ↗](#) **IMPORTANT** Model QP 1.5 (2M) · CIE-2 Jan'25 A4 (2M, with example)

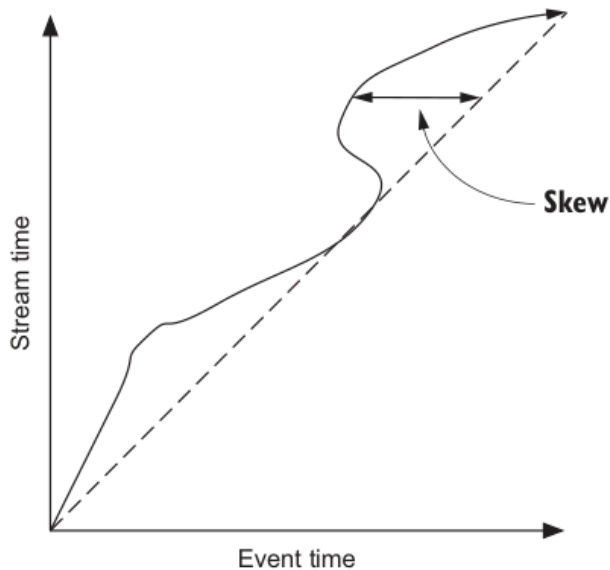


Figure 5.1 Time skew between event time and stream time

Fig 5.1 Time skew between event time and stream time (p.80)

Stream time	Event time
Time at which the event enters the streaming system (analysis tier)	Time at which the event actually occurs (e.g. on the device)
Fitbit: the reading reaches the analysis tier	Fitbit: the moment the runner's reading was recorded on the wrist

The gap is **time skew** (can be significant). Windows built on stream time give wrong averages if events arrive late.

⚠ **CONFLICT / TRAP — different "time" vocabularies**

Streaming Data book: **stream time vs event time** (use these two words in Unit 3 answers). The Kafka book (Unit 5, Ch.14) uses **event time, log-append (ingestion) time, processing time**; its "processing time" $\approx$ the book's stream time, and it calls processing time "unreliable and best avoided". Don't mix them in one answer.

2.2 Windows of time (BOOK p.80 ↗)

- The engine cannot keep the infinite stream in memory, so computations run on **windows** — a small amount of data at a point in time (Fig 5.2). Every windowing technique has two policies: **Trigger policy** = rule that tells the system **when to run** your code on the window; **Eviction policy** = rule that decides **when data leaves** the window. Both are driven by **time or count**. (BOOK p.81 ↗) **IMPORTANT** CIE-2 Jan'25 A5

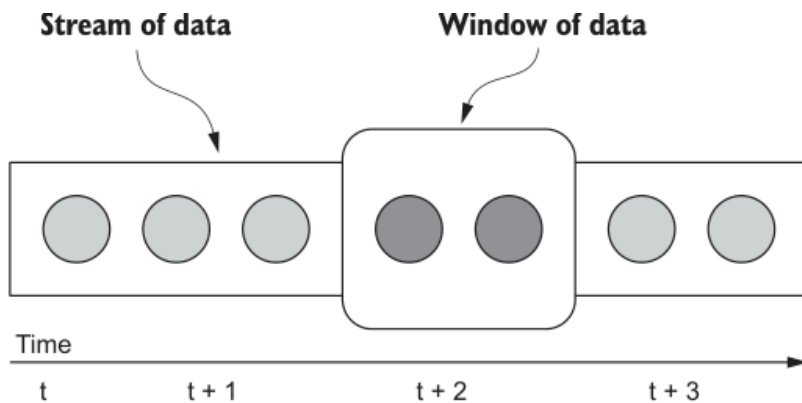


Figure 5.2 A window of data in perspective to the rest of the stream

Fig 5.2 A window of data in perspective to the rest of the stream (p.81)

2.3 Sliding window (BOOK p.81 ↗) **IMPORTANT** Model QP 5a (10M) · CIE-2 Jan'25 4a (10M) · SEE Feb'26 6a

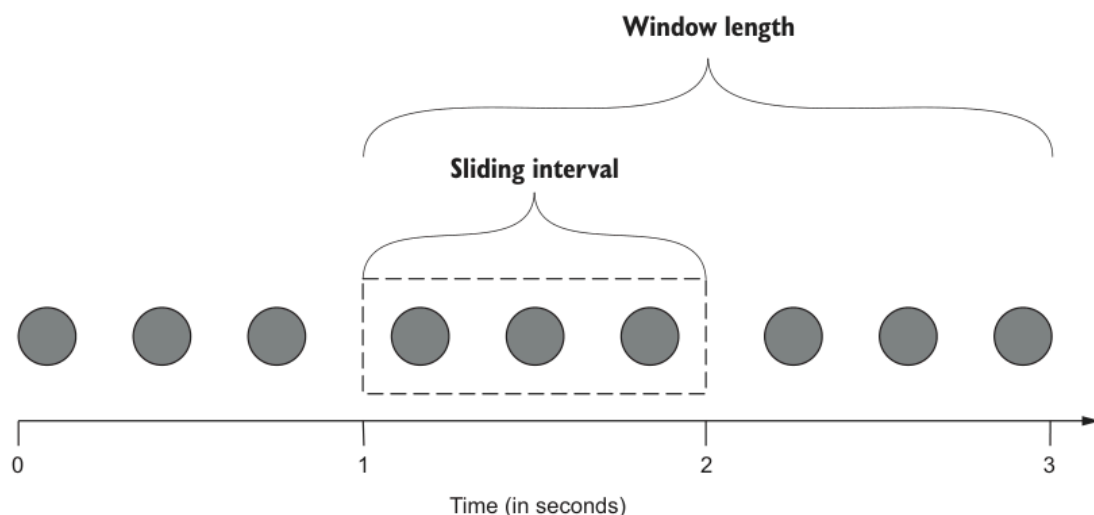
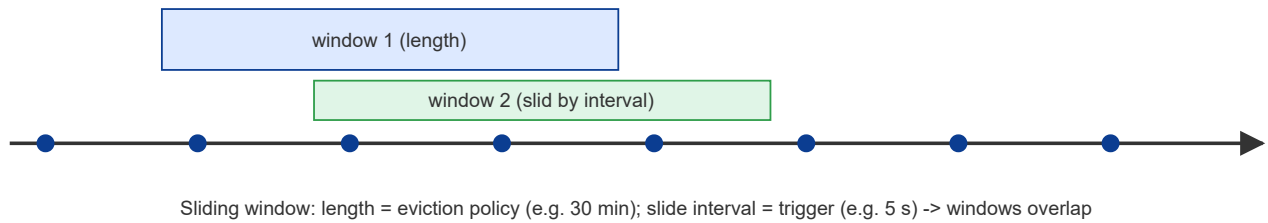


Figure 5.3 Sliding window showing the slide interval and the window length

Fig 5.3 Sliding window: window length = 2 s, sliding interval = 1 s (p.81)

- Policies are **time-based**: **window length = eviction policy** (how long data is kept; older data evicted) and **sliding interval = trigger policy** (how often your code runs). Windows **overlap** when interval $<$ length, so one event can appear in several windows.

- Fitbit example (book p.82):** dashboard of **average speed per age group** (12–17, 18–24, 25–34...), updated **every 5 s**, averages over the **last 30 min** ⇒ **sliding window, length = 30 minutes, sliding interval = 5 seconds**. Because of time skew, window should use **event time** if the framework allows.



Sliding window diagram (our redraw): length vs slide interval, overlapping windows

Framework support — Table 5.2 [BOOK p.82 ↗](#) **IMPORTANT** Model QP 6b · CIE-2 Jan'25 4a(ii)

Framework	Sliding window	Event or stream time	Comment
Spark Streaming	Yes	Stream time only	No custom policies
Storm	No (native)	N/A	Can be implemented using timers
Flink	Yes	Both	User-defined custom trigger/eviction policies
Samza	No	N/A	No direct support

2.4 Tumbling window [BOOK p.83 ↗](#) **IMPORTANT** SEE Feb'26 5a (10M), 6a (6M) · CIE-3 Jan'26 A4 (2M)

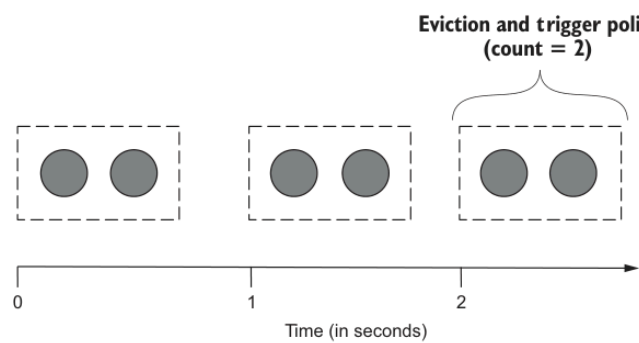


Figure 5.4 Count-based tumbling window with an eviction and trigger policy of two

Fig 5.4 Count-based tumbling, trigger = eviction = 2 items (p.83)

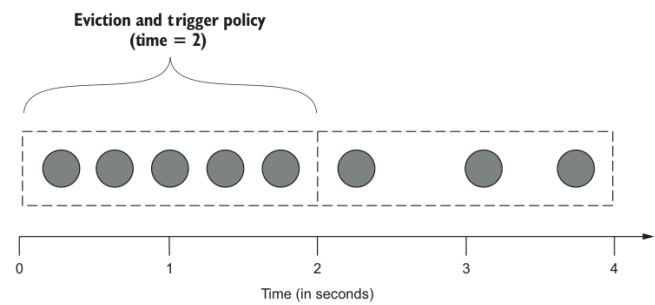


Figure 5.5 Temporal tumbling window with an eviction and trigger policy of two seconds

Fig 5.5 Temporal tumbling, trigger = eviction = 2 s (p.83)

- Eviction policy = window full** (always). Trigger = count of items **or** time. When it fires, the window is **drained completely** ⇒ windows **never overlap**; each event is in exactly one window.
- Two types: **count-based** (fires when N items, irrespective of time) and **temporal-based** (fires when T time passes, irrespective of item count).

- **Book example (bicycles):** average speed every 30 s $\Rightarrow$ temporal tumbling window 30 s; alert every time > 100 people ride in a city $\Rightarrow$ count-based tumbling window of 100, one stream/window per city. To split by geography later keep the full message (GPS) rather than pre-filter. ([BOOK p.84 ↗](#))

Framework support — Table 5.3 ([BOOK p.84 ↗](#))

Framework	Count	Temporal	Comment
Spark Streaming	No	No	You must build it
Storm	Yes	Yes	No native windowing but easy to implement
Flink	Yes	Yes	Built-in support for both
Samza	No	Yes	(comment in book mistakenly says "sliding")

⚠ **CONFLICT / TRAP — Table 5.3 vs the text on page 85, and "sliding" vs "hopping"**

(a) Text on p.85 says Flink is **the only** framework with built-in tumbling support, while the table lists Storm Yes/Yes (implemented easily, not native) — write "Flink has built-in; Storm can implement easily".

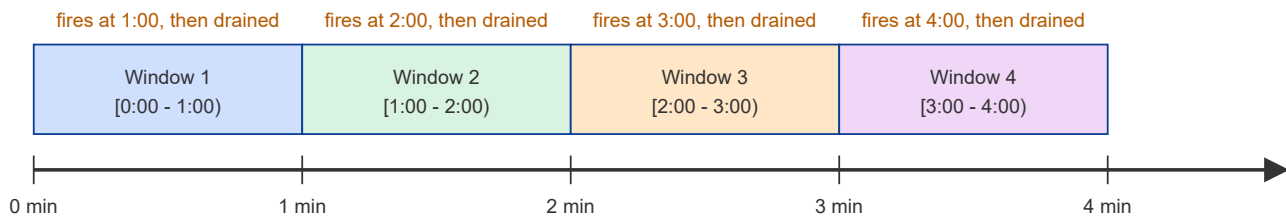
(b) The Kafka book (Unit 5) calls a fixed-size window that advances by less than its size a **hopping window** (tumbling = advance equals size) — e.g. its stock example uses a **5 s window advancing every 1 s**. The Streaming Data book (and the syllabus) call this a **sliding window**. In Unit 3 answers say "sliding"; in Kafka Streams answers say "hopping (what Unit 3 calls sliding)".

Sliding vs tumbling (**IMPORTANT** [SEE Feb'26 6a \(6M\)](#))

	Sliding	Tumbling
Eviction policy	Time-based window length (old data evicted as new arrives)	Window full (all items drained)
Trigger policy	Time-based sliding interval	Count or time
Overlap	Overlapping windows; event in many windows	Non-overlapping ; event in one window
Types	One (time-based)	Count-based, temporal
Output rate	Frequent, smooth moving results	One independent result per window
Use cases	Moving average over last 30 min refreshed every 5 s (Fitbit speed dashboard)	Total energy every 1 min; bikes' average speed every 30 s; alert when 100 riders
Framework support	Spark, Flink native	Flink native (Storm, Samza partial)

2.5 Worked answer — smart treadmills ([SEE Feb'26 Q5a, 10M](#)) (**IMPORTANT**)

Two metrics: (1) total energy consumption of all treadmills every 1 minute, one independent result per minute, no overlap, global for now (may split by gym later); (2) detect any treadmill exceeding a critical temperature in a 1-minute interval, reset every minute, non-overlapping.



Tumbling window: size = 1 min, slide = 1 min, no overlap; every reading belongs to exactly one window

Temporal tumbling window of 1 minute for the treadmill streams (our diagram)

1. **Window type: temporal tumbling window, size 1 minute** (trigger = eviction = 1 min).
Justification: results must be **independent, non-overlapping** and reset every minute $\Rightarrow$ eviction when the window is full/time elapsed; a sliding window would overlap and double-count energy.
2. **Metric 1:** create a stream carrying `power consumption` (keep the complete message incl. gym location); at each 1-minute trigger **sum** all tuples in the window $\Rightarrow$ total energy for that minute; drain window.
3. **Future split by gym:** because full messages are kept, group by gym id inside the window; avoid pre-filtering by location in the collection tier (inflexible, boundaries must be decided up front — book p.84).
4. **Metric 2:** on the same 1-min tumbling window, filter tuples with `machine temperature > threshold`; if any exist at the trigger, raise a maintenance alert; window resets so the next minute is clean.
5. **Time:** use event time (sensor timestamp) if the framework supports it to handle late tuples; otherwise stream time. **Framework:** Flink has built-in temporal tumbling windows (Table 5.3).

🔥 HOW TO WRITE IT — Fitbit question (Model QP 5a / CIE-2 Jan'25 4a)

Sliding window, **length 30 min, slide 5 s**, group by age band, average speed; draw Fig 5.3 style diagram labelled with those numbers; discuss stream vs event time (skew from device delay) and framework support (Flink: both times + custom policies; Spark: stream time only).

3 · Summarization techniques BOOK p.85 IMPORTANT SEE Feb'26 5b, 6b · CIE-3 Jan'26

A5, Q5 · Model QP 5b

Why summarise? The stream is unbounded and cannot fit in memory, so exact answers are costly or impossible; a good **estimate with high confidence** is usually enough (and the "picture of the business" doesn't change). Four techniques, each answers one question:

Technique	Question answered	Algorithm	Error type
Random sampling	Give me a random sample of the stream	Reservoir sampling	—
Counting distinct elements	How many <i>distinct</i> items?	Bit-pattern: LogLog, HyperLogLog , HLL++ · Order statistics: MinCount, Bar-Yossef	Approximate ($\approx 2\%$ with 1.5 KB for 1 billion)
Frequency	How many times has element X occurred?	Count-Min Sketch	Never under-counts, may over-count
Membership	Has X occurred before?	Bloom filter	False positives possible, no false negatives

🗨️ **MNEMONIC — "SD-FM" → Sampling, Distinct, Frequency, Membership = "Reservoir, HyperLogLog, Count-Min, Bloom"**

3.1 Random sampling — reservoir sampling [BOOK p.86 ↗](#) **IMPORTANT** Model QP 5b (6M)

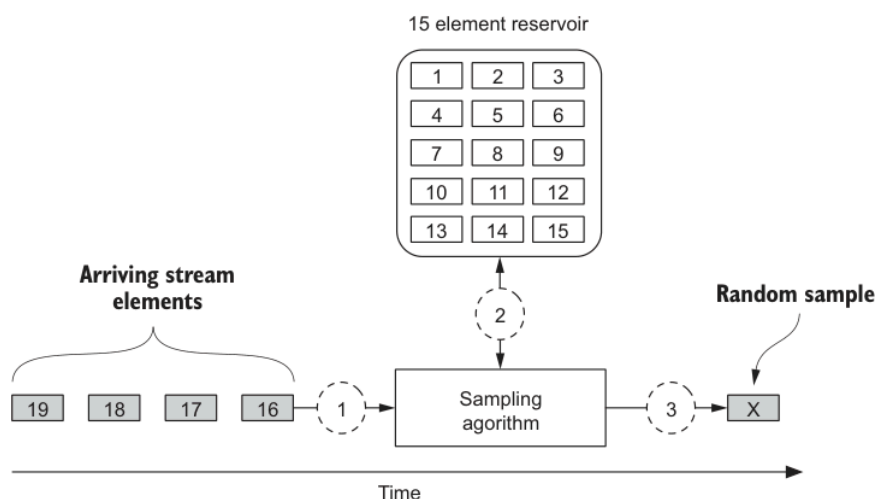


Figure 5.6 General flow of reservoir sampling with first new data item about to be processed

Fig 5.6 General flow of reservoir sampling; first new item (16th) about to be processed (p.86)

- Problem: ad network, 10 M ad views/min, statistical analysis needed, data doesn't fit memory or disk $\Rightarrow$ keep a **reservoir of fixed size k** .
- Fill the reservoir with the **first k** items. For the **n -th** item ($n > k$): include it with probability **k/n** ; if chosen, it **replaces a randomly chosen** element already in the reservoir. Every element seen so far has equal probability **k/n** of being in the reservoir.
- Book example ($k = 15$): the 16th item enters with probability **$15/16$** (generate a random number in $[0,1]$; if $< 15/16$ add it and displace a random reservoir item). [BOOK p.87 ↗](#)
- Not built into Spark/Storm/Samza/Flink but easy to implement. Reference: Vitter 1985.

3.2 Counting distinct elements — HyperLogLog [BOOK p.87 ↗](#)

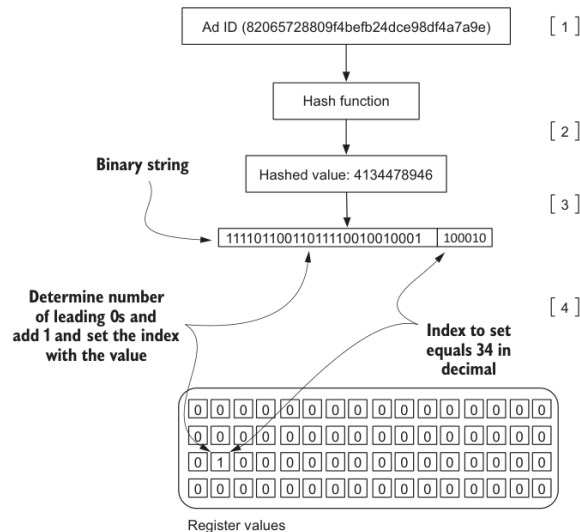


Figure 5.7 Processing a single stream element with the HyperLogLog algorithm

Fig 5.7 Processing one stream element with HyperLogLog (p.89)

1. Take the element (ad ID) → **hash** it to a binary string.
 2. The **last 6 bits** (precision = 6) give the **register index** (100010 = 34).
 3. Count the **leading zeros** of the remaining bits + 1; store it at that register if larger.
 4. Estimate = **harmonic mean** of all register values (averaging many estimates reduces bias; harmonic mean reduces outlier impact).
- Uses tiny space (**1 billion distinct items at 2% accuracy in 1.5 KB**) and is **distributable**: union of two HLL structures ⇒ merge per-node summaries. HLL++ = less memory, better accuracy. Needed e.g. for a 40 Gbps intrusion-detection system (~78 M packets/s).

[BOOK p.90 ↗](#)

3.3 Frequency — Count-Min Sketch [BOOK p.90 ↗](#)

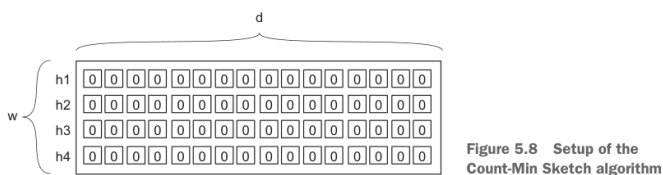


Fig 5.8 Setup: $w \times d$ counters, one hash per row (p.91)

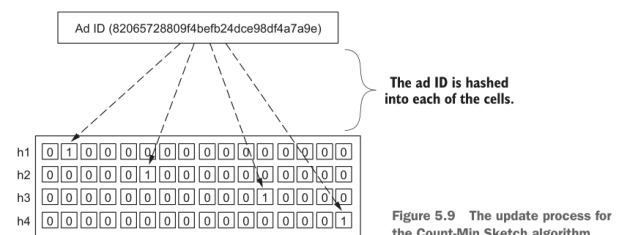


Fig 5.9 Update: hash into each row, add 1 (p.91)

- 2-D array of counters initialised to 0; each **row has its own pairwise-independent hash function** (example 4×16).
- **Update**: hash the element with each row's hash, **increment** that cell in every row. **Query**: **estimate = min** of the cells the element hashes to — the minimum reduces collision error. **Never under-counts, may over-count.**
- Answers **point queries** (one element), **range queries**, **inner-product queries** (join size of two sketches). Accuracy: 8×128 array ⇒ $\approx 1.5\%$ relative error with 99.6% probability. Uses: how often an ad was viewed; DoS detection. [BOOK p.92 ↗](#)

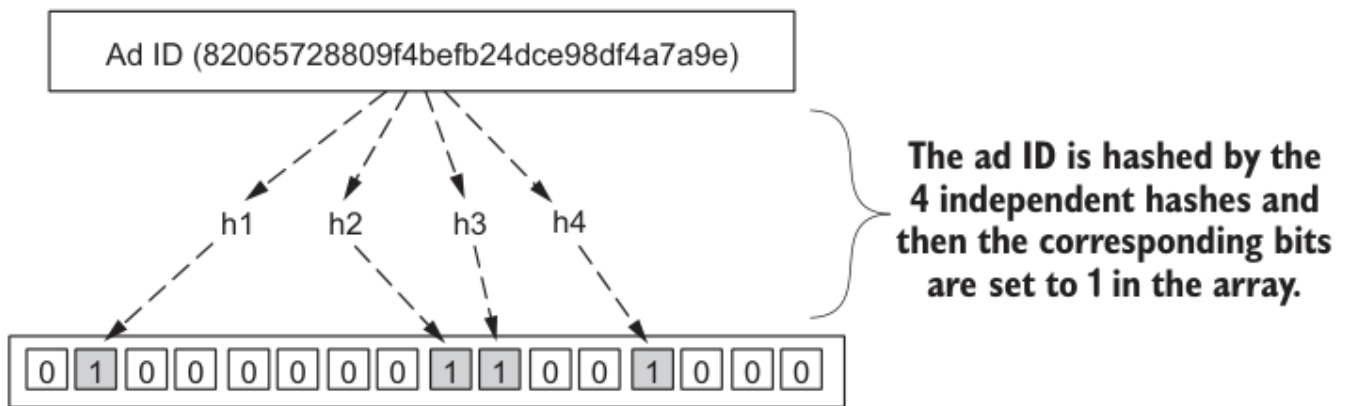


Figure 5.10 A Bloom filter showing one stream element being processed

Fig 5.10 A Bloom filter processing one element: 4 hashes set 4 bits to 1 (p.93)

- Binary **bit array of length m** (space = $m/8$ bytes) + several independent hash functions.
Insert: hash the element with each function and **set those bits to 1** (a set bit stays set on collision).
- **Query:** compute the hashes; if **all** bits are 1 $\Rightarrow$ "probably seen", if **any** is 0 $\Rightarrow$ **definitely never seen**. So **false positives possible, false negatives impossible**.
- **Membership queries** ("does this element belong to the set/group?") are exactly what it solves: check if an ad/packet signature/bot host has appeared **before paying for an expensive lookup** in an external store; e.g. network monitoring first checks the filter for known bad hosts, then confirms positives with the costly check. [BOOK p.94 ↗](#)

Compare the structures

	Count-Min Sketch	Bloom filter
Structure	Several counter arrays (width w, length m), 1 hash per array	One bit array length m, several hashes
Update	Increment counters	Set bits to 1
Answer	Approx. count (min of counters)	Yes/no membership (AND of bits)
Error	Over-estimates only	False positives only

HOW TO WRITE IT — "Role of summarisation with example and classification" (SEE 5b, 6M) and "list and elaborate any two" (SEE 6b, 10M)

5b: why (unbounded stream, memory, approximate OK) $\rightarrow$ table of 4 classes with the question each answers + one example (ad network: sample views, distinct ads/min, views of ad X, was ad X shown before?). 6b: list 4, then elaborate two (choose **reservoir sampling** and **Count-Min/Bloom** — steps are easy to draw; give formula k/n or min-of-counters) with a diagram from Fig 5.6/5.9/5.10 and one example each, 5 marks each.

4 · Storing the analysed or collected data BOOK p.95 ↗

After analysis there are **four options**: (1) **analyse and discard**; (2) **analyse and push back into the streaming platform** (output of one job = input of another; core of Samza); (3) **analyse and store for real-time usage**; (4) **analyse and store for batch/offline access**. BOOK p.95 ↗

IMPORTANT CIE-3 Jan'26 3a

4.1 Long-term storage — three ways to write BOOK p.97 ↗ **IMPORTANT** CIE-3 Jan'26 2b (6M), 3a (6M)

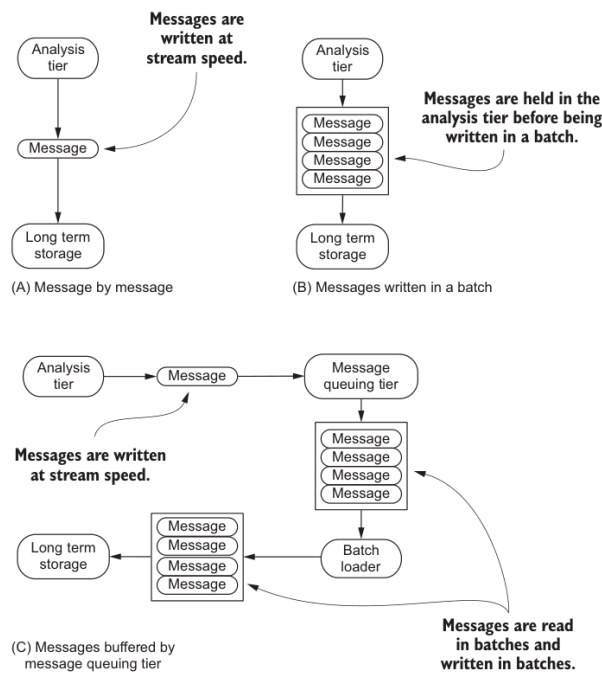


Figure 6.2 Three ways to write data to long-term storage

Fig 6.2 Three ways to write data to long-term storage (p.97) — DRAW THIS

Option	How	Type	Risk / benefit
A. Message by message	Write each message as processed — at stream speed	Direct writing	Store may not keep up ⇒ hurts stream processing; batch stores (HDFS, S3) are not made for continuous writes
B. Batch from analysis tier	Hold messages in analysis tier until batch size or time reached, then write	Direct writing	Optimises for slow writes but still couples analysis to store speed
C. Via message queuing tier + batch loader	Analysis tier writes to queue at stream speed; batch loader reads in batches and writes in batches	Indirect writing	Decouples analysis from store performance; queue is built for stream speed; loader is dedicated bulk-load tool; extra component = complexity

- **Batch loader** = component that reads batches of messages from the queue tier and writes them to long-term storage; typical ETL features: scheduling, error handling, data-quality checks, publishing, monitoring, scaling, fault tolerance. Tools: **Gobblin** (LinkedIn), **Secor** (Pinterest) for Kafka → HDFS/S3. BOOK p.98 ↗ **IMPORTANT** Model QP 1.6 (define, one benefit)
- **Video-analytics (CIE-3 3a):** CCTV results for compliance and model training ⇒ option C is best (queue absorbs frame-result bursts, loader writes Parquet/files to S3/HDFS); A or B for

low volume.

4.2 Keeping it in memory [BOOK p.99 ↗](#)

- Goal: act on data in real time, so current/recent data must be in memory ("RAM is the new disk"). SSD seek $\approx 100,000$ ns vs DRAM ≈ 100 ns vs L1 ≈ 0.5 ns; a 1 TB DRAM cluster $\approx$ \$5,000/month on EC2. [BOOK p.100 ↗](#)
- Three categories: **embedded in-memory / flash-optimised**, **caching systems**, **IMDB / IMDG**.

(a) Embedded in-memory / flash-optimised [BOOK p.100 ↗](#)

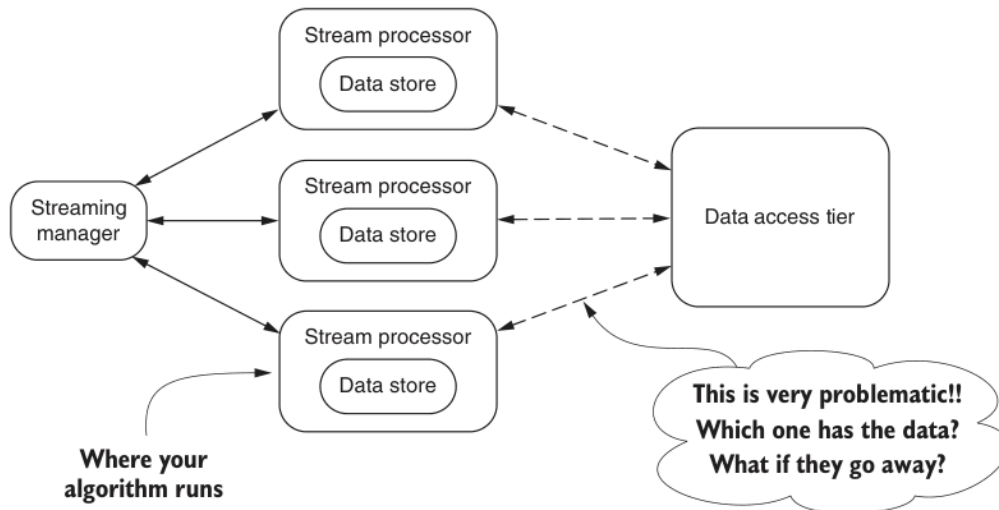


Figure 6.3 Architectural view of using an embedded data store with our analysis nodes

Fig 6.3 Embedded store on each analysis node — "very problematic" (p.100)

- Single-node, embedded in your software, no cross-node access/management $\Rightarrow$ **poor fit** for distributed streaming: which node has the data? what if it goes away? Nodes then do processing + storage + serving (fragile). Products: SQLite, RocksDB, LevelDB, LMDB, Perst.

(b) Caching systems — strategies [BOOK p.101 ↗](#) **IMPORTANT** Model QP 6a · CIE-2 Jan'25 5a (any three, 10M)

Object cache / in-memory key-value store, often **DRAM only**. Strategies related to **persisting cache entries and keeping them fresh**:

Strategy	How it works	Pros / cons
Read-through (Fig 6.4)	On a miss the cache loads from the persistent store transparently to the client	Penalty on first read; updates must still be written to store
Refresh-ahead (Fig 6.5)	Cache refreshes recently used entries before they expire	Avoids read-through penalty, fresher; hard to sync with a fast stream
Write-through (Fig 6.6)	Cache writes through to the backing store ; write acknowledged only after the store write	No separate sync process; extra write latency
Write-around (Fig 6.7)	Store updated out-of-band ; another process updates the cache afterwards	Cache needn't talk to store; dual update complexity
Write-back / write-behind (Fig 6.8)	Write acknowledged after cache; store updated in the background	No I/O wait; risk of data loss until flushed

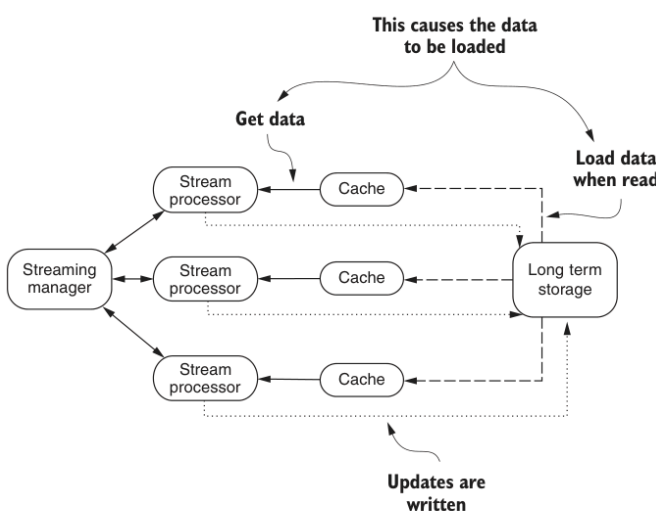


Figure 6.4 Read-through caching strategy showing how the data is loaded from long-term storage if it's not in the cache

Fig 6.4 Read-through (p.102)

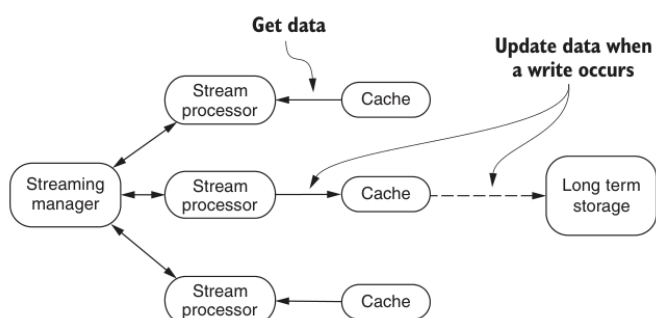


Figure 6.6 Write-through cache strategy showing how data is written all the way through to the long-term storage

Fig 6.6 Write-through (p.103)

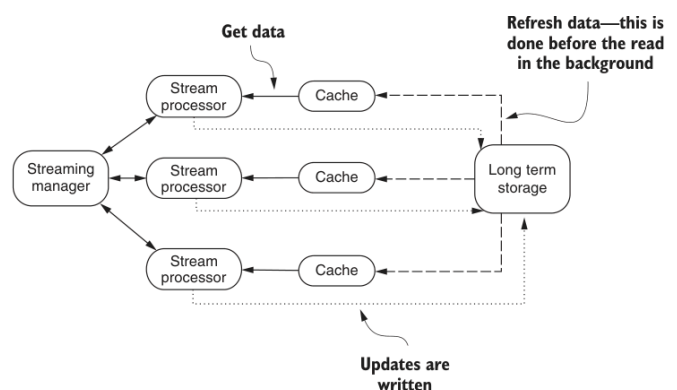


Figure 6.5 Refresh-ahead cache strategy, showing how data is refreshed from long-term memory

Fig 6.5 Refresh-ahead (p.103)

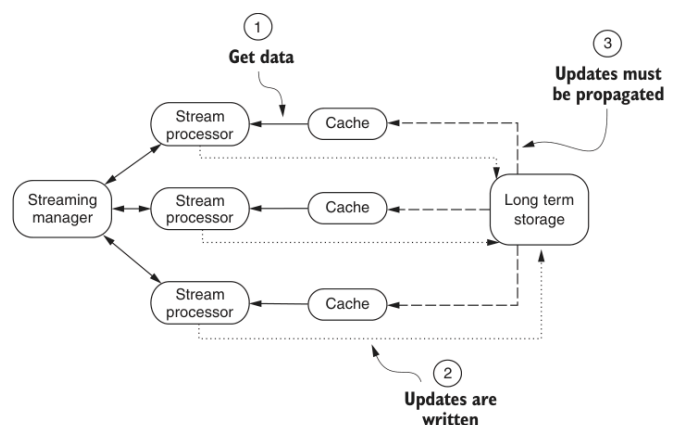


Figure 6.7 Write-around caching strategy showing how data is written to long-term storage and then has to be propagated to the cache

Fig 6.7 Write-around (p.104)

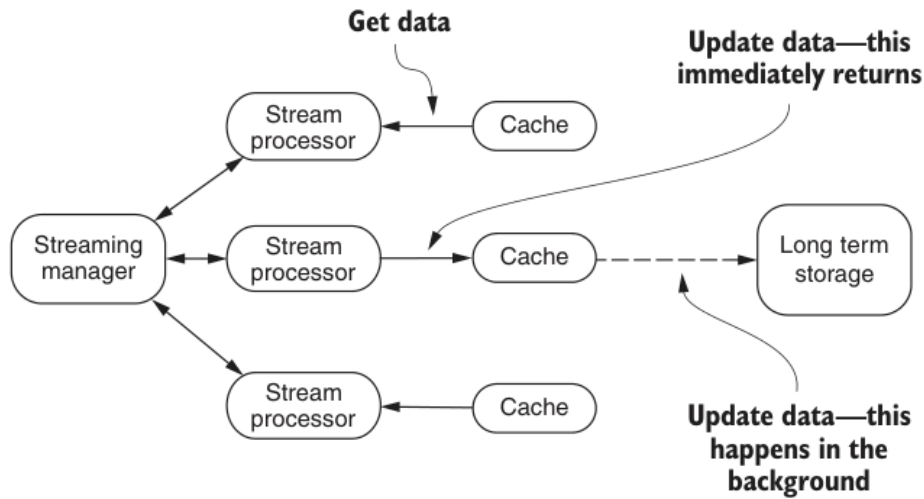


Figure 6.8 Write-back caching strategy

Fig 6.8 Write-back / write-behind (p.104)

- Risks: data only in memory $\Rightarrow$ losing a cache node loses data; dual writing of a constant stream is a **bottleneck**. Products: **Memcached** (use write-around), **EHCache** (read/write-through, write-behind), **Hazelcast** (read/write-through), **Redis** (own persistence; you build the strategy). ([BOOK p.105 ↗](#))

🧠 **MNEMONIC — "RRW-WWW": Read-through, Refresh-ahead | Write-through, Write-around, Write-back**

(c) In-memory database (IMDB) and in-memory data grid (IMDG) ([BOOK p.105 ↗](#)) **IMPORTANT**

SEE Feb'26 1.6

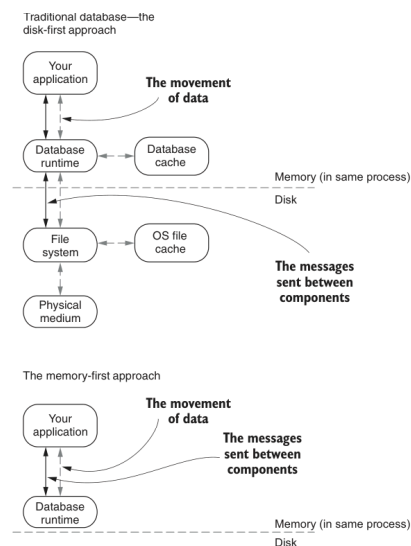


Figure 6.9 The two different approaches to database design: disk-first and memory-first

Fig 6.9 Disk-first (traditional DB) vs memory-first design (p.106)

- **Purpose of an IMDB:** keep the working data **memory-first** for very fast real-time reads/writes while still using **disk for durability** (logging + periodic snapshots) and replication — so analysis results can be served in real time to clients (e.g. session personalisation) without losing data on a crash.
- Unlike caches, they persist to disk; unlike traditional DBs (SQL Server, Oracle, Cassandra — **disk-first, memory-second**) they are **memory-first, disk second**. IMDG = distributed,

compute close to data, non-SQL API; IMDB = often SQL API; the lines are blurring.

- Products: MemSQL, VoltDB, Aerospike (TTL per record $\Rightarrow$ session expiry), Apache Geode, Couchbase, Apache Ignite, Hazelcast, Infinispan. [BOOK p.107 ↗](#)

Category	Persistence	Distributed?	Fit for streaming
Embedded	Local	No	Poor
Caching system	Usually none (DRAM)	Often	OK for transient results; dual-write cost
IMDB / IMDG	Disk logging + snapshots, replicated	Yes	Best (in-session personalisation use case)

Use case (§6.3, p.108): in-session personalisation (30-minute session, click-stream: show helmets matching yellow sunglasses; coupon for sunglasses removed from cart) $\Rightarrow$ embedded = bad, cache = thrashing (read-update-write whole session), **IMDB/IMDG = good**. Energy smart-grid exercise p.111.

5 · Quick revision — Unit 3

Ad-hoc vs continuous query?	Once vs always-on.
Four constraints?	One-pass, concept drift, resource constraints (load shedding), domain constraints.
Stream time vs event time?	Enters system vs occurs at source; gap = time skew.
Trigger vs eviction policy?	When to process vs when to drop data from window.
Sliding window controlled by?	Window length (eviction) + sliding interval (trigger).
Tumbling window types?	Count-based, temporal.
Fitbit dashboard window?	Sliding, length 30 min, interval 5 s.
Which framework supports sliding + event time?	Flink (Spark = stream time only).
Reservoir sampling probability for n-th item?	k/n .
HyperLogLog counts what?	Distinct elements (leading zeros of hash; harmonic mean).
Count-Min Sketch error?	Never under-counts; estimate = min over rows.
Bloom filter error?	False positives yes, false negatives no.
Three ways to write to long-term store?	Message-by-message, batch from analysis tier, via queue + batch loader.
Batch loader tools?	Gobblin, Secor.
Cache strategies?	Read-through, refresh-ahead, write-through, write-around, write-back.
Disk-first vs memory-first?	Traditional DB vs IMDB/IMDG.
Which cache needs write-around?	Memcached.

STREAM PROCESSING & ANALYTICS

Unit 4

Introduction to Apache Kafka

Why Kafka · Kafka Ecosystem · Kafka Origin · Producers & Consumers

One-night exam revision notes — crisp points, diagrams from the textbook, important items starred, and clickable links back to the exact book page.

How to read these notes	
★ IMPORTANT box	Exam-critical. Either asked in past CIE/SEE papers or a core definition. Learn these first.
Yellow highlight	Key term / keyword to underline in your answer.
→ See in book · Part 1 p.3	Blue clickable chip = opens the textbook PDF at that page (Part 1 = Ch.1 Meet Kafka, Part 2 = Ch.3 Producers + Ch.4 Consumers). Revise this page later from the book.
✍ EXAM TIP	How to frame the answer / what the examiner looks for (diagram + points).

REMEMBER Link note

- The blue links point to the book PDFs in Unitwise/Unit4/ (Part1.pdf, Part2.pdf). Open this notes PDF in **Chrome / Edge / Acrobat** on the same laptop for the links to jump to the page. If you move the folder, the links will break.

Syllabus Map & Exam Hotlist

What Unit 4 covers and what the question papers actually asked

Syllabus topic (Unit IV, 9 hrs)	Where in these notes	Book source
Why Kafka	Part A §A5	→ Ch.1 · Part 1 p.14
Kafka Ecosystem	Part A §A6 (+ pub/sub background §A1–A4)	→ Ch.1 · Part 1 p.17
Kafka Origin	Part A §A7	→ Ch.1 · Part 1 p.21
Kafka Producers	Part B (Chapter 3)	→ Ch.3 · Part 2 p.1
Kafka Consumers	Part C (Chapter 4)	→ Ch.4 · Part 2 p.45
Quick revision: tables, Q&A, flash cards	Part D	

Where Unit 4 topics appeared in past papers (taken from the CIE-1/2 2022 solutions, SEE 2022 and the Model QP in your QP folder):

Question that came	Paper	Go to
Schematically represent the Kafka producer components	CIE-1 2022	B1 (Fig 3-1)
Early (eager) vs cooperative rebalance — two differences	CIE-1 2022	C3
What happens if a broker crashes ?	CIE-1 & 2, SEE	A4 (replication) + D3
Kafka data ecosystem with a neat diagram	CIE-1, Model QP	A6 (Fig 1-9)
Producer program with high throughput + no duplicates (idempotence)	CIE-1 2022	B4, B9 code
Range assignor: 6 partitions P0–P5, 2 consumers , then add C3	CIE-1, SEE 2022	C5 worked example
Consumer that deserializes JSON to dictionary (code)	CIE-1 2022	C9 code
Consumers and consumer groups with example	SEE 2022	C1
Three ways to send messages; acks = ? + retries for at-least-once	SEE 2022 (fill-ups)	B3, B4
Commits and offsets ; two offset-commit strategies	SEE 2022	C6
Role of MirrorMaker ; reasons to choose Kafka (2 reasons)	Model QP	A4, A5
Producer–broker–consumer with a message-queue layer (diagram)	CIE-2 2022, SEE	A2 + A3
Producer + consumer implementation for a scenario (code)	CIE-2 2022	B9, C9

🔑 EXAM TIP If you only have 2 hours

- Learn: pub/sub, topic/partition/offset, broker/controller/leader, consumer group, Why-Kafka list, Fig 1-9 ecosystem.
- Learn: producer flow (Fig 3-1), 3 send methods, **acks 0/1/all**, **idempotence**, partitioner rules.
- Learn: consumer group rules (Figs 4-1 to 4-5), **eager vs cooperative**, **commit strategies**, **assignors**, auto.offset.reset.
- Then rehearse the two code programs in Part D.

Part A — Introduction to Kafka

Chapter 1 · Meet Kafka · Textbook Part 1 (26 pp.)

A1. Publish / Subscribe messaging

→ Book · Part 1 p.1 → Fig 1-1-1-4 · Part 1 p.3

- **Pub/Sub** = the **sender (publisher)** does **not** address a specific receiver. It **classifies** the message; **subscribers** pick the classes they want.
- A **broker** (central point where messages are published) usually sits in the middle and decouples the two sides.
- How it starts: one app pushes metrics straight to a dashboard (point-to-point). Add a storage/analysis service, more apps, polling servers ... connections multiply into a tangle = **technical debt**.
- Fix: a **single metrics pub/sub system** (one central app that receives everything and answers queries). Then logging and user-tracking teams build **their own** pub/sub systems → duplicated effort and bugs.
- Goal: **one centralised, generic system** that grows with the business → **Apache Kafka**.

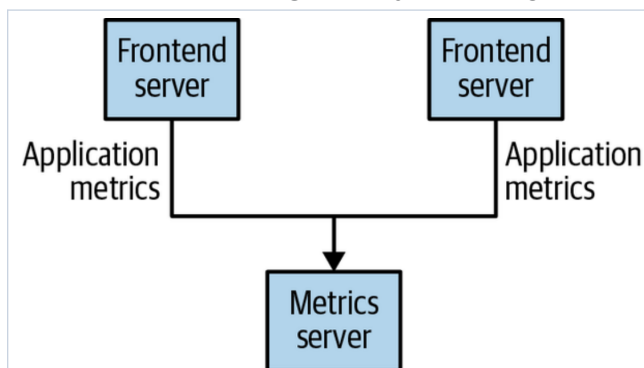


Fig 1-1. Single direct metrics publisher
→ See in book · Part 1 p.3

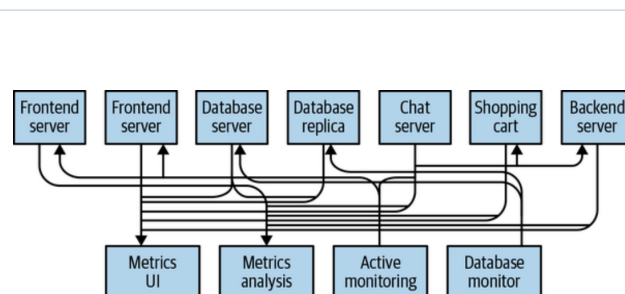


Fig 1-2. Many publishers, direct connections (messy)
→ See in book · Part 1 p.4

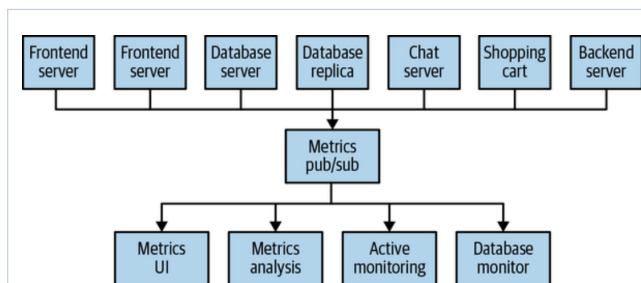


Fig 1-3. A single metrics pub/sub system
→ See in book · Part 1 p.4

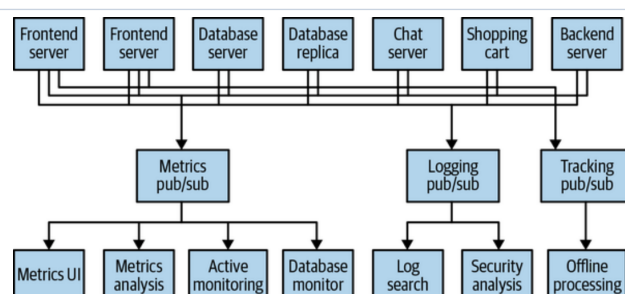


Fig 1-4. Multiple separate pub/sub systems (duplication)
→ See in book · Part 1 p.5

A2. What is Kafka? Messages, batches, schemas

→ Book · Part 1 p.5 → Messages & Batches · Part 1 p.6 → Schemas · Part 1 p.7

★ IMPORTANT Definition (write this first in any answer)

- **Apache Kafka** is a **publish/subscribe messaging system**, described as a **distributed commit log** or (more recently) a **distributed streaming platform**.
- Like a database commit log: data is stored **durably, in order** and can be **read deterministically** (replayed to rebuild state). It is also **distributed** for fault-tolerance and scaling.

Term	Meaning
Message	Unit of data in Kafka (like a row/record). To Kafka it is just a byte array ; no format/meaning is imposed.
Key	Optional metadata (also a byte array). Used to write to partitions in a controlled way: hash(key) mod #partitions → same key always lands in the same partition (as long as partition count is unchanged).
Batch	Collection of messages for the same topic + partition , written together. Fewer network round trips. Trade-off: latency vs throughput (bigger batch = more throughput, higher latency). Batches are usually compressed (saves bandwidth/storage, costs CPU).

Term	Meaning
Schema	Structure imposed on message content. JSON/XML: easy & readable but weak type handling and no version compatibility. Apache Avro (preferred): compact binary, schema kept separate from payload, no code regeneration on change, strong typing, schema evolution (forward + backward compatible).

REMEMBER Why schemas matter

- A consistent format **decouples reading from writing**: publishers can change format without coordinating with every subscriber. Schemas live in a common repository (Schema Registry, see B6).

A3. Topics, partitions, producers, consumers

→ Topics & Partitions · Part 1 p.8 → Producers & Consumers · Part 1 p.9 → Fig 1-5, 1-6 · Part 1 p.9

- Topic** = category of messages (like a **DB table / folder**). Split into **partitions**.
- Partition** = a single **append-only log**, read in order from beginning to end. **Ordering is guaranteed only within a partition, NOT across the whole topic**.
- Partitions give **redundancy + scalability**: each partition can live on a different server (horizontal scaling) and can be **replicated**.
- Stream** = usually one topic regardless of partition count (producers → consumers). Stream processing (Kafka Streams, Samza, Storm) works in **real time**, unlike offline bulk frameworks such as Hadoop.
- Producer** (publisher/writer) creates messages for a topic. Default: **balances evenly over partitions**; with a key + partitioner (hash) or a **custom partitioner** it targets specific partitions.
- Consumer** (subscriber/reader) subscribes to topics and reads each partition **in order**. It tracks progress by the **offset**.
- Offset** = ever-increasing integer metadata that Kafka adds to each message; unique per partition. Storing the next offset (in Kafka) lets a consumer **stop and restart without losing its place**.
- Consumer group** = consumers working together on a topic; **each partition is consumed by only one member** (partition ownership). Lets consumers scale horizontally; if one fails, others take over its partitions.
- Other clients: **Kafka Connect API** (data integration) and **Kafka Streams** (stream processing) — built on producers/consumers.

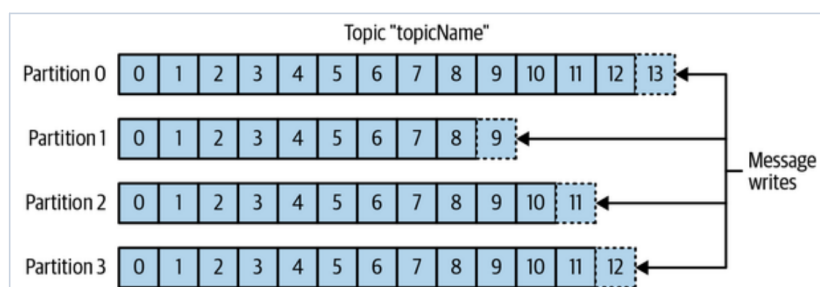


Fig 1-5. A topic with multiple partitions (writes appended to the end of each) → See in book · Part 1 p.9

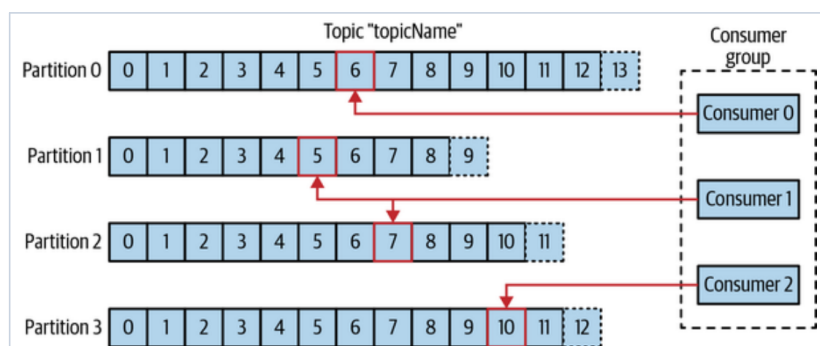


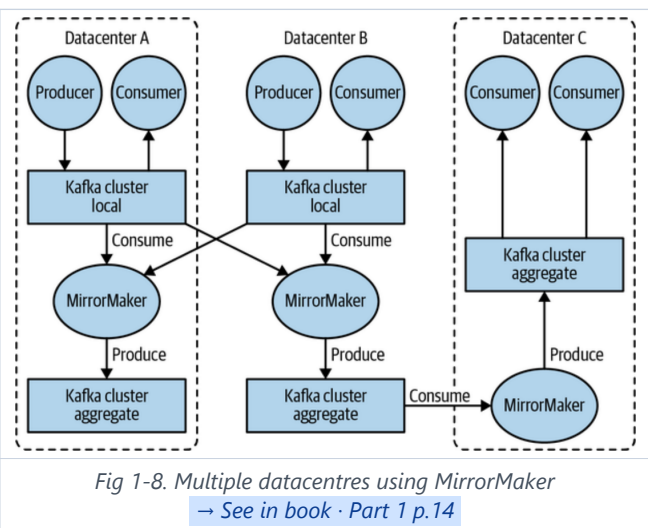
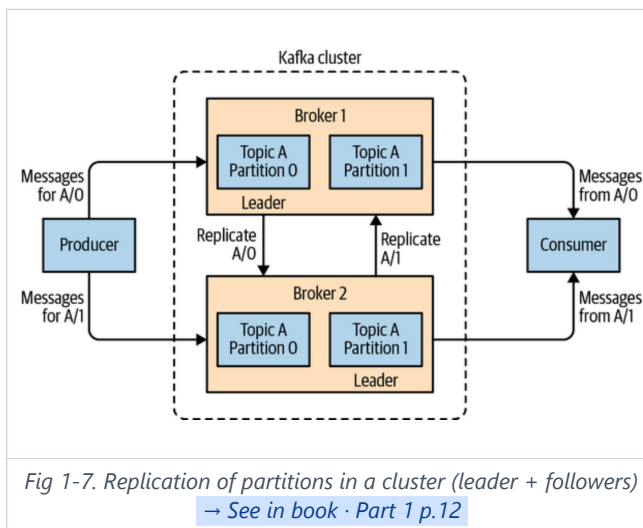
Fig 1-6. A consumer group reading from a topic (3 consumers, 4 partitions) → See in book · Part 1 p.11

A4. Brokers, clusters, retention, MirrorMaker

→ Brokers & Clusters · Part 1 p.11 → Retention · Part 1 p.12 → Multiple clusters / MirrorMaker · Part 1 p.13 → Fig 1-7, 1-8 · Part 1 p.12

- Broker** = a single Kafka server. Receives from producers, **assigns offsets, writes to disk**; serves consumer fetch requests. One broker can handle **thousands of partitions and millions of messages/sec**.

- Brokers form a **cluster**. One broker is automatically elected **cluster controller** → assigns partitions to brokers and monitors broker failures.
- A partition is owned by one broker = **leader**; copies on other brokers = **followers** (replication). **Producers must write to the leader**; consumers may read from the leader **or a follower**.
- Broker crash**: a follower (an in-sync replica) is promoted to leader; clients reconnect and consumers resume from the last committed offset.
- Retention** = durable storage for a period (e.g. **7 days**) **or** size (e.g. **1 GB** per partition); then messages expire. Configurable **per topic**.
- Log compaction**: Kafka keeps only the **last message per key** — ideal for changelog data.
- Multiple clusters** are used for: segregating data types, security isolation, and **multiple datacentres (disaster recovery)**.
- Replication inside Kafka works **only within one cluster**. Between clusters use **MirrorMaker** = simply a **Kafka consumer and producer linked by a queue**: consumes from cluster 1, produces to cluster 2.



★ IMPORTANT Model-QP question: "Role of MirrorMaker?"

- Copies data **between Kafka clusters / datacentres** (intra-cluster replication cannot). Acts as consumer of source cluster + producer to target. Used for aggregation of local clusters into a central aggregate cluster, DR, and sharing user activity across sites.

A5. Why Kafka? (Syllabus topic)

→ Book · Part 1 p.14 pp.14–17

★ IMPORTANT Learn all six — asked as "give any two reasons to support Kafka"

- 1. Multiple producers** — many front-end systems write to the same topic in a common format; consumers see one consistent stream (e.g. page views from many microservices).
- 2. Multiple consumers** — many clients read the same stream **without interfering**; unlike queues where a message is gone once consumed. Consumers can share a stream in a group so each message is processed once per group.
- 3. Disk-based retention** — consumers need not be real-time; slow consumers/bursts/maintenance cause **no data loss**; restart and continue where they left off. Per-topic rules.
- 4. Scalable** — start with 1 broker → 3-broker dev cluster → tens/hundreds in production; expand **online without downtime**; higher replication factor tolerates more failures.
- 5. High performance** — producers, consumers and brokers scale out; **sub-second latency** from produce to consume under high load.
- 6. Platform features** — Kafka Connect and Kafka Streams APIs/libraries (see A6).

Feature	Traditional queue	Kafka
After a message is consumed	Gone for others	Remains (retention); every consumer group can read it
Consumer position	Broker tracks acks per message	Consumer tracks offset ; commits last processed
Scaling reads	Limited	Add consumers to a group (up to #partitions)

Feature	Traditional queue	Kafka
Storage	Transient	Durable, ordered log on disk

A6. Kafka Ecosystem (data ecosystem) & use cases

→ Platform features · Part 1 p.16 → Data Ecosystem + Fig 1-9 · Part 1 p.17 → Use cases · Part 1 p.18

- **Inputs** = applications that create data; **outputs** = metrics, reports, data products. **Loops**: read from the system, transform with other data, write back.
- Kafka is the "**circulatory system**" of the data ecosystem: carries messages between all members with **one consistent interface**. With a schema system, producers and consumers need **no tight coupling or direct connections**; components can be added/removed freely, producers need not know who consumes.
- **Kafka Connect**: pulls data from a source system into Kafka / pushes from Kafka into a sink system. **Kafka Streams**: library for scalable, fault-tolerant stream-processing apps.
- Surrounding projects: **Cruise Control**, **Kafka Monitor**, **Burrow** (LinkedIn); **ksqlDB**, **Schema Registry**, **REST proxy** (Confluent); Hadoop, Elasticsearch as downstream consumers.

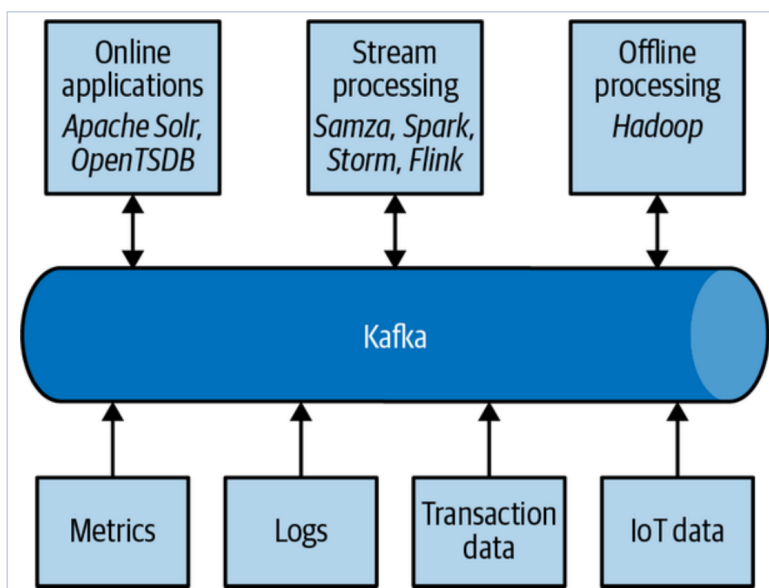


Fig 1-9. A big-data ecosystem with Kafka in the centre → See in book · Part 1 p.18

★ IMPORTANT Use cases — one line each

- **Activity tracking** (original LinkedIn use): page views, clicks, profile updates → topics → reports, ML, search updates.
- **Messaging**: apps just produce; one app formats (**decorates**), batches multiple messages into one notification, applies user preferences.
- **Metrics & logging**: many apps produce same message type; consumed by monitoring/alerting, Hadoop (long-term), Elasticsearch/security tools. Changing the destination needs no change in front ends.
- **Commit log**: publish DB changes; replicate to remote systems; consolidate changes; use **log-compacted** topics.
- **Stream processing**: like Hadoop MapReduce but **in real time** (counting metrics, partitioning, joining sources).

Producer → broker (queue) → consumer (asked in CIE-2 / SEE): the producer sends a message to a broker; the broker stores it in a topic partition (the queue / log); the consumer pulls it. This **message-queuing tier** decouples producers from consumers, buffers bursts (back-pressure), and lets each side fail or scale independently. Draw Fig 1-3 / 1-6 combined.

A7. Kafka Origin (LinkedIn)

→ Origin · Part 1 p.21 → Birth of Kafka · Part 1 p.23 → Open source / Commercial · Part 1 p.24

LinkedIn had two broken systems	Faults
Metrics system (CPU, app performance, request tracing)	Polling-based collection; large intervals between metrics; owners could not manage own metrics; high-touch (manual work); inconsistent metric names .
User-activity tracking (HTTP service, XML batches from front ends)	Inconsistent, costly-to-parse XML; changing tracked activity needed coordinated work; broke on schema changes; hourly batch → no real time.

- The two could **not share one backend** (polling vs push, clunky vs fragile) although correlating them was desirable (a drop in some user activity can reveal an app problem — but hours of delay hid it).
- Off-the-shelf options were tried: **ActiveMQ** prototype could not handle the scale and its brokers **paused**, backing up clients → decision to build **custom infrastructure**.
- Team led by **Jay Kreps** (earlier built Voldemort), with **Neha Narkhede** and later **Jun Rao**.

★ **IMPORTANT Kafka's four design goals (frequently asked)**

- **Decouple** producers and consumers using a **push-pull** model.
- Provide **persistence** of message data so **multiple consumers** can read it.
- Optimise for **high throughput**.
- Allow **horizontal scaling** as data streams grow.
- Result: messaging-style interface + **log-aggregation-style storage**; with **Avro** it handled metrics + activity tracking at **billions of messages/day** (LinkedIn: >7 trillion msgs produced, >5 PB consumed daily, as of Feb 2020).

When	Milestone
Late 2010	Released as open source on GitHub
July 2011	Accepted as Apache Software Foundation incubator project
Oct 2012	Graduated from incubator
Fall 2014	Kreps, Narkhede, Rao leave LinkedIn to found Confluent (support, training, managed Kafka on GCP/AWS/Azure; ksqldb, schema registry, REST proxy)
2016 →	Kafka Summit conferences (US & London)

Name: chosen by Jay Kreps because Kafka is optimised for *writing*, so a writer's name (Franz Kafka) seemed fitting — "basically there is not much of a relationship". Users today include Netflix and Uber. [→ Name · Part 1 p.25](#)

Part B — Kafka Producers

Chapter 3 · Writing Messages to Kafka · Textbook Part 2, pp.1–43

B1. Producer overview and components

→ Overview · Part 2 p.2 → Fig 3-1 flow · Part 2 p.4

- Every use of Kafka = a **producer** writing, a **consumer** reading, or an app doing both (e.g. credit-card system: store → Kafka → rules engine → Kafka → store/DB).
- Requirements differ: **can we lose messages? duplicates OK? latency / throughput limits?** Credit-card: never lose/duplicate, <500 ms, ~1 M msgs/s. Click-stream: some loss/duplicates OK, latency can be higher.
- Third-party clients exist via Kafka's **binary wire protocol** (Python, C++, Go ...).

★ IMPORTANT Producer data flow (draw Fig 3-1) — CIE-1 2022

1. Create a **ProducerRecord**: **topic + value** are mandatory; *[partition]*, *[key]*, *[timestamp]*, *[headers]* optional.
2. **send()** → **Serializer** converts key and value to **byte arrays**.
3. If no partition given → **Partitioner** picks one (usually from the key).
4. Record is added to a **batch** for that topic-partition (in an in-memory buffer).
5. A **separate sender thread** ships batches to the right brokers.
6. Broker replies: success → **RecordMetadata** (topic, partition, offset); failure → error → **retry** if possible, else exception.

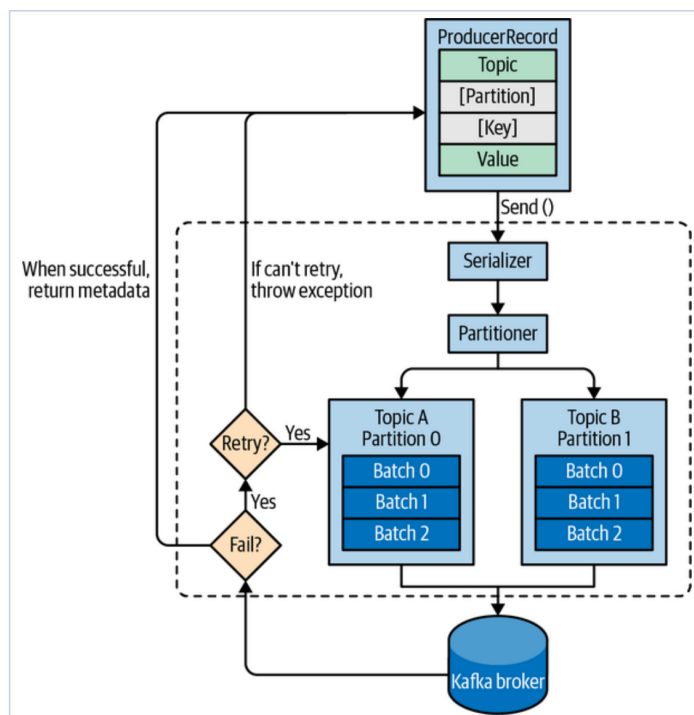


Fig 3-1. High-level overview of Kafka producer components → See in book · Part 2 p.4

B2. Constructing a producer — mandatory properties

→ Book · Part 2 p.5 pp.5–7

Property	Purpose
bootstrap.servers	List of host:port of brokers for the initial connection . Need not list all brokers (more info is fetched after connecting); give at least two in case one is down.
key.serializer	Class implementing Serializer that turns the key object into bytes. Required even if you only send values (use Void type + VoidSerializer). Built-ins: ByteArray , String , Integer ...
value.serializer	Same, for the value.

```
Properties kafkaProps = new Properties();
kafkaProps.put("bootstrap.servers", "broker1:9092,broker2:9092");
kafkaProps.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
kafkaProps.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
producer = new KafkaProducer<String, String>(kafkaProps);
```

B3. Three ways to send a message

→ Book · Part 2 p.7 → Fire & forget · Part 2 p.8 → Sync · Part 2 p.9 → Async + callback · Part 2 p.11

★ **IMPORTANT SEE 2022 fill-up: "The three primary methods of sending messages are ____"**

- **1. Fire-and-forget** — send and ignore result. Fastest; message can be **lost silently** on non-retriable error/timeout.
- **2. Synchronous send** — `producer.send(record).get()` waits on the **Future**; catches errors and gets RecordMetadata. **Poor throughput** (thread idles 2 ms—seconds per request) so rarely used in production.
- **3. Asynchronous send** — `send(record, callback); onCompletion(RecordMetadata, Exception)` fires on response. High throughput + error handling. **Recommended.**
- Technically the producer is **always asynchronous**; `get()` just waits for the Future.
- Async example: RTT 10 ms → 100 sync sends ≈ **1 s**; async sends barely take time.
- A producer object is **thread-safe** and can be shared by many threads.
- **Retriable errors** (connection lost, *not leader for partition*) → producer retries automatically. **Non-retriable** (e.g. *message size too large*) → exception returned immediately.
- Exceptions before sending: **SerializationException**, **BufferExhaustedException/TimeoutException** (buffer full), **InterruptedException**.
- Callbacks run in the producer's **main thread** (order preserved per partition) → keep them **fast / non-blocking**.

B4. Important producer configuration

→ Configs start · Part 2 p.12 → acks · Part 2 p.13 → Delivery time · Part 2 p.15 → linger/batch · Part 2 p.19 → idempotence · Part 2 p.22

★ **IMPORTANT acks — how many replicas must receive the record before the write is "successful"**

- **acks=0**: producer does **not wait**. Highest throughput, **may lose data silently**.
- **acks=1** (default before Kafka 3.0): success once the **leader** has it. Loss possible if leader crashes before replicating.
- **acks=all**: success once **all in-sync replicas** have it. **Safest**, highest latency.
- End-to-end latency is the **same** for all three, because consumers can only read records already written to all in-sync replicas. (Reliability trades off only *producer* latency.)
- **At-least-once delivery**: `acks=all + retries` / `large delivery.timeout.ms`. Add `enable.idempotence=true` to remove duplicates (SEE 2022 fill-up).

Config	What it does	Notes
client.id	Logical name of the client app	Used in logs, metrics, quotas ; makes troubleshooting easy
max.block.ms	How long <code>send()</code> / <code>partitionsFor()</code> may block (buffer full / no metadata)	Then timeout exception
delivery.timeout.ms	Total time from batch-ready until success/give-up including retries	Must be > <code>linger.ms</code> + <code>request.timeout.ms</code> . Prefer this over retries (e.g. 120 s for ~30 s leader election)
request.timeout.ms	Wait per request for a broker reply	Excludes retries
retries / retry.backoff.ms	Number of retries / wait between (default 100 ms)	Not recommended to tune; <code>retries=0</code> is the only way to disable retries
linger.ms	Wait for more messages before sending the batch	Default 0. >0 → slightly more latency, much higher throughput , better compression
batch.size	Memory (bytes , not messages) per batch	Sends half-full batches too; big = more memory, small = overhead
buffer.memory	Total memory buffering unsent records	When full, <code>send()</code> blocks <code>max.block.ms</code> then throws
compression.type	none (default), snappy, gzip, lz4, zstd	Snappy : low CPU, good speed. Gzip : more CPU, best ratio (bandwidth-limited)

Config	What it does	Notes
max.in.flight.requests.per.connection	Unacknowledged batches allowed at once	Default 5; more = throughput & memory
max.request.size	Caps size of a request / largest message (default 1 MB)	Match broker's <code>message.max.bytes</code>
send/receive.buffer.bytes	TCP socket buffers (-1 = OS default)	Increase for cross-datacentre links
enable.idempotence	Producer adds a sequence number to each record; broker rejects duplicates	Needs acks=all, retries>0, max.in.flight ≤ 5; keeps order with up to 5 in flight

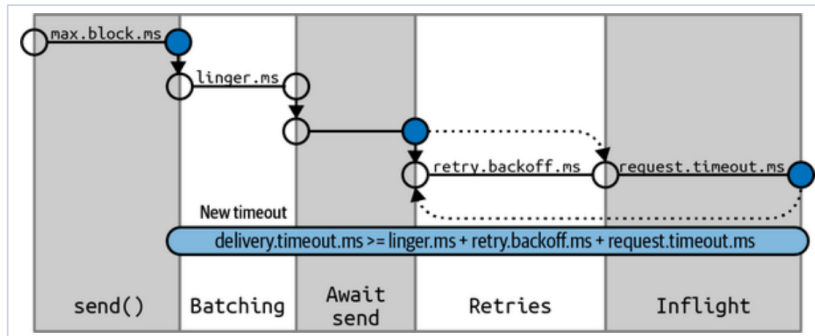


Fig 3-2. Delivery-time breakdown inside the producer (send() → batch → callback) → See in book · Part 2 p.16

★ IMPORTANT Ordering & duplicates (frequently confused)

- Kafka preserves order **within a partition**. With **retries > 0** and **max.in.flight > 1**, batch 1 may fail, batch 2 succeed, batch 1 retried → **order reversed**.
- **Duplicate case**: broker writes + replicates the record but crashes before replying; the producer times out and retries to the new leader → two copies.
- **Fix**: `enable.idempotence=true` → sequence numbers; broker rejects the 2nd copy (harmless `DuplicateSequenceException`) and order is guaranteed with ≤5 in-flight. This is a building block of **exactly-once** (Kafka ≥ 0.11).

B5. Serializers

→ Serializers · Part 2 p.23 → Custom serializer code · Part 2 p.24

- Brokers store **bytes**; serializers convert objects → bytes. Built-ins: String, Integer, ByteArray ...
- **Custom serializer** (implements `Serializer<T>`: configure / serialize / close) is **fragile**: changing a field type or adding a field breaks old vs new messages; debugging means comparing raw bytes; all teams must change code simultaneously.
- → Use a generic library: **JSON, Avro, Thrift, Protobuf** (Avro most common with Kafka).

B6. Avro + Schema Registry

→ Avro · Part 2 p.26 → Schema Registry + Fig 3-3 · Part 2 p.29 → Avro producer code · Part 2 p.30

- **Avro**: language-neutral serialization format (created by **Doug Cutting**). Schema written in **JSON**; data usually **binary**.
- Big benefit: writer can move to a **new, compatible schema** and readers keep working with **no change**. Example: field `faxNumber` replaced by `email` (both with default null) → old readers get **null** for email, new readers get **null** for fax; no exceptions.
- Two caveats: schemas must be **compatible**; the deserializer needs **access to the writing schema**.
- Storing the whole schema in every record would more than **double** record size → use a **Schema Registry** (not part of Apache Kafka; e.g. Confluent): store all schemas there, put only the **schema ID** in each record. Serializer registers/fetches it; the deserializer looks it up by ID.
- Producer config: `KafkaAvroSerializer` for key/value + `schema.registry.url`. Use generated Avro classes (not POJOs) or **GenericRecord** + schema.

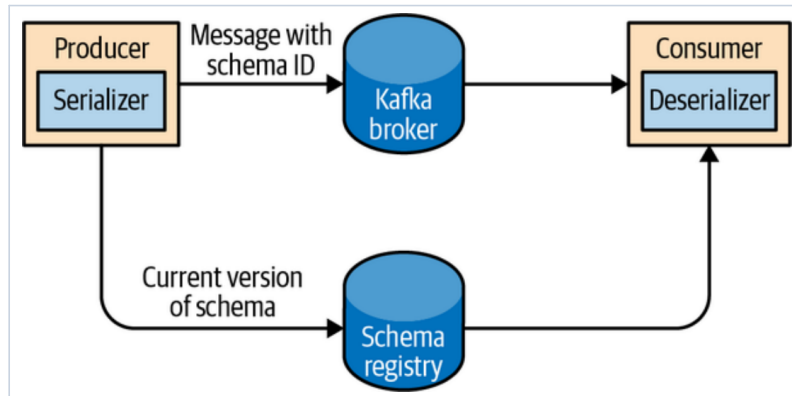


Fig 3-3. Serialization and deserialization of Avro records using the Schema Registry → See in book · Part 2 p.30

B7. Partitions & partitioners

→ Partitions · Part 2 p.33 → Sticky null-key · Part 2 p.34 → Custom partitioner code · Part 2 p.35

★ IMPORTANT Partitioner rules

- **Key = null** (default partitioner): spreads records over available partitions; since Kafka 2.4 the round-robin is **sticky** — fills one batch for a partition before moving on → fewer requests, lower latency & CPU.
- **Key present**: Kafka **hashes the key** (own hash, murmur2) and maps it onto **all** partitions (not just available ones) → **same key** → **same partition**. If that partition is down you may get an error.
- **Mapping holds only while partition count is constant**. Adding partitions later breaks it → create topics with enough partitions and never add.
- Alternatives: **RoundRobinPartitioner**, **UniformStickyPartitioner** (even load even with keys / skewed keys).
- **Custom partitioner** implements **Partitioner** (configure/partition/close). Example "Banana": a customer with > 10% of traffic is sent to the **last partition**; the rest are hashed over the others.

B8. Headers, interceptors, quotas

→ Headers · Part 2 p.37 → Interceptors · Part 2 p.37 → Quotas & throttling · Part 2 p.40

Feature	Key points
Headers	Ordered key/value pairs (key = String, value = any serialized object) holding metadata without touching the key/value . Uses: lineage (source of data), routing/tracing without parsing (even if encrypted). <code>record.headers().add("privacy-level", bytes)</code>
Interceptors	ProducerInterceptor changes behaviour without modifying app code (just add the jar + <code>interceptor.classes</code> config). <code>onSend(record)</code> — before serialization, may modify and must return a record; <code>onAcknowledgement(metadata, exception)</code> — on broker ack, cannot modify. Uses: monitoring/tracing, standard headers, redacting sensitive data .
Quotas	Brokers limit rates. 3 types : produce , consume (bytes/sec) and request (% of broker processing time). Defaults e.g. <code>quota.producer.default=2M</code> ; per-client override; dynamic via <code>kafka-configs.sh/AdminClient</code> (static ones need broker restart). On breach the broker throttles (delays responses and mutes the channel). Visible via <code>produce-throttle-time-avg/max</code> metrics.

REMEMBER Warning from the book

- If an async producer keeps sending faster than the broker accepts, records queue in client memory → buffer fills → `send()` blocks → **TimeoutException** (or batches expire after `delivery.timeout.ms`). Plan and monitor broker capacity.

B9. Exam-ready producer program (Python)

CIE-1 2022 asked a producer for an orders topic with **high throughput and no duplicate orders**. Same idea in confluent-kafka:

```

from confluent_kafka import Producer
import json

conf = {
    "bootstrap.servers": "localhost:9092", # initial broker list
    "acks": "all", # wait for all in-sync replicas
    "enable.idempotence": True, # no duplicates, keeps order
    "linger.ms": 20, # small wait -> bigger batches
    "batch.size": 65536, # throughput
    "compression.type": "snappy", # less network, little CPU
}
producer = Producer(conf)
TOPIC = "orders"

def on_delivery(err, msg): # asynchronous callback
    if err is not None:
        print("Delivery failed:", err)
    else:
        print(f"Delivered to {msg.topic()} [{msg.partition()}] @ offset {msg.offset()}")

order = {"order_id": 101, "item": "laptop", "qty": 1}
producer.produce(TOPIC, key=str(order["order_id"]), # same key -> same partition
                 value=json.dumps(order), callback=on_delivery)
producer.poll(0) # serve delivery callbacks
producer.flush() # block until all messages are sent

```

EXAM TIP Say in the answer

- acks=all + idempotence → no loss, no duplicates; linger.ms + batch.size + compression → throughput; key → per-key ordering; callback → async error handling.

Part C — Kafka Consumers

Chapter 4 · Reading Data from Kafka · Textbook Part 2, pp.45–98

C1. Consumers and consumer groups

→ Book · Part 2 p.45 → Fig 4-1 → 4-5 · Part 2 p.47

- A single consumer can **fall behind** if producers write faster than it processes → need to **scale consumption** by splitting a topic among several consumers.
- Consumers normally belong to a **consumer group** (group.id). In a group, **each consumer reads a different subset of the topic's partitions**.

★ IMPORTANT The four rules (draw Figs 4-1 to 4-5)

- **1 consumer** in group G1, topic T1 with 4 partitions → it gets **all 4** partitions (Fig 4-1).
- **2 consumers** → 2 partitions each (Fig 4-2). **4 consumers** → 1 each (Fig 4-3).
- **More consumers than partitions** → the extra consumers are **idle** (Fig 4-4). Never use more consumers than partitions → create topics with many partitions up front.
- **Each application needs its own consumer group** to receive **all** messages (Fig 4-5): G2 gets everything independently of G1. Within one group each partition has **one owner**.
- To **scale** an app: add consumers to the same group. To **add another app**: new group. Kafka scales to many groups without performance loss.

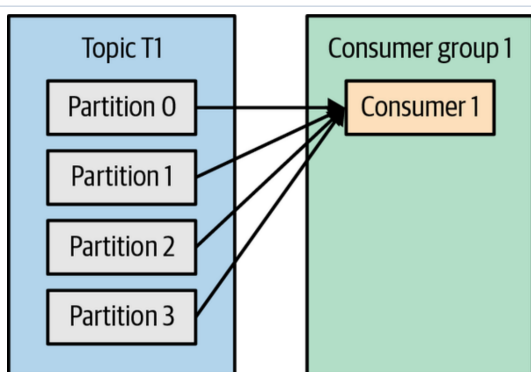


Fig 4-1. One consumer, four partitions
→ See in book · Part 2 p.47

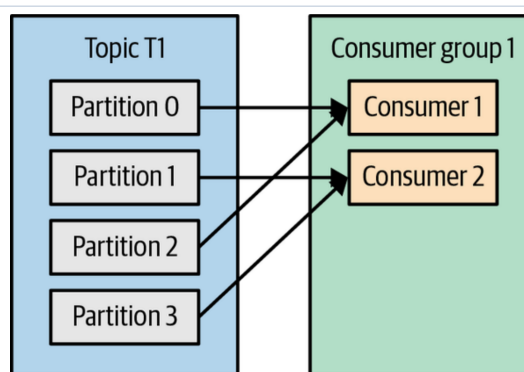


Fig 4-2. Four partitions split over two consumers
→ See in book · Part 2 p.48

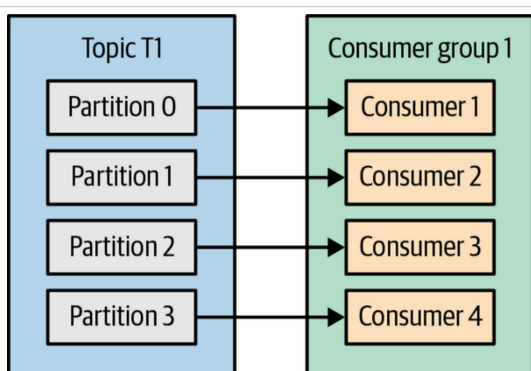


Fig 4-3. Four consumers, one partition each
→ See in book · Part 2 p.49

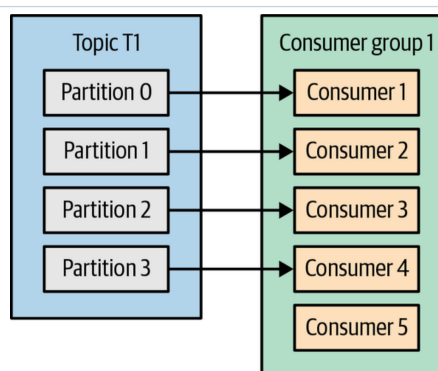


Fig 4-4. More consumers than partitions → idle consumer
→ See in book · Part 2 p.50

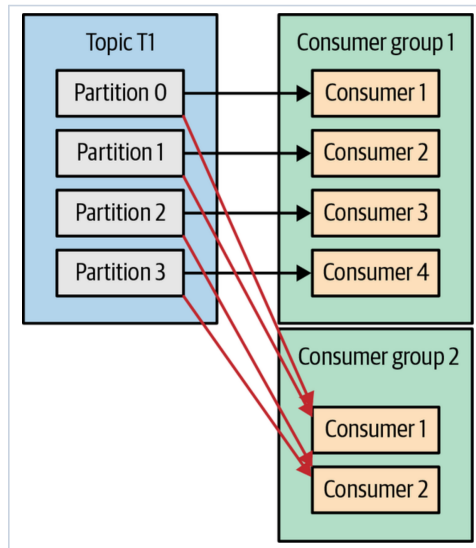


Fig 4-5. Adding a new consumer group — both groups receive all messages → See in book · Part 2 p.52

C2. Consumer-group rebalance

→ Rebalance · Part 2 p.53 → Heartbeats / coordinator · Part 2 p.55 → Static membership · Part 2 p.56

- **Rebalance** = moving partition ownership between consumers. Triggers: **new consumer joins**, consumer **shuts down / crashes**, or the topic changes (e.g. **new partitions** added).
- Good: gives **high availability and scalability**. Bad: during a rebalance consumption pauses.

	Eager (stop-the-world) rebalance	Cooperative (incremental) rebalance
What happens	All consumers stop, revoke all partitions , rejoin, receive a brand-new assignment	Only a small subset of partitions moves; others keep being consumed
Phases	2: (1) everyone gives up partitions, (2) new assignment	2 or more: leader announces which partitions will be lost, those are released, then reassigned (may take a few iterations)
Impact	Short window of unavailability of the whole group ; grows with group size	No complete "stop the world"; matters most for large groups
Assignor	Range, RoundRobin, Sticky	CooperativeStickyAssignor

CIE-1 2022: "two differences between early and cooperative rebalance" — use rows 1 & 3 (what is revoked; availability impact).

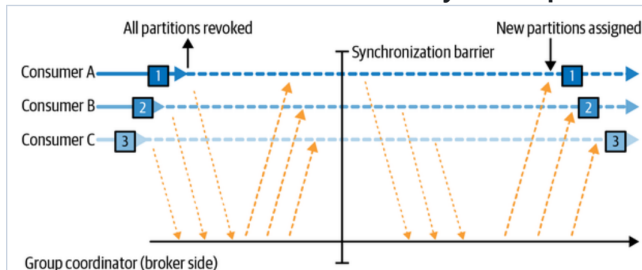


Fig 4-6. Eager: revoke all, pause, reassign

→ See in book · Part 2 p.54

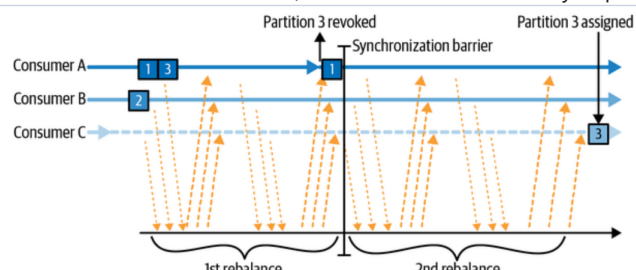


Fig 4-7. Cooperative: pause only moved partitions

→ See in book · Part 2 p.55

★ IMPORTANT Heartbeats, group coordinator, group leader

- Consumers send **heartbeats** (background thread) to a broker acting as the **group coordinator**. Heartbeats on time = alive.
- No heartbeat for `session.timeout.ms` → coordinator declares it dead → **rebalance**. A clean `close()` notifies the coordinator → immediate rebalance (smaller gap).
- **Join process**: consumer sends **JoinGroup** to the coordinator. The **first** consumer to join becomes the **group leader**; it gets the member list and, using a **PartitionAssignor**, decides who gets which partitions, then sends the plan to the coordinator which tells each member (each consumer sees only its own assignment).
- **Static membership**: set a unique `group.instance.id`. A restarting static member keeps its **same partitions without a rebalance** (good for local state/cache). Cost: partitions sit idle until it returns or `session.timeout.ms` expires; duplicate instance id → error.

C3. Creating a consumer, subscribing, the poll loop

→ Creating · Part 2 p.58 → Subscribe · Part 2 p.59 → Poll loop · Part 2 p.60 → Thread safety · Part 2 p.62

Mandatory / common property	Purpose
bootstrap.servers	Initial connection string to the cluster (same as producer)
key.deserializer, value.deserializer	Classes that turn byte arrays back to objects (mirror of serializers)
group.id	Consumer group name (not strictly mandatory but used almost always)

- `consumer.subscribe(Collections.singletonList("topic"))`; or a **regex** `subscribe(Pattern.compile("test.*"))` — a new matching topic triggers a rebalance. Regex filtering is **client-side** (costly with ~30,000+ partitions; needs describe permission on all topics).
- **Poll loop** is the heart of the Consumer API: `while(true){ records = consumer.poll(Duration.ofMillis(100)); for(record : records){ process } }`.

★ IMPORTANT "Consumers must keep polling or they are considered dead"

- The **first poll()** finds the GroupCoordinator, joins the group and gets the partition assignment. Rebalances and callbacks are handled **inside poll()**.
- If `poll()` is not called for longer than `max.poll.interval.ms` (default 5 min) the consumer is evicted and its partitions reassigned.
- Each ConsumerRecord holds **topic, partition, offset, key, value**. `poll(timeout)` = how long to block when no data is buffered.
- **Thread safety:** one consumer per thread. Cannot share a consumer across threads. Run multiple consumers via `ExecutorService`, or one consumer + a worker-thread queue. `wakeup()` is the only method safe to call from another thread.

C4. Important consumer configuration

→ Configs · Part 2 p.63 → session / heartbeat · Part 2 p.66 → auto.offset.reset · Part 2 p.68 → enable.auto.commit · Part 2 p.69

Config	Meaning / default	Why it matters
fetch.min.bytes	Min data broker returns per fetch (default 1 B)	Raise to cut CPU/broker load on low-traffic topics (adds latency)
fetch.max.wait.ms	Max wait for fetch.min.bytes (default 500 ms)	Caps added latency. Broker answers when min bytes OR max wait is reached first
fetch.max.bytes	Max bytes per poll (default 50 MB)	Limits consumer memory
max.poll.records	Max number of records per poll()	Controls processing time per loop
max.partition.fetch.bytes	Max bytes per partition (default 1 MB)	Book recommends fetch.max.bytes instead
session.timeout.ms	Time without heartbeat before dead (default 10 s)	Lower = faster failure detection but spurious rebalances
heartbeat.interval.ms	How often heartbeats are sent	Must be < session timeout, usually 1/3 of it
max.poll.interval.ms	Max gap between polls (default 5 min)	Catches a hung main thread even if heartbeats continue
default.api.timeout.ms	Timeout for most API calls (default 1 min)	<code>poll()</code> always takes an explicit timeout
request.timeout.ms	Wait for broker reply (default 30 s)	Do not lower (avoid reconnect overhead)
auto.offset.reset	What to do with no valid committed offset: latest (default), earliest , none (throw)	earliest = read from the beginning; latest = only new records
enable.auto.commit	Auto-commit offsets (default true); auto.commit.interval.ms = 5 s	Set false for manual control (fewer duplicates / no loss)
partition.assignment.strategy	Range (default), RoundRobin, Sticky, CooperativeSticky	See C5
client.rack	Zone of the consumer	Fetch from nearest replica (needs RackAwareReplicaSelector)
group.instance.id	Unique id for static membership	Avoids rebalance on restart
offsets.retention.minutes	(Broker) offsets of an empty group kept 7 days	After that the group behaves like new

C5. Partition assignment strategies + worked example

→ Assignors · Part 2 p.69

Assignor	How it works	Remarks
Range (default)	For each topic separately gives each consumer a consecutive range of partitions	If partitions don't divide evenly the first consumers get more ; bias accumulates over many topics
RoundRobin	Takes all partitions of all topics and deals them out one by one	Counts differ by at most 1 when all consumers subscribe to the same topics
Sticky	As balanced as possible and on rebalance keeps as many existing assignments as possible	Fewer partition moves; better balance than RoundRobin when subscriptions differ
CooperativeSticky	Same as Sticky but supports cooperative rebalance	Consumers keep consuming non-moved partitions

★ IMPORTANT Worked example — CIE-1 2022 & SEE 2022: topic Sales_data, 6 partitions P0–P5, group with C1, C2

- (i) **RangeAssignor with 2 consumers** ($6 / 2 = 3$ each, contiguous): **C1 = P0, P1, P2; C2 = P3, P4, P5**.
- (ii) **Add C3** → rebalance → 2 each: **C1 = P0, P1; C2 = P2, P3; C3 = P4, P5**.
- **Issue:** a rebalance is triggered; with the (eager) default all consumers **stop and revoke everything** (brief unavailability) and even C2 loses P4/P5 and gets P2/P3 — many partitions move; any **local state/cache** tied to partitions is lost. With a different partition count Range can also be **uneven** (e.g. 7 partitions → 3/2/2 in favour of earlier consumers) and across many topics the first consumer is overloaded.
- **Remedies:** use **Sticky / CooperativeSticky** (minimal movement, no stop-the-world), RoundRobin for even spread (would give C1 = P0,P3; C2 = P1,P4; C3 = P2,P5), and/or **static membership**.

C6. Commits and offsets (asked in SEE 2022)

→ Commits & Offsets · Part 2 p.73 → Auto · Part 2 p.75 → commitSync · Part 2 p.77 → commitAsync · Part 2 p.78 → Specific offset · Part 2 p.81

- Kafka does **not** track per-message acknowledgements like JMS queues. Consumers track their **own position**. Updating the current position = an **offset commit**.
- A consumer commits by sending a message to Kafka, which writes it to the special `__consumer_offsets` topic (per partition). After a rebalance, a consumer reads the **latest committed offset** of each partition and continues from there.
- Kafka does **not commit records individually**: committing offset N implies **everything before N** was processed. Committed offset = **offset of the next message to read** (last processed + 1).

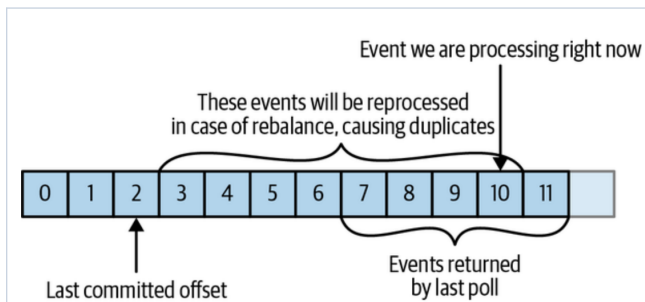


Fig 4-8. Committed offset < last processed → messages reprocessed (duplicates)

→ See in book · Part 2 p.74

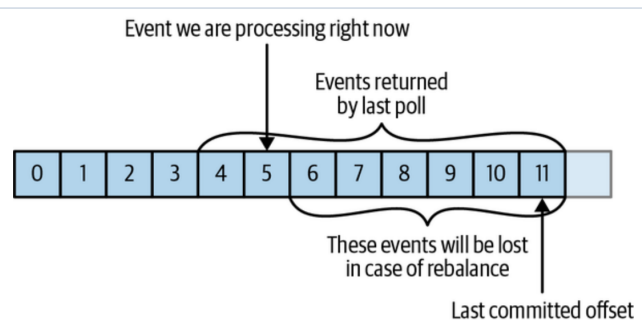


Fig 4-9. Committed offset > last processed → messages missed (lost)

→ See in book · Part 2 p.75

★ IMPORTANT Why it matters

- **Committed < processed** → messages between them are **processed twice** (duplicates).
- **Committed > processed** → messages between them are **missed** (data loss for the group).

Strategy	How	Pros	Cons
Automatic commit	<code>enable.auto.commit=true</code> ; every 5 s (auto.commit.interval.ms) the next poll() commits the last offset returned by the previous poll	Simplest	Crash 3 s after a commit → 3 s of events processed twice . Cannot eliminate duplicates. Must process all records before calling poll() again

Strategy	How	Pros	Cons
Commit current offset (commitSync)	<code>enable.auto.commit=false</code> ; call <code>commitSync()</code> after the batch is processed. Retries until success or unrecoverable error	Most reliable	App is blocked until the broker replies → lower throughput. Commit before finishing = risk of loss
Asynchronous commit (commitAsync)	Send commit and continue; optional <code>OffsetCommitCallback</code>	Higher throughput	No retry (a retry of offset 2000 could overwrite committed 3000 → duplicates). Use a sequence number if retrying
Combine sync + async	<code>commitAsync()</code> in the loop, <code>commitSync()</code> in finally before close/rebalance	Fast and safe at shutdown	Slightly more code
Commit specific offsets	Pass a <code>Map<TopicPartition, OffsetAndMetadata></code> e.g. every 1000 records; commit offset+1	Commit mid-batch	Must track offsets for every partition

🔪 EXAM TIP Exam tip — "any two offset commit strategies"

- Pick **automatic** and **manual synchronous** (or async). For each: how it is set, one advantage, one drawback (duplicates vs blocking). Draw Fig 4-8 / 4-9 to show duplicates/missed messages.

```
while (true) {
    ConsumerRecords<String,String> records = consumer.poll(Duration.ofMillis(100));
    for (ConsumerRecord<String,String> r : records) { process(r); }
    consumer.commitAsync(); // fast; next commit acts as retry
}
...
finally { consumer.commitSync(); consumer.close(); } // last commit must succeed
```

C7. Rebalance listeners, seeking, clean exit

→ Rebalance listener · Part 2 p.83 → Seek · Part 2 p.87 → Exit / wakeup · Part 2 p.88

ConsumerRebalanceListener method	When called / what to do
<code>onPartitionsAssigned</code>	After reassignment, before consuming. Load state, seek to offsets (must return within max.poll timeout)
<code>onPartitionsRevoked</code>	When giving up partitions (eager: before rebalance starts after consumption stops; cooperative: at end, only for the revoked subset). Commit offsets here , close resources
<code>onPartitionsLost</code>	Cooperative only, exceptional case: partitions were taken without being revoked first ; clean up carefully (new owner may already hold state)

- Pass the listener to `subscribe(topics, listener)`. Typical use: `commitSync(currentOffsets)` in `onPartitionsRevoked`.
- **Consuming from specific offsets:** `seekToBeginning()`, `seekToEnd()`, `seek(tp, offset)`; use `offsetsForTimes()` (timestamp index) to rewind to e.g. one hour ago — useful to recover a lost file or skip ahead when lagging.
- **Clean exit:** another thread (e.g. ShutdownHook) calls `consumer.wakeup()` → `poll()` throws **WakeupException** (ignore it) → finally call `consumer.close()`, which **commits offsets** and tells the coordinator it is leaving → **immediate rebalance**.

C8. Deserializers

→ Deserializers · Part 2 p.90 → Custom · Part 2 p.92 → Avro deserializer · Part 2 p.94

- Mirror of serializers: **bytes** → **objects**. The deserializer **must match** the serializer (IntSerializer + StringDeserializer fails).
- **Custom deserializer** (implements `Deserializer<T>`) **tightly couples** producers and consumers; fragile — **not recommended**. Use JSON / Thrift / Protobuf / Avro.
- **Avro:** set `value.deserializer=KafkaAvroDeserializer`, `schema.registry.url`, and `specific.avro.reader=true` to get the generated class. Incompatibilities are caught with clear errors instead of debugging raw bytes.

C9. Standalone consumer (no group) + exam-ready consumer program

→ Standalone consumer · Part 2 p.96

- Use when a single consumer must read **all partitions** (or specific ones) — no groups, no rebalances. **Do not subscribe; instead assign()** partitions (get them via `partitionsFor(topic)`). A consumer either **subscribes OR assigns**, not both.
- `group.id` is still needed to commit offsets. Trade-off: the consumer is **not notified when partitions are added** — recheck `partitionsFor()` periodically or restart.

CIE-1 / CIE-2 2022: consume a **JSON** topic `sales_data` and auto-convert to a Python dictionary (kafka-python):

```
from kafka import KafkaConsumer
import json

def json_deserializer(data):
    # bytes -> dict
    return json.loads(data.decode("utf-8"))

consumer = KafkaConsumer(
    "sales_data", # topic
    bootstrap_servers="localhost:9092",
    group_id="sales_group", # consumer group
    auto_offset_reset="earliest", # no committed offset -> from beginning
    enable_auto_commit=True, # auto commit every 5 s
    value_deserializer=json_deserializer, # automatic conversion
)
print("Listening on 'sales_data' ...")
for message in consumer: # the poll loop
    print(f"Topic: {message.topic}, Partition: {message.partition}, Offset: {message.offset}")
    print("Value (dict):", message.value)
```

EXAM TIP Say in the answer

- `group_id` → consumer group; `value_deserializer` → bytes → dict; `auto_offset_reset` → where to start; the for-loop is the poll loop; use `enable_auto_commit=False` + `consumer.commit()` for tighter control.

Part D — Rapid Revision

Cheat sheets · model answers · flash Q&A

D1. One-page cheat sheet

Concept	One-line answer
Kafka	Distributed commit-log / streaming platform; durable, ordered, replicated, scalable pub/sub
Message / key / batch	Byte array / optional byte array deciding the partition / group of messages for one topic-partition
Topic / partition	Category / one append-only log; ordering only within a partition
Offset	Increasing id of a message within its partition; consumer's bookmark
Broker / controller	One Kafka server / broker elected to manage partition assignment & failures
Leader / follower	Partition owner (producers write here) / replica copy that can take over
Retention	Time (7 d) or size (1 GB) based deletion; log compaction keeps last per key
MirrorMaker	Consumer + producer copying data between clusters / datacentres
Consumer group	Consumers sharing a topic: 1 partition = 1 consumer in a group; each group gets all messages
Rebalance	Reassigning partitions on join / leave / crash / partition change
Group coordinator / leader	Broker tracking heartbeats / first consumer, computes assignment
Heartbeat	Liveness signal; miss session.timeout.ms → dead → rebalance
Idempotent producer	Sequence numbers → broker discards duplicate retries
Schema Registry	Stores Avro schemas; records carry only the schema id
__consumer_offsets	Internal topic storing committed offsets

D2. Producer vs Consumer at a glance

	Producer	Consumer
Object	KafkaProducer / ProducerRecord	KafkaConsumer / ConsumerRecord
Mandatory config	bootstrap.servers, key.serializer, value.serializer	bootstrap.servers, key.deserializer, value.deserializer (+ group.id)
Converts	Object → bytes (serializer)	Bytes → object (deserializer)
Chooses partition	Partitioner (hash of key / sticky / custom)	Assignor (Range, RoundRobin, Sticky, CoopSticky) via group leader
Reliability knob	acks, retries/delivery.timeout.ms, idempotence	Offset commit strategy, auto.offset.reset
Throughput knob	linger.ms, batch.size, compression.type	fetch.min.bytes, max.poll.records, more consumers/partitions
Talks to	Partition leader	Leader or a follower (client.rack)
Loop	send() (fire-forget / sync / async+callback)	poll() loop + commit

D3. Model answers (outline)

★ IMPORTANT Q. What happens if a broker crashes? (CIE-1, CIE-2, SEE)

- Controller detects failure; for each partition led by that broker, a **follower from the in-sync replicas is elected leader** (needs replication factor > 1).
- Producers get *not-leader* errors, refresh metadata and **retry** (**delivery.timeout.ms** should exceed the ~30 s election time) — with acks=all no acknowledged data is lost.
- Consumers reconnect to new leaders and resume from the **last committed offset**; messages are replayed from the log (some duplicates possible).
- Retention lets the cluster keep data while the broker is repaired; capacity restored by adding a broker online.

★ IMPORTANT Q. Explain consumers and consumer groups with an example (SEE 2022)

- Define group (consumers sharing **group.id**, each partition owned by one member).
- Topic T1 with 4 partitions: C1 alone → all 4; add C2 → 2 + 2; 4 consumers → 1 each; 5th consumer idle. (Draw Figs 4-1 to 4-4.)
- Second application = **new group G2** → also receives all messages (Fig 4-5).
- Failure/join → **rebalance**, heartbeats and coordinator; offsets stored in `__consumer_offsets` so work resumes.

★ **IMPORTANT Q. Reasons to choose Kafka / data ecosystem (Model QP)**

- List six: multiple producers, multiple consumers, disk-based retention, scalable, high performance, platform features (Connect, Streams).
- Ecosystem: producers → Kafka (circulatory system) → consumers/stream processors/Hadoop/search; Schema Registry decouples them; draw Fig 1-9.

D4. Flash Q&A; — cover the right column and test yourself

#	Question	Answer
1	Unit of data in Kafka?	A message (byte array) with optional key.
2	Is ordering guaranteed across a topic?	No — only within a partition.
3	What does a producer need at minimum?	<code>bootstrap.servers</code> , <code>key.serializer</code> , <code>value.serializer</code> .
4	Name the 3 send methods.	Fire-and-forget, synchronous, asynchronous (callback).
5	<code>acks=0 / 1 / all</code> ?	No wait / leader only / all in-sync replicas.
6	How to avoid duplicate writes on retry?	<code>enable.idempotence=true</code> (sequence numbers).
7	Same key always same partition — until?	The partition count changes.
8	What does <code>linger.ms</code> do?	Waits to fill batches: +latency, +throughput.
9	Where do Avro schemas live in Kafka setups?	In a Schema Registry; messages carry the schema id.
10	Who can a producer write to?	The partition leader only.
11	3 quota types?	Produce, consume, request.
12	More consumers than partitions?	Extra consumers are idle.
13	How does a second app get all messages?	Give it its own consumer group.
14	Eager vs cooperative rebalance?	Revoke all & stop the world vs move only a subset incrementally.
15	Default assignor?	<code>RangeAssignor</code> .
16	Heartbeat vs session timeout ratio?	<code>heartbeat.interval.ms</code> ≈ 1/3 of <code>session.timeout.ms</code> .
17	What if <code>poll()</code> isn't called for <code>max.poll.interval.ms</code> ?	Consumer is considered dead; rebalance.
18	<code>auto.offset.reset</code> values?	latest (default), earliest, none.
19	Default auto-commit interval?	5 seconds (<code>auto.commit.interval.ms</code>).
20	Committed offset > processed offset?	Messages missed (lost).
21	Committed offset < processed offset?	Messages reprocessed (duplicates).
22	<code>commitSync</code> vs <code>commitAsync</code> ?	Blocks & retries vs non-blocking & no retry.
23	Where are offsets stored?	Internal topic <code>__consumer_offsets</code> .
24	Safe way to stop a consumer?	<code>wakeup()</code> from another thread, then <code>close()</code> .
25	<code>assign()</code> vs <code>subscribe()</code> ?	Manual partitions, no group/rebalance vs group-managed.
26	Four goals of Kafka at LinkedIn?	Decouple (push-pull), persistence, high throughput, horizontal scaling.
27	Who created Kafka?	Jay Kreps, Neha Narkhede, Jun Rao at LinkedIn; Confluent founded 2014.

D5. Jump back to the book

Everything in blue opens the textbook at the right page:

Part 1 (Ch.1)

→ Pub/sub · Part 1 p.2 → Messages & batches · Part 1 p.6 → Topics & partitions · Part 1 p.8 → Producers & consumers · Part 1 p.9 → Brokers & clusters · Part 1 p.11 → Why Kafka · Part 1 p.14 → Ecosystem · Part 1 p.17 → Use cases · Part 1 p.18

→ Origin · Part 1 p.21

Part 2 – Producers (Ch.3)

→ Overview · Part 2 p.2 → Constructing · Part 2 p.5 → Sending · Part 2 p.8 → Config · Part 2 p.12 → Serializers · Part 2 p.23
→ Avro · Part 2 p.26 → Partitions · Part 2 p.33 → Interceptors · Part 2 p.37 → Quotas · Part 2 p.40

Part 2 – Consumers (Ch.4)

→ Groups · Part 2 p.45 → Rebalance · Part 2 p.53 → Create / subscribe · Part 2 p.58 → Poll loop · Part 2 p.60 → Config ·
Part 2 p.63 → Commits · Part 2 p.73 → Listeners · Part 2 p.83 → Deserializers · Part 2 p.90 → Standalone · Part 2 p.96

EXAM TIP

- Revise in this order tonight: **Part D flash Q&A** → **IMPORTANT boxes** in A, B, C → the two code programs → figures. Good luck!

Kafka in Python — All Code Together

Unit IV · Producers + Consumers · Python versions of every example in the textbook

The textbook (Chapters 3 and 4) shows **Java**. Below is the same logic in **Python**. Most use confluent-kafka (the library behind the CIE solutions' producer); the two exam programs also have kafka-python versions, as used in your CIE-1 / CIE-2 2022 answer keys. Blue chips open the Java original in the textbook.

```
pip install confluent-kafka      # producer / consumer (librdkafka based)
pip install "confluent-kafka[avro]" # + Avro & Schema Registry
pip install kafka-python         # simpler pure-python client (used in CIE keys)
```

REMEMBER Index

- **Producers:** P1 basic config · P2 fire-and-forget · P3 synchronous · P4 async + callback · P5 reliable + high-throughput (EXAM) · P6 custom serializer · P7 Avro + Schema Registry · P8 custom partitioner · P9 headers · P10 interceptor-style counter · P11 JSON producer (kafka-python, EXAM)
- **Consumers:** C1 basic poll loop · C2 auto commit · C3 commit sync · C4 commit async · C5 async + sync combined · C6 commit specific offsets · C7 rebalance listener · C8 seek / offsets by time · C9 clean exit · C10 custom deserializer · C11 Avro deserializer · C12 standalone consumer (assign) · C13 assignor + static membership · C14 JSON consumer (kafka-python, EXAM) · C15 one consumer per thread

★ IMPORTANT Python ↔ Java name map

- Java bootstrap.servers etc. are the **same config keys** in a dict. send() → produce(); Callback.onCompletion → on_delivery(err, msg); Future.get() → flush().
- In confluent-kafka the callback fires inside poll() / flush(), so call producer.poll(0) regularly.
- Consumers: poll(timeout) returns **one message** (or None) in confluent-kafka; check msg.error() before using it.

Part 1 — Producer Code

Chapter 3 equivalents

P1. Create a producer (mandatory properties)

Java original: [→ Constructing · Part 2 p.5](#)

Mandatory: **bootstrap.servers**. In Python, serialization is done by you (str → bytes) or by a SerializingProducer; there is no key.serializer class setting for the plain Producer.

```
from confluent_kafka import Producer

conf = {"bootstrap.servers": "broker1:9092,broker2:9092"} # list >= 2 brokers
producer = Producer(conf)
```

P2. Fire-and-forget

Send and ignore the result (may lose messages silently). Java original: [→ Fire & forget · Part 2 p.8](#)

```
# topic, key, value (strings are encoded to bytes automatically)
producer.produce("CustomerCountry", key="Precision Products", value="France")
producer.poll(0)      # let the client do background work
producer.flush()      # make sure it leaves the buffer before exit
```

P3. Synchronous send

Waits for the broker reply (slow — rarely used in production). Java original: [→ Sync send · Part 2 p.9](#)

Python has no send().get(); the equivalent is to **flush()** after every record and check the callback result.

```

result = {}

def on_delivery(err, msg):
    result["err"], result["msg"] = err, msg

producer.produce("CustomerCountry", key="Precision Products", value="France",
                 callback=on_delivery)
producer.flush() # blocks until broker replies (or fails)

if result["err"] is not None:
    print("Send failed:", result["err"])
else:
    m = result["msg"]
    print(f"OK -> {m.topic()} [{m.partition()}] @ offset {m.offset()}")

```

P4. Asynchronous send with callback

Recommended: high throughput + error handling. Java original: [→ Async + callback · Part 2 p.11](#)

```

def on_delivery(err, msg):
    # = Callback.onCompletion(metadata, exception)
    if err is not None:
        print("Delivery failed:", err) # log / write to an errors file
    else:
        print(f"Delivered to {msg.topic()} [{msg.partition()}] offset {msg.offset()}")

for i in range(100):
    producer.produce("CustomerCountry", key=str(i), value="USA", callback=on_delivery)
    producer.poll(0) # serve callbacks without blocking
producer.flush()

```

P5. Reliable + high-throughput producer (no duplicates)

EXAM — CIE-1 2022: "high throughput and no duplicate orders". Java original: [→ Config · Part 2 p.12](#) [→ idempotence · Part 2 p.22](#)

acks=all + idempotence → no loss / no duplicates; linger.ms + batch.size + compression → throughput; key → per-key ordering.

```

from confluent_kafka import Producer
import json

conf = {
    "bootstrap.servers": "localhost:9092",
    "client.id": "order-service", # shows up in logs / quotas
    "acks": "all", # all in-sync replicas
    "enable.idempotence": True, # sequence numbers, no duplicates
    "delivery.timeout.ms": 120000, # keep retrying ~2 min (leader election)
    "linger.ms": 20, # wait to fill batches
    "batch.size": 65536, # bytes per batch
    "compression.type": "snappy",
    "max.in.flight.requests.per.connection": 5,
}
producer = Producer(conf)
TOPIC = "orders"

def on_delivery(err, msg):
    if err: print("FAILED:", err)
    else: print(f"{msg.topic()} [{msg.partition()}] @ {msg.offset()}")

order = {"order_id": 101, "item": "laptop", "qty": 1}
producer.produce(TOPIC, key=str(order["order_id"]), # same key -> same partition
                 value=json.dumps(order), callback=on_delivery)
producer.poll(0)
producer.flush()

```

P6. Custom serializer (Customer: id + name)

Java's CustomerSerializer: 4-byte id, 4-byte name length, then UTF-8 name. Java original: [→ Custom serializer · Part 2 p.24](#)

Fragile: adding a field breaks old data — prefer JSON / Avro / Protobuf (see P7).

```
import struct

class Customer:
    def __init__(self, cid, name):
        self.id, self.name = cid, name

def serialize_customer(c: Customer) -> bytes:
    if c is None:
        return None
    name = (c.name or "").encode("utf-8")
    # ">ii" = big-endian two 4-byte ints, like ByteBuffer.putInt()
    return struct.pack(">ii", c.id, len(name)) + name

producer.produce("customerCountries", key="Banana",
                 value=serialize_customer(Customer(7, "Banana Inc")))
producer.flush()
```

P7. Avro serializer with Schema Registry

Schema stored in the registry; only the schema id travels in each record. Java original: [→ Avro producer · Part 2 p.30](#)

```
from confluent_kafka import SerializingProducer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka.serialization import StringSerializer

schema_str = """
{"namespace": "customerManagement.avro", "type": "record", "name": "Customer",
 "fields": [{"name": "id", "type": "int"},
             {"name": "name", "type": "string"},
             {"name": "email", "type": ["null", "string"], "default": null}]}
"""

registry = SchemaRegistryClient({"url": "http://localhost:8081"}) # schema.registry.url
avro_ser = AvroSerializer(registry, schema_str, lambda obj, ctx: obj) # dict -> Avro

producer = SerializingProducer({
    "bootstrap.servers": "localhost:9092",
    "key.serializer": StringSerializer("utf_8"),
    "value.serializer": avro_ser,
})

for n in range(3):
    customer = {"id": n, "name": f"exampleCustomer{n}", "email": f"example{n}@example.com"}
    producer.produce(topic="customerContacts", key=customer["name"], value=customer)
producer.flush()
```

P8. Custom partitioner ("Banana" gets its own partition)

The Python client lets you pick the partition explicitly. Java original: [→ Custom partitioner · Part 2 p.35](#)

Rule: "Banana" → last partition; every other key hashed over the remaining partitions. (Java version implements the Partitioner interface.)

```
import zlib

def banana_partition(topic, key: str) -> int:
    n = len(producer.list_topics(topic, timeout=10).topics[topic].partitions)
    if key is None:
        raise ValueError("We expect all messages to have customer name as key")
    if key == "Banana":
        return n - 1 # always the last partition
    return zlib.crc32(key.encode()) % (n - 1) # hash over the rest

key = "Acme"
producer.produce("customerCountries", key=key, value="USA",
                 partition=banana_partition("customerCountries", key))
producer.flush()
```

P9. Record headers

Metadata (lineage, routing, tracing) without touching key/value. Java original: [→ Headers · Part 2 p.37](#)

```
producer.produce("CustomerCountry", key="Precision Products", value="France",
                 headers=[("privacy-level", b"YOLO"), ("source", b"web-shop")])
producer.flush()
```

P10. Interceptor-style counter (onSend / onAcknowledgement)

Python has no interceptor plug-in; wrap produce() and the callback. Java original: [→ Interceptors · Part 2 p.37](#)

```
import threading, time
from confluent_kafka import Producer

class CountingProducer:
    def __init__(self, conf, window_s=10):
        self.p = Producer(conf)
        self.sent = self.acked = 0
        self.lock = threading.Lock()
        threading.Thread(target=self._report, args=(window_s,), daemon=True).start()

    def _ack(self, err, msg):          # like onAcknowledgement()
        with self.lock: self.acked += 1

    def produce(self, topic, **kw):    # like onSend(): count before sending
        with self.lock: self.sent += 1
        self.p.produce(topic, callback=self._ack, **kw)
        self.p.poll(0)

    def _report(self, window):
        while True:
            time.sleep(window)
            with self.lock:
                print("sent:", self.sent, "acked:", self.acked)
                self.sent = self.acked = 0

cp = CountingProducer({"bootstrap.servers": "localhost:9092"})
cp.produce("interceptor-test", value="hello")
cp.p.flush()
```

P11. JSON producer with kafka-python (orders_topic)

EXAM — CIE-2 2022: "Implement Kafka producer and consumer for the given scenario". Java original: [→ Sending · Part 2 p.8](#)

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),  # dict -> bytes
    acks="all",
    retries=5,
)
order_data = {"order_id": 1, "customer": "Asha", "amount": 2499.0}

future = producer.send("orders_topic", order_data)  # asynchronous
meta = future.get(timeout=10)                       # optional: synchronous wait
producer.flush()
print(f"Order {order_data['order_id']} sent to Kafka! partition={meta.partition} offset={meta.offset}")
```

Part 2 — Consumer Code

Chapter 4 equivalents

C1. Basic consumer + poll loop

Creating, subscribing, polling. Java original: [→ Poll loop · Part 2 p.60](#)

Mandatory: bootstrap.servers + group.id. Deserialization is done by you (msg.value().decode()) or by a DeserializingConsumer.

```

from confluent_kafka import Consumer

conf = {
    "bootstrap.servers": "broker1:9092,broker2:9092",
    "group.id": "CountryCounter",
    "auto.offset.reset": "earliest",
}
consumer = Consumer(conf)
consumer.subscribe(["customerCountries"])      # or regex: ["^test.*"]

country_count = {}
try:
    while True:
        msg = consumer.poll(1.0)              # the poll loop - keep polling!
                                                # timeout in seconds
        if msg is None:
            continue
        if msg.error():
            print("Error:", msg.error()); continue
        country = msg.value().decode("utf-8")
        country_count[country] = country_count.get(country, 0) + 1
        print(f"topic={msg.topic()} partition={msg.partition()} "
              f"offset={msg.offset()} key={msg.key()} value={country}")
        print(country_count)
finally:
    consumer.close()

```

C2. Automatic commit

Every 5 s the offsets from the previous poll are committed. Java original: [→ Automatic commit · Part 2 p.75](#)
 Convenient but a crash re-processes up to 5 s of events (duplicates).

```

conf = {
    "bootstrap.servers": "localhost:9092",
    "group.id": "CountryCounter",
    "enable.auto.commit": True,                # default
    "auto.commit.interval.ms": 5000,           # default 5 s
}

```

C3. Manual commit — synchronous

Most reliable; blocks until the broker confirms; retries. Java original: [→ commitSync · Part 2 p.77](#)

```

conf = {"bootstrap.servers": "localhost:9092", "group.id": "g1",
        "enable.auto.commit": False}          # we commit ourselves
consumer = Consumer(conf); consumer.subscribe(["customerCountries"])

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.topic(), msg.partition(), msg.offset(), msg.key(), msg.value())
    try:
        consumer.commit(message=msg, asynchronous=False) # commits offset+1 of this msg
    except Exception as e:
        print("commit failed", e)

```

C4. Manual commit — asynchronous

Faster; no retry (a late retry could overwrite a newer commit). Java original: [→ commitAsync · Part 2 p.78](#)

```

def on_commit(err, partitions):                # = OffsetCommitCallback.onComplete
    if err is not None:
        print("Commit failed for", partitions, err)

conf = {"bootstrap.servers": "localhost:9092", "group.id": "g1",
        "enable.auto.commit": False, "on_commit": on_commit}
consumer = Consumer(conf); consumer.subscribe(["customerCountries"])

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.offset(), msg.key(), msg.value())
    consumer.commit(message=msg, asynchronous=True) # send and carry on

```

C5. Combine async (loop) + sync (shutdown)

Common production pattern. Java original: [→ Combined · Part 2 p.80](#)

```
closing = False
try:
    while not closing:
        msg = consumer.poll(1.0)
        if msg is None or msg.error():
            continue
        print(msg.offset(), msg.key(), msg.value())
        consumer.commit(message=msg, asynchronous=True)    # fast; next commit = retry
        consumer.commit(asynchronous=False)                # last commit must succeed
except Exception as e:
    print("Unexpected error", e)
finally:
    consumer.close()
```

C6. Commit specific offsets (every 1000 records)

Commit **offset + 1** = the next message to read. Java original: [→ Specific offsets · Part 2 p.81](#)

```
from confluent_kafka import TopicPartition

current_offsets = {}                                # (topic, partition) -> next offset
count = 0
while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.topic(), msg.partition(), msg.offset(), msg.key(), msg.value())
    current_offsets[msg.topic(), msg.partition()] = msg.offset() + 1
    if count % 1000 == 0:
        offsets = [TopicPartition(t, p, o) for (t, p), o in current_offsets.items()]
        consumer.commit(offsets=offsets, asynchronous=True)
    count += 1
```

C7. Rebalance listener (commit before losing partitions)

on_assign / on_revoke / on_lost = onPartitionsAssigned / Revoked / Lost. Java original: [→ Rebalance listeners · Part 2 p.83](#)

```
from confluent_kafka import TopicPartition

current_offsets = {}

def on_assign(consumer, partitions):
    print("Assigned:", partitions)                # load state / seek here

def on_revoke(consumer, partitions):
    print("Lost partitions in rebalance. Committing:", current_offsets)
    offsets = [TopicPartition(t, p, o) for (t, p), o in current_offsets.items()]
    if offsets:
        consumer.commit(offsets=offsets, asynchronous=False)    # sync before rebalance

def on_lost(consumer, partitions):
    print("Partitions lost (cooperative):", partitions)

consumer.subscribe(["customerCountries"],
                   on_assign=on_assign, on_revoke=on_revoke, on_lost=on_lost)

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(msg.topic(), msg.partition(), msg.offset(), msg.key(), msg.value())
    current_offsets[msg.topic(), msg.partition()] = msg.offset() + 1
```

C8. Seek: beginning, end, or one hour ago

Start reading from a different offset. Java original: [→ Seek · Part 2 p.87](#)

```

import time
from confluent_kafka import TopicPartition, OFFSET_BEGINNING, OFFSET_END

# beginning / end of every assigned partition
def seek_beginning(consumer, partitions):
    for p in partitions: p.offset = OFFSET_BEGINNING # or OFFSET_END
    consumer.assign(partitions)
consumer.subscribe(["customerCountries"], on_assign=seek_beginning)

# rewind to one hour ago (offsets_for_times uses a timestamp in ms)
one_hour_ago = int((time.time() - 3600) * 1000)
tps = [TopicPartition(p.topic, p.partition, one_hour_ago) for p in consumer.assignment()]
for tp in consumer.offsets_for_times(tps, timeout=10):
    if tp.offset >= 0:
        consumer.seek(tp) # next poll starts here

```

C9. Clean exit (wakeup / shutdown hook)

Stop the loop, then **close()** so offsets are committed and a rebalance happens immediately. Java original: [→ Exit · Part 2 p.88](#)

```

import signal

running = True
def shutdown(signum, frame): # = ShutdownHook calling consumer.wakeup()
    global running
    print("Starting exit...")
    running = False
signal.signal(signal.SIGINT, shutdown)
signal.signal(signal.SIGTERM, shutdown)

try:
    while running:
        msg = consumer.poll(10.0) # long timeout = low CPU on quiet topics
        print("waiting for data..." if msg is None else msg.value())
        if msg is not None and not msg.error():
            consumer.commit(message=msg, asynchronous=False)
finally:
    consumer.close() # commits offsets, leaves the group
    print("Closed consumer and we are done")

```

C10. Custom deserializer (Customer)

Mirror of P6; producer and consumer must agree on the format. Java original: [→ Custom deserializer · Part 2 p.92](#)

```

import struct

def deserialize_customer(data: bytes):
    if data is None:
        return None
    if len(data) < 8:
        raise ValueError("Size of data received by deserializer is shorter than expected")
    cid, name_size = struct.unpack(">ii", data[:8])
    name = data[8:8 + name_size].decode("utf-8")
    return {"id": cid, "name": name}

msg = consumer.poll(1.0)
if msg and not msg.error():
    c = deserialize_customer(msg.value())
    print("current customer Id:", c["id"], "and name:", c["name"])

```

C11. Avro deserializer with Schema Registry

Uses the schema id in the record to fetch the writer schema. Java original: [→ Avro deserializer · Part 2 p.94](#)

```

from confluent_kafka import DeserializingConsumer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroDeserializer
from confluent_kafka.serialization import StringDeserializer

registry = SchemaRegistryClient({"url": "http://localhost:8081"})
consumer = DeserializingConsumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "CountryCounter",
    "key.deserializer": StringDeserializer("utf_8"),
    "value.deserializer": AvroDeserializer(registry),      # schema comes from the registry
    "auto.offset.reset": "earliest",
})
consumer.subscribe(["customerContacts"])
while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print("Current customer name is:", msg.value()["name"]) # value() is a dict
    consumer.commit(message=msg, asynchronous=False)

```

C12. Standalone consumer (no group, no rebalance)

assign() the partitions yourself — do not subscribe(). Java original: [→ Standalone · Part 2 p.96](#)

Not notified when partitions are added: re-check list_topics() periodically or restart.

```

from confluent_kafka import Consumer, TopicPartition

consumer = Consumer({"bootstrap.servers": "localhost:9092",
                    "group.id": "standalone",           # still needed to commit offsets
                    "enable.auto.commit": False})
parts = consumer.list_topics("topic", timeout=10).topics["topic"].partitions
consumer.assign([TopicPartition("topic", p) for p in parts]) # all partitions of "topic"

while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print(f"topic={msg.topic()} partition={msg.partition()} offset={msg.offset()} "
          f"key={msg.key()} value={msg.value()}")
    consumer.commit(message=msg, asynchronous=False)

```

C13. Assignor + static membership + key timeouts

Config only. Java original: [→ Assignors · Part 2 p.69](#) [→ Static membership · Part 2 p.56](#)

Strategies: range (default), roundrobin, cooperative-sticky (incremental rebalance). In librdkafka the plain "sticky" is not offered separately.

```

conf = {
    "bootstrap.servers": "localhost:9092",
    "group.id": "sales_group",
    "partition.assignment.strategy": "cooperative-sticky", # or "range" / "roundrobin"
    "group.instance.id": "consumer-1",                    # static member: restart w/o rebalance
    "session.timeout.ms": 10000,                          # dead if no heartbeat for this long
    "heartbeat.interval.ms": 3000,                        # ~ 1/3 of session timeout
    "max.poll.interval.ms": 300000,                      # max gap between poll() calls
    "fetch.min.bytes": 1,
    "fetch.wait.max.ms": 500,                             # = fetch.max.wait.ms
    "max.partition.fetch.bytes": 1048576,
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,
}

```

C14. JSON consumer with kafka-python (sales_data)

EXAM — CIE-1 2022 & CIE-2 2022: auto-convert JSON into a Python dictionary. Java original: [→ Deserializers · Part 2 p.90](#)

```

from kafka import KafkaConsumer
import json

def json_deserializer(data):
    return json.loads(data.decode("utf-8"))

consumer = KafkaConsumer(
    "sales_data",
    bootstrap_servers="localhost:9092",
    group_id="sales_group",
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    value_deserializer=json_deserializer,
)
print("Consumer is listening to 'sales_data' topic...")
for message in consumer:
    print(f"Topic: {message.topic}, Partition: {message.partition}, Offset: {message.offset}")
    print(f"Message Value (Deserialized): {message.value}")

```

C15. One consumer per thread (same group)

A consumer object is not thread-safe: never share one across threads. Java original: [→ Thread safety · Part 2 p.62](#)

```

import threading
from confluent_kafka import Consumer

def run_consumer(name, stop):
    c = Consumer({"bootstrap.servers": "localhost:9092", "group.id": "CountryCounter",
                  "auto.offset.reset": "earliest"})
    c.subscribe(["customerCountries"])
    try:
        while not stop.is_set():
            msg = c.poll(1.0)
            if msg and not msg.error():
                print(name, msg.partition(), msg.offset(), msg.value())
    finally:
        c.close()

stop = threading.Event()
threads = [threading.Thread(target=run_consumer, args=(f"C{i}", stop)) for i in range(3)]
for t in threads: t.start()
# ... later: stop.set(); [t.join() for t in threads]

```

EXAM TIP Before the exam

- Programs most likely to be asked: **P5 / P11** (producer) and **C14** (consumer). Be able to explain each config line.
- For any consumer program mention: group.id, deserializer, auto_offset_reset, commit strategy, and that the loop is the poll loop.
- These were written from the textbook's Java examples and the CIE answer keys; install the libraries and a local broker (localhost:9092) to run them.

Unit 5 — Building Data Pipelines & Kafka Streams

Syllabus: building data pipelines · when to use pipelines · when to use Kafka Connect vs producer/consumer · Kafka Streams · stream-processing design patterns · architecture overview · how to choose a stream-processing framework · examples: word count, stock-market statistics, click-stream enrichment

Sources: Shapira et al. *Kafka: The Definitive Guide* 2nd ed. Ch.9 (Building Data Pipelines) and Ch.14 (Stream Processing) + all question papers. **BOOK p.xxx** ↗ opens *Kafka.pdf* at that page. **Note:** *Kafka.pdf* is a reflowed e-book with **no printed page numbers**, so p.xxx = PDF page number (offset 0). Your *Unit5.pdf* extract skips the Kafka Connect internals (pp.340–368) and the stream-processing concepts (pp.598–613); both are filled here from the full book. Blue **highlight** = must-remember. **IMPORTANT** = asked before.

0 · Hot-list for Unit 5 (Q9/Q10 — choose one, 16 marks)

Rank	Topic	Asked in
1	Kafka Connect vs custom producer/consumer ; why Connect; its features	SEE Feb'26 9a (10M), 10b (8M) · CIE-1 Dec'24 5a
2	Considerations when building a data pipeline (fraud-detection example)	CIE-2 Dec'25 Q3 (10M) · Model QP 9a
3	Kafka pipeline for a scenario (logs from microservices)	SEE Feb'26 9b (6M)
4	Joins : stream–table, table–table, stream–stream (windowed)	CIE-2 Dec'25 Q4 (10M) · Model QP 10b
5	Processing with local state (diagram); internal vs external state	SEE Feb'26 10a (8M) · Model QP 1.10
6	Single-event processing pattern (order-confirmation e-mail)	Model QP 10a
7	Connect: connector types, RegexRouter, TimestampRouter, SMTs, "how Connect deals with connectors"	CIE-2 Dec'25 A4, A5 · Model QP 9b
8	Kafka Streams examples (word count, stock stats, click-stream) – "list two examples"	SEE Feb'26 1.10 · syllabus
9	Kafka as pipeline – why (decoupling buffer)	CIE-2 Dec'25 A3

1 · Building data pipelines **BOOK p.327** ↗

- Two common cases: (1) Kafka is **one of the endpoints** (Kafka → S3, MongoDB → Kafka); (2) Kafka is the **intermediary** between two systems (Twitter → Kafka → Elasticsearch). **Kafka Connect was added in version 0.9.**
- Main value of Kafka in pipelines: a very large, reliable buffer between stages** — it **decouples producers and consumers**, lets the same source data feed many targets with different timeliness/availability needs; with reliability, security and efficiency it fits most pipelines. **BOOK p.327** ↗ **IMPORTANT** CIE-2 Dec'25 A3 "Justify Kafka as an effective pipeline"

- Think of the bigger picture (≥ 2 endpoints) — short-term point-to-point integrations create a costly integration mess. [BOOK p.328 ↗](#)

1.1 Considerations when building pipelines [BOOK p.328 ↗](#) **IMPORTANT** CIE-2 Dec'25 Q3 (10M)

· Model QP 9a

#	Consideration	Key points	How Kafka helps
1	Timeliness BOOK p.329 ↗	Some systems want daily bulk, others ms latency; support both and ease migration	Kafka is a giant buffer decoupling time-sensitivity : real-time producers, batch consumers (or vice versa); easy back-pressure (consumers drive rate)
2	Reliability BOOK p.329 ↗	No single point of failure, fast automatic recovery; delivery guarantees — at-least-once (duplicates on retry) or exactly-once	Kafka gives at-least-once itself; exactly-once with transactional/unique-key stores; Connect API helps end-to-end exactly-once via offsets
3	High & varying throughput BOOK p.330 ↗	Scale to very high rates and adapt to sudden spikes	Buffer removes need for complex back-pressure; add producers/consumers independently; compression; hundreds of MB/s on modest clusters; Connect parallelises
4	Data formats BOOK p.331 ↗	Reconcile types (XML/relational in → Avro in Kafka → JSON to ES, Parquet to HDFS, CSV to S3); also pull vs push sources	Kafka and Connect are format-agnostic ; pluggable converters ; schema carried so changes (new MySQL column) can propagate
5	Transformations: ETL vs ELT BOOK p.332 ↗	ETL = transform in the pipeline (saves storage but ties hands of downstream users). ELT = minimal change, raw data to target, transform there (flexible, costs target CPU/storage)	Connect SMTs for light changes; joins/aggregations → Kafka Streams. Don't clean too early — keep raw data
6	Security BOOK p.334 ↗	Who can access ingested data, encryption in transit across data centres, who may change pipelines, authenticating to external stores, PII compliance	Wire encryption, SASL authentication, authorization, audit log; external secret providers (Vault) in Connect
7	Failure handling BOOK p.336 ↗	Plan for bad records: prevent, recover, fix and reprocess, detect late	Kafka stores events long-term so you can go back in time and replay
8	Coupling & agility BOOK p.336 ↗	Avoid ad-hoc pipelines per pair, loss of metadata (no schema evolution), extreme processing	One generic platform; preserve raw data and schemas; let downstream apps decide

🧠 **MNEMONIC** — "T-R-H-D-T-S-F-C" → "Tom Reads Huge Data To Stay Fully Current":
Timeliness · Reliability · High-throughput · Data formats · Transformations · Security · Failure handling · Coupling

HOW TO WRITE IT — CIE-2 Dec'25 Q3 / Model QP 9a — real-time fraud detection in online transactions

Apply each point to fraud: **Timeliness** ms–seconds (decision before card approval); **Reliability** no loss, at-least-once/exactly-once, replicated brokers (acks=all); **Throughput** spikes (sales days), partitions + consumer groups; **Formats** Avro + schema registry, schema evolution as new fraud features added; **Transformations** ELT — keep raw transactions, enrich in Kafka Streams; **Security** TLS, SASL, ACLs, PII masking (MaskField SMT), audit; **Failure handling** dead-letter queue for malformed events, replay; **Coupling** keep source systems decoupled via Kafka. (CIE-2 Dec'25 solution lists: scalability, reliability & fault tolerance, data format & schema evolution, timeliness, ETL vs ELT, access control, failure handling and coupling.)

2 · When to use Kafka Connect vs producer/consumer [BOOK p.338 ↗](#)

IMPORTANT SEE Feb'26 9a, 10b · CIE-1 Dec'24 5a

	Kafka clients (producer/consumer)	Kafka Connect
Use when	You can modify the application's code and want to push/pull data to/from Kafka (clients are embedded in your app)	Connecting Kafka to datastores you did not write and cannot/will not modify
Effort	You write code: data types, configuration, error handling, offsets, scaling, monitoring	Usually just write configuration files — connectors exist for most systems
Built-in features	You build them	Configuration management, offset storage, parallelisation, error handling, support for different data types, standard REST management API
Example	E-commerce app publishes order events; custom sensor service	MySQL/PostgreSQL → Kafka; Kafka → S3/Elasticsearch/data warehouse

- If no connector exists you can write an app with clients **or** use the Connect API; **Connect is recommended** because it supplies those features and is a standard part of the Kafka ecosystem. [BOOK p.339 ↗](#)

When a custom producer/consumer is better (limitations of Connect) **IMPORTANT**

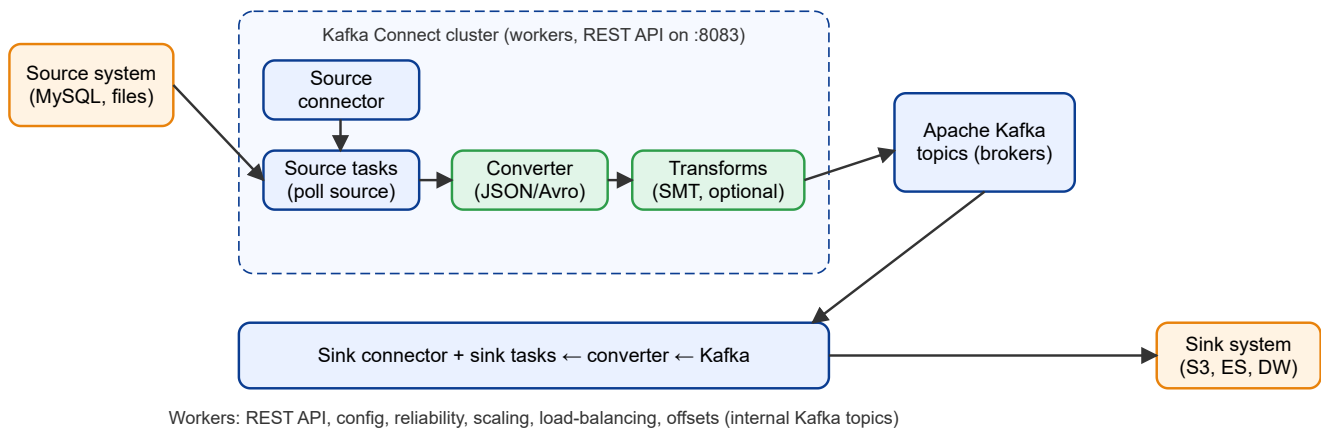
SEE Feb'26 9a(iii), 10b

- No connector exists for a proprietary/odd source **and** you control the source code ⇒ embed a producer directly (no extra moving part).
- Need **complex per-record business logic**, joins or aggregation — SMTs are stateless and single-message; use Kafka Streams/consumer code.
- Need **very low latency / fine control** over batching, partitioning, keys, transactions in application code.
- Source pushes data (events inside your app) rather than being a datastore to be polled; JDBC-style connectors poll tables with timestamps (inefficient/inaccurate) ⇒ use CDC (Debezium) or app-level events.

- Limitations of Connect themselves: extra cluster to operate; connector quality varies (FileStream connectors not for production); polling-based JDBC source adds load and may miss updates. (Our synthesis from pp.338, 347, 352.)

3 · Kafka Connect in detail [BOOK p.339 ↗](#)

Gap in *Unit5.pdf* — read from *Kafka.pdf* pp.339–368.



Kafka Connect architecture (redrawn — our diagram; the book describes it in text only, pp.360–365)

DEFINITION — Kafka Connect

Part of Apache Kafka: a **scalable and reliable way to copy data between Kafka and other datastores**. It provides APIs and a runtime for **connector plug-ins**. It runs as a **cluster of worker processes**; connectors are configured/managed through a **REST API**; connectors start **tasks** to move data in parallel; **converters** store data in Kafka in different formats.

[BOOK p.339 ↗](#)

Concept	Role (pp.360–364)
Connector	Decides how many tasks run, how to split the copying work between them, gets task configs from workers (e.g. JDBC source: min(tasks.max, number of tables)). Two types: Source connector pulls data from an external system into Kafka; Sink connector sends data from Kafka to an external system. IMPORTANT CIE-2 Dec'25 A4
Tasks	Actually move data in/out of Kafka. Source tasks poll the external system and return records; sink tasks receive records from Kafka via the worker and write to the target .
Workers	"Container" processes that run connectors and tasks: handle the REST requests , store connector config in an internal Kafka topic, start tasks, auto-commit offsets , handle retries; if a worker dies others reassign its connectors/tasks (heartbeats via the consumer protocol); new workers get load ⇒ scalability, fault tolerance, load balancing . Connectors/tasks = "moving data"; workers = REST, config, reliability, HA, scaling.
Converters & data model	Connect has its own in-memory Schema + Value objects. Converters (primitive types, byte arrays, strings, Avro, JSON, JSON Schema, Protobuf) turn them into bytes in Kafka and back ⇒ any connector works with any data format . JSON converter can include or omit schema.
Offset management (BOOK p.364 ↗)	Source records carry a logical partition and offset of the source system (file + line; table + timestamp/ID). After Kafka acknowledges, worker stores the offset (internal topic <code>offset.storage.topic</code> ; also <code>config.storage.topic</code> , <code>status.storage.topic</code>) ⇒ restart from last offset. Sink: workers commit Kafka offsets after <code>put()</code> succeeds.

Running Connect (BOOK p.340 ↗)

- Ships with Kafka; run on **separate servers** from brokers for production. Start: `bin/connect-distributed.sh config/connect-distributed.properties`; **standalone mode** (`connect-standalone.sh`) runs everything in one worker — used when a connector must run on a specific machine (e.g. syslog listener). (BOOK p.344 ↗)
- Key worker configs: `bootstrap.servers` (give ≥3), `group.id` (same id = same Connect cluster), `plugin.path` (connector jars), `key.converter` / `value.converter` (default JSON), `rest.host.name` / `rest.port` (REST on **8083**).
- Create a connector by POSTing JSON (name + config: `connector.class`, source/target, topic) to `/connectors`; example **FileStreamSource** → **topic** → **FileStreamSink** (pp.344–347, not for production), and **MySQL** → **Kafka (JDBC source, mode=timestamp, topic.prefix)** → **Elasticsearch (sink, key.ignore=true)** (pp.347–355).
- CDC**: JDBC connector polls tables (timestamp/ID) = inefficient; **change data capture** reads the DB transaction log — use the **Debezium** project. (BOOK p.352 ↗)

Single Message Transformations (SMTs) (BOOK p.356 ↗) **IMPORTANT** CIE-2 Dec'25 A5

(RegexRouter, TimestampRouter)

Stateless per-record changes done inside Connect while copying, often without code (set via `transforms` config). More complex (joins/aggregation) ⇒ stateful Kafka Streams.

SMT	Does
Cast	Change a field's data type
MaskField	Replace a field with null — remove sensitive/PII data
Filter	Drop/include messages matching a condition (topic name, header, tombstone)
Flatten	Nested structure → flat (concatenated field names)
HeaderFrom / InsertHeader	Move/copy fields into header / add a static header (e.g. lineage "MessageSource")
InsertField	Add a field (from metadata like offset, or static)
RegexRouter	Change the destination topic using a regular expression and a replacement string
ReplaceField	Remove or rename a field
TimestampConverter	Change time format of a field (Unix epoch → string)
TimestampRouter	Modify the topic based on the message timestamp — useful in sinks to write to time-partitioned tables

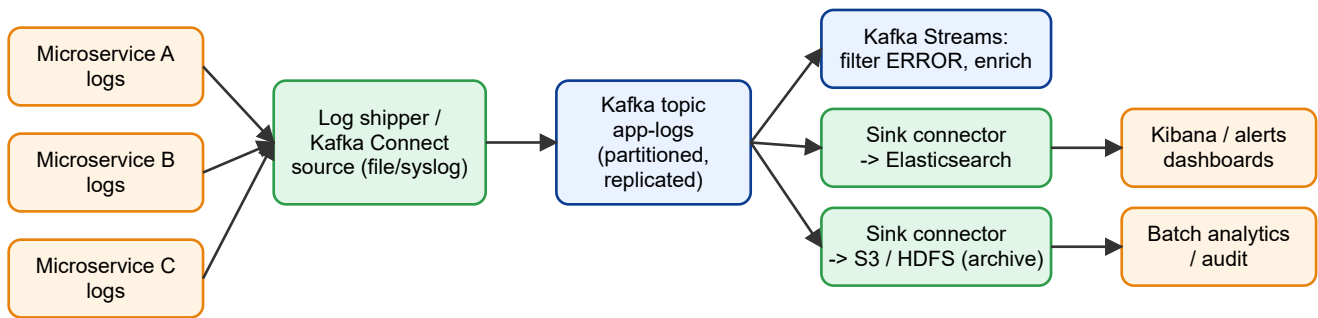
Error handling: `error.tolerance` lets a connector silently drop corrupt messages or route them to a **dead letter queue** topic. ([BOOK p.360](#) ➤) **Alternatives to Connect** ([BOOK p.366](#) ➤): Flume (Hadoop), Logstash/Fluentd (Elasticsearch) when Kafka is just one input; GUI ETL tools (Informatica, NiFi, StreamSets); stream-processing frameworks with built-in connectors (harder to troubleshoot lost/corrupt messages).

3.1 Worked answer — SEE Feb'26 9a (10M): MySQL + PostgreSQL + cloud data warehouse → Kafka **IMPORTANT**

- 1. Why Connect (3M):** sources are **existing datastores whose code you can't/won't change**; ready connectors exist (JDBC/Debezium source, warehouse sink) ⇒ only configuration; built-in config management, offset storage, parallelism, error handling, data-type support, REST API; Kafka decouples MySQL/PostgreSQL from the warehouse and gives timeliness from batch to near-real-time; avoids ad-hoc point-to-point pipelines.
- 2. Scalability & fault tolerance (4M):** distributed workers; connector splits work into `tasks.max` tasks (e.g. one per table); worker failure ⇒ tasks reassigned to other workers; offsets, configs and status stored in internal Kafka topics ⇒ resume after crash; converters + schemas handle schema changes; error tolerance with dead-letter queue; REST API; **CDC with Debezium** for efficient, accurate change capture.
- 3. Two limitations / when custom producer-consumer is better (3M):** (a) complex business logic, joins and aggregations (SMTs stateless) ⇒ Kafka Streams/consumer code; (b) no connector for the system or source you control and need precise control over keys/partitioning/transactions or ultra-low latency; also JDBC polling load and extra cluster to run.

3.2 SEE Feb'26 9b (6M): logs from multiple microservices → Kafka pipeline

IMPORTANT



Kafka pipeline for microservice logs (redrawn — our diagram)

1. Each microservice writes logs; a **source connector / log shipper** (file/syslog, or direct producers) publishes to the **app-logs topic** (partitioned by service/host, replicated).
2. Kafka is the **decoupling buffer**; retention lets late or slow consumers catch up.
3. **Kafka Streams** (single-event filter/map): route ERROR events to a high-priority topic, enrich with metadata (pattern of Fig 14-3).
4. **Sink connectors** to Elasticsearch (search/dashboards, alerts) and S3/HDFS (archive, audit, batch analytics). Use **SMT** TimestampRouter for dated indices, MaskField for PII; dead-letter queue for bad records; TLS/SASL security.

4 · Stream processing — concepts [BOOK p.596 ↗](#)

Gap in *Unit5.pdf* — read from *Kafka.pdf* pp.596–613.

- **Data stream (event stream) = abstraction of an unbounded dataset** — infinite, ever growing. Attributes: **ordered**, **immutable records** (cancellations are new events), **replayable**. [BOOK p.599 ↗](#)
- **Stream processing** = ongoing processing of one or more event streams; a programming paradigm like request-response and batch. [BOOK p.601 ↗](#)

Paradigm	Latency	Mode
Request-response	Lowest (sub-ms to few ms), OLTP	Blocking
Batch processing	High latency/high throughput (minutes–hours), data warehouse/BI	Scheduled
Stream processing	Between the two: continuous	Continuous, non-blocking

Key concepts [BOOK p.603 ↗](#)

Concept	Meaning
Topology BOOK p.603 ↗	Source streams → graph of stream processors (filter, count, group-by, left-join) connected by streams → sink streams. A DAG .
Time BOOK p.604 ↗	Event time (when it occurred — usually what matters), log append (ingestion) time (arrived at broker), processing time (when the app saw it — unreliable, avoid). Kafka Streams uses <code>TimestampExtractor</code> . Standardise time zones.
State BOOK p.607 ↗	Information kept across events (counts, averages, join data). Local/internal state : only that instance, embedded in-memory DB — very fast but limited by memory. External state : external datastore (Cassandra) — unlimited size, shared , but extra latency/complexity/availability issues. IMPORTANT Model QP 1.10 "Internal vs external state"
Stream-table duality BOOK p.609 ↗	Table = current state (mutable); stream = history of changes. Table → stream : capture changes (CDC). Stream → table : apply all changes = materialise . Example: shoe-store inventory (Fig 14-1).
Time windows BOOK p.611 ↗	Choose window size , advance interval (hopping; tumbling when advance = size), grace period (how long it stays updatable for late events); session window = size set by an inactivity gap; aligned to clock or not.
Processing guarantees BOOK p.613 ↗	Exactly-once via Kafka transactions + idempotent producer; enable with <code>processing.guarantee=exactly_once</code> (<code>exactly_once_beta</code> from Streams 2.6 / brokers 2.5 — more efficient).

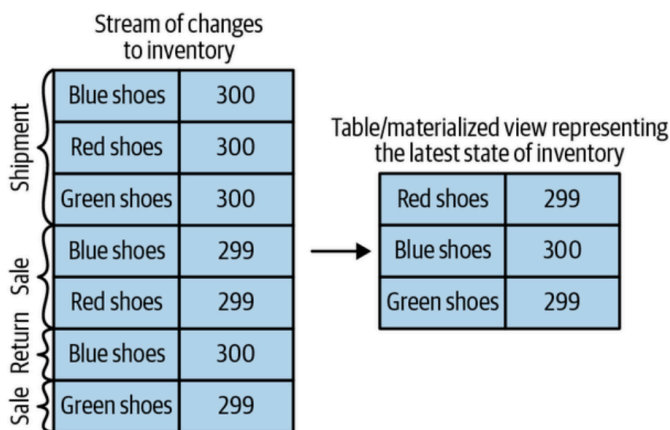


Fig 14-1 Materialising inventory changes (stream → table)

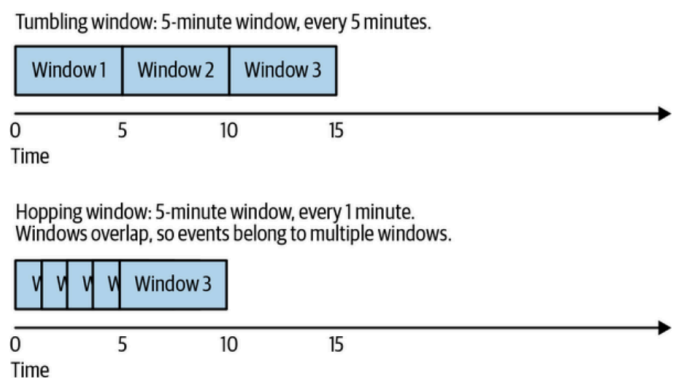


Fig 14-2 Tumbling vs hopping window

5 · Stream-processing design patterns [BOOK p.614 ↗](#)

5.1 Single-event processing (map/filter) [BOOK p.614 ↗](#) **IMPORTANT** [Model QP 10a \(order-confirmation e-mail\)](#)

- Process **each event in isolation**: consume, modify/filter, produce to another stream. **No state** ⇒ recovery/load-balancing trivial (hand the events to another instance). Examples: ERROR log lines → high-priority topic, others → low-priority; JSON → Avro. Can be done with a plain producer + consumer.

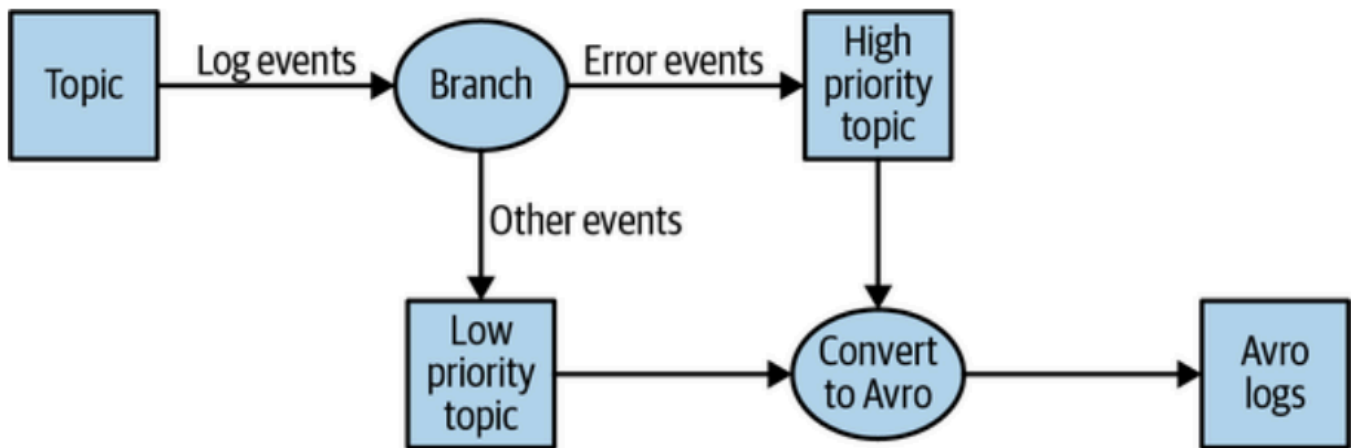


Fig 14-3 Single-event processing topology: Branch → error events to high-priority, others to low-priority → convert to Avro

5.2 Processing with local state BOOK p.615 ↗ **IMPORTANT** SEE Feb'26 10a (8M)

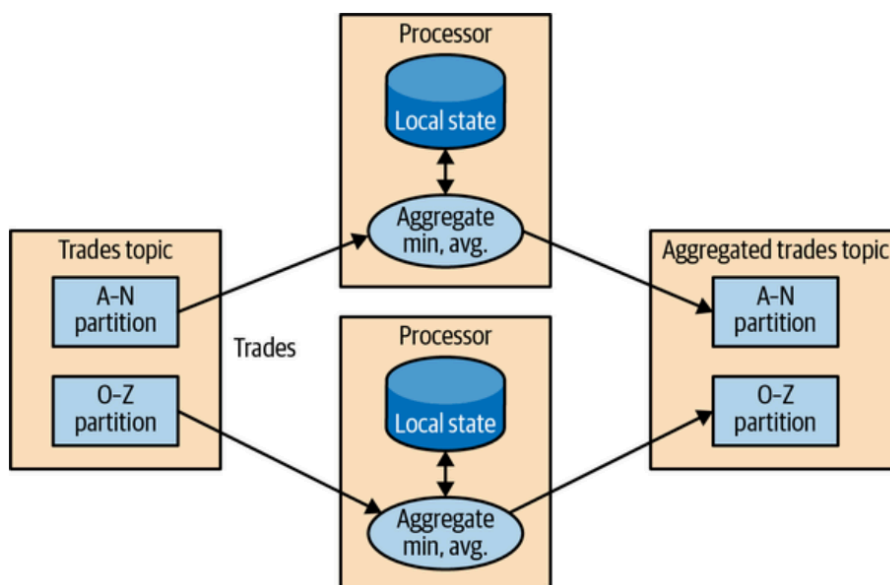


Fig 14-4 Topology for event processing with local state (min/avg price per stock)

- Windowed/group-by aggregates (min and average stock price per day) need state: store minimum, sum and count. Use **local state** because the aggregation is **per key** (stock symbol); a **partitioner** sends all events for one symbol to the same partition ⇒ each instance holds state only for its partitions.
- **Issues:** **Memory usage** (state should fit memory; spilling to disk hurts), **Persistence** (state must survive restarts — Kafka Streams keeps it in embedded **RocksDB** and also sends every change to a **Kafka changelog topic** (log-compacted), so it can be rebuilt by re-reading), **Rebalancing** (when a partition moves, the new owner must recover the right state).

🔗 **HOW TO WRITE IT — SEE Feb'26 10a (8M) "processing with local state — elucidate with diagram"**

Definition (the statement given) → Fig 14-4 → stock min/avg example → partitioner by key ⇒ local state → 3 issues with RocksDB + changelog solution → contrast with external state (§4) → when not enough (needs multiphase, below).

5.3 Multiphase processing / repartitioning [BOOK p.618 ↗](#)

- Local state only gives group-by results. For a global result (top 10 stocks of the day) use **two phases**: compute per-symbol daily gain on each instance (local state), write to a **new single-partition topic**, one instance finds the top 10. (MapReduce-like, but all steps in one app.)

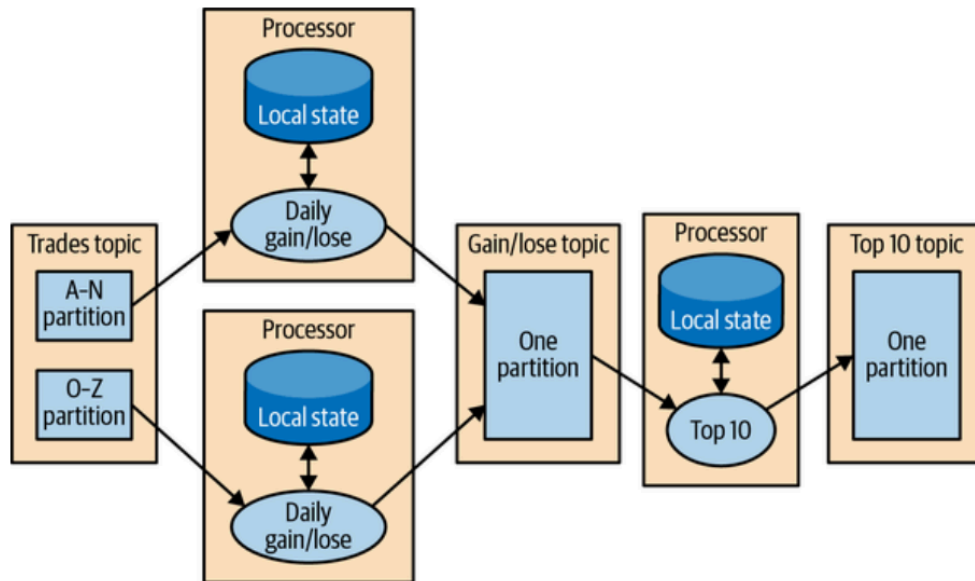


Fig 14-5 Local state + repartitioning steps

5.4 External lookup → stream-table join [BOOK p.619 ↗](#)

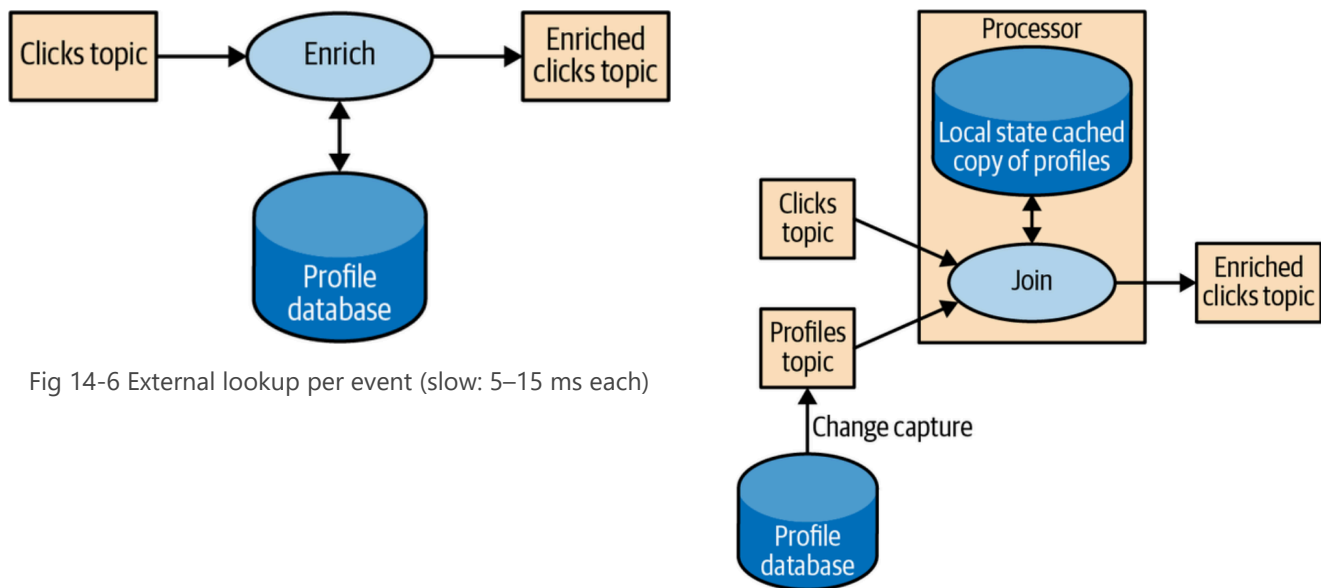


Fig 14-6 External lookup per event (slow: 5–15 ms each)

Fig 14-7 CDC keeps a local cached copy of the table (stream-table join)

- Direct DB lookup per event adds **5–15 ms latency**, overloads the DB (stream processors handle 100K–500K events/s, DB maybe 10K/s) and hurts availability. Caching goes stale. **Solution:** capture DB changes with **CDC** (Kafka Connect/Debezium) into a topic, keep a **local copy of the table in state** and join events against it.

Join	What is joined	Windowed?	Notes
Stream-table (enrichment) BOOK p.619 ↗	Each event of a stream with the current value in a table (materialised via changelog/CDC)	No	Inner, left; e.g. click stream enriched with user profile (like fact table joined with dimension)
Table-table BOOK p.622 ↗	Two materialised tables — current state of both at the time of the operation	No (always non-windowed)	Equi-join : same key, same partitioning ⇒ efficiently distributed; also foreign-key join (key of one joined with any field of other). Inner, left, outer.
Stream-stream (streaming / windowed) BOOK p.623 ↗	Two real event streams — entire history matched by same key within a time window	Yes	e.g. search queries ↔ clicks within seconds; keep a small window of each stream in RocksDB state

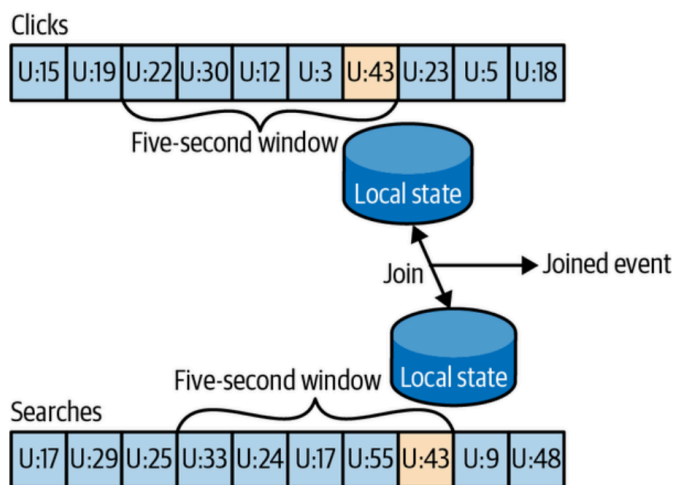


Fig 14-8 Joining two streams: windows on each side (user 43 appears in both)

- Streams in a join must be **partitioned on the same key with the same number of partitions**; Kafka Streams assigns the matching partitions to the **same task**. [BOOK p.624 ↗](#)

5.6 Out-of-sequence events & reprocessing [BOOK p.625 ↗](#)

- Out-of-sequence** (IoT device offline for hours): recognise it (event time older than current), define a **grace period** to reconcile, reconcile **in-band** (no "rerun yesterday's job"), and **update results**. Kafka Streams keeps multiple windows updatable in local state and writes a new aggregate result that replaces the old one (compacted result topic). Longer retention ⇒ more memory.
- Reprocessing**: (1) run the new app version as a **new consumer group** from the first offset, compare results, then switch clients (safer); (2) reset the buggy app (reset tool, clear state/output) — riskier. [BOOK p.627 ↗](#) **Interactive queries** read results straight from the state store. [BOOK p.628 ↗](#)



Fig 14-9 Out-of-sequence events (4:45–4:47 arriving late)

6 · Kafka Streams by example [BOOK p.629 ↗](#) **IMPORTANT** [SEE Feb'26 1.10 · syllabus](#)

- Kafka Streams = library (since 0.10.0) to consume, process and produce events in your own app — **no external framework or cluster**; just start more instances. Two APIs: low-level **Processor API** and high-level **Streams DSL** (`StreamsBuilder` → topology (DAG) → `KafkaStreams` object → `start()`). [BOOK p.629 ↗](#)
- Every app needs an `application.id` (unique per cluster; names internal topics/stores), `bootstrap.servers`, default Serdes (serialisation/deserialisation).

Example	Pattern shown	Key steps
Word count BOOK p.630 ↗	Map/filter + simple aggregate (local state)	Read <code>wordcount-input</code> lines → <code>flatMapValues</code> split on regex → map word as key → filter out "the" → <code>groupByKey</code> → <code>count()</code> → convert Long to String → <code>to("wordcount-output")</code>
Stock-market statistics BOOK p.633 ↗	Window aggregation (windowed local state)	Trades (ticker, ask price, ask size) → <code>groupByKey()</code> → <code>windowedBy(TimeWindows.of(5 s).advanceBy(1 s))</code> → aggregate into a <code>TradeStats</code> object (min price, count, sum) with a named state store (<code>Materialized</code> , <code>trade-aggregates</code>) → compute average → write windowed result to <code>stockstats-output</code> . Stats updated every second: min ask price, number of trades, average ask price per 5-s window.
Click-stream enrichment BOOK p.637 ↗	Joins (stream-table, stream-stream)	Streams: page views, searches; <code>KTable</code> : user profiles. <code>views.leftJoin(profiles)</code> (stream-table, keeps clicks of unknown users) → <code>leftJoin(searches, JoinWindows.of(1 s).before(0 s))</code> (stream-stream: clicks within 1 s after a search) ⇒ 360° view of user activity for recommendations.

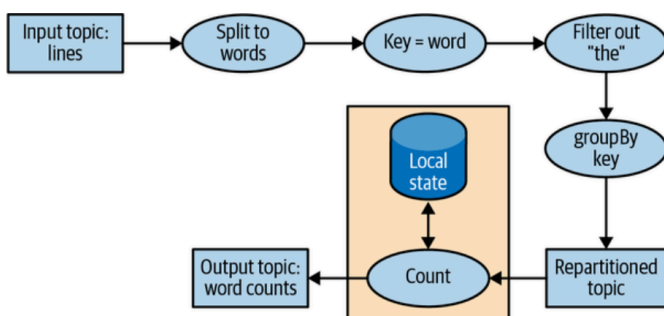


Fig 14-10 Word-count topology (source, split, key=word, filter, groupBy → repartition topic, count with local state, sink)

⚠ CONFLICT / TRAP — Window naming in the stock example

The Kafka book's stock example is a 5-second window advancing every 1 second = **hopping** window in Kafka terms, which Unit 3 (Streaming Data book) calls a **sliding** window. If the question says "window techniques", name it *sliding (hopping)* window and give size 5 s / advance 1 s.

7 · Kafka Streams architecture [BOOK p.641 ↗](#)

- **Topology** = set of operations/transitions each event goes through, made of **processors** (nodes): **source processors** read from a topic, **sink processors** write to a topic. Execution: define logical topology (KStream/KTable DSL) → `build()` creates the physical topology (optimisations possible) → `KafkaStreams.start()`. Test with **TopologyTestDriver** and integration tests (Testcontainers). [BOOK p.641 ↗](#)
- **Scaling** [BOOK p.644 ↗](#): Streams splits the topology into **tasks**; **number of tasks = number of partitions** of the input topics; each task processes its partitions independently with its own local state ⇒ **task = unit of parallelism**. More threads per instance, more instances on more servers; Kafka coordinates. A join puts all needed partitions in the same task ⇒ topics in a join need the same partition count and join-key partitioning. **Repartitioning** (groupBy new key) writes to a new topic and splits the topology into two sub-topologies that run independently.
- **Surviving failures** [BOOK p.649 ↗](#): data is in highly available Kafka; restart resumes from last committed offset; lost local state is rebuilt from the Kafka **changelog**; failed tasks restart on other threads/instances (consumer-group coordination, cooperative rebalancing). Faster recovery: aggressive log compaction, and **standby replicas** that keep state warm.

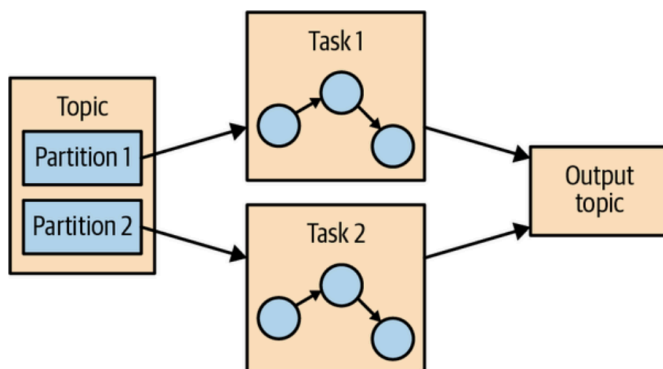


Fig 14-11 Two tasks, one per partition

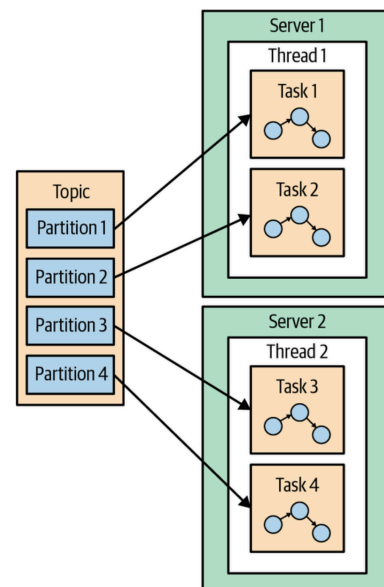


Fig 14-12 Tasks on multiple threads and servers

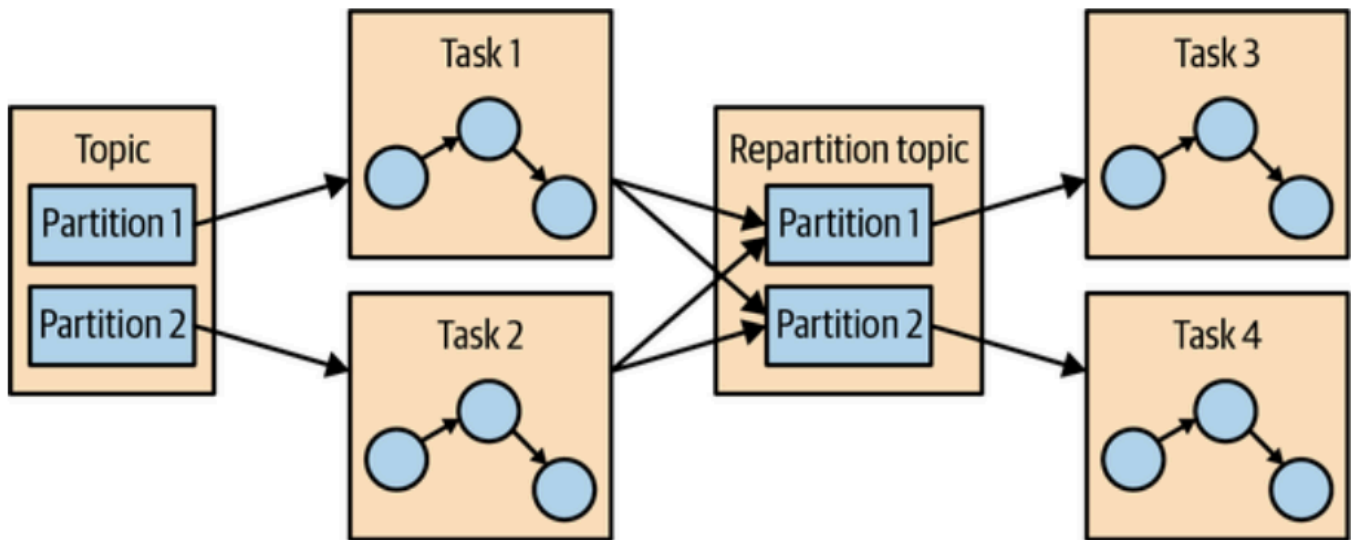


Fig 14-13 Repartition topic separates two sets of tasks

8 · Use cases & how to choose a stream-processing framework

[BOOK p.651 ↗](#)

- **Use cases:** customer service (instant reservation updates), Internet of Things (predictive maintenance), **fraud/anomaly detection** (e.g. beaconing in cybersecurity). [BOOK p.651 ↗](#)

Application type BOOK p.653 ↗	What to look for
Ingest (move data with small changes)	Reconsider — simpler Kafka Connect may do; else need good-quality connectors
Low-millisecond actions	Request-response may be better; else event-by-event low-latency model, not micro-batches
Asynchronous microservices	Integrates with message bus, change-data-capture, good local store as cache/materialised view
Near-real-time analytics	Great local-store support; advanced aggregations, windows, joins

Global criteria: **operability** (deploy, monitor, scale, reprocess), **usability of APIs / ease of debugging**, makes hard things easy, **community**. [BOOK p.655 ↗](#)

9 · Quick revision — Unit 5

Main value of Kafka in a pipeline?	Large reliable buffer decoupling producers and consumers.
Eight pipeline considerations?	Timeliness, reliability, throughput, data formats, transformations (ETL/ELT), security, failure handling, coupling/agility.
ETL vs ELT?	Transform in pipeline vs load raw and transform at target.
When Connect, when clients?	Connect: datastores you can't modify; clients: apps you can modify.
Connect added in Kafka version?	0.9.
Two connector types?	Source (into Kafka), sink (out of Kafka).
Connector vs task vs worker?	Plan/split work; move data; run both + REST/config/HA.
Default Connect REST port?	8083.
Converters available?	Primitive, bytes, string, Avro, JSON, JSON Schema, Protobuf.
Internal topics of Connect?	offset.storage.topic, config.storage.topic, status.storage.topic.
RegexRouter / TimestampRouter?	Change topic by regex / by message timestamp.
MaskField?	Null out sensitive field.
Standalone mode used for?	Connectors that must run on a specific machine.
Better than JDBC polling?	CDC (Debezium) reading the transaction log.
Local vs external state?	Fast but memory-bound vs unlimited but slower.
Where does Streams persist local state?	RocksDB + changelog Kafka topic.
Which join is non-windowed?	Table–table (and stream–table); stream–stream is windowed.
Number of Streams tasks?	= number of input partitions.
Standby replica?	Shadow task keeping state warm for fast failover.
Stock example window?	5 s, advance 1 s (hopping).
Three Kafka Streams examples?	Word count, stock-market statistics, click-stream enrichment.
Enable exactly-once in Streams?	processing.guarantee=exactly_once (or exactly_once_beta).

Quick Revision — Stream Processing & Analytics

Last-night sheet: numbers table · mnemonics · 80 rapid-fire Q&As · conflicts at a glance · Model-QP-only topics. Unit 4 (Kafka intro) has its own notes; only a few Unit 4 one-liners are repeated here because Part A covers the whole syllabus.

Paper names: **SEE Feb'26** = *SPA SEE 2022.pdf* (the exam it contains is dated Feb 2026) · **Model QP** · **CIE-1 Dec'24** and **CIE-2 Jan'25** (files in *QP/CIE/2021*, papers dated 26/12/2024 & 27/01/2025) · **CIE-1 Nov'25**, **CIE-2 Dec'25**, **CIE-3 Jan'26** (files in *QP/CIE/2022*). Blue **highlight** = must-remember. **BOOK p.x ↗** = Streaming-Data[1].pdf printed page (PDF = +20) or Kafka.pdf PDF page.

1 · Exam pattern in one box

Part A (20)	Q1: 10 questions × 2 marks, whole syllabus (mix of "define/list/draw", fill-ups and one-line scenario questions).
Part B (80)	5 questions × 16: Q2 compulsory (Unit 1) ; Q3/4 (Unit 2), Q5/6 (Unit 3), Q7/8 (Unit 4), Q9/10 (Unit 5) — choose one in each; max 3 sub-parts; SEE Feb'26 splits like 8+8, 10+6, 6+10, 8+8.
Marks tip	10-mark ≈ 1 diagram + 6–8 points + example; 6-mark ≈ diagram + 4 points; 2-mark ≈ 2 lines.

2 · Numbers & facts table

Fact	Value	Where
Latency: hard / soft / near	μs –ms / ms–s / s–min	Unit 1, Table 1.1 BOOK p.5 ↗
Interaction patterns	5 (req/resp, req/ack, pub/sub, one-way, stream)	Unit 1 BOOK p.15 ↗
Streaming blueprint tiers	5 + optional long-term storage	Unit 1 BOOK p.9 ↗
Google search stream example	46,000 events/s	Unit 1 BOOK p.27 ↗
NFL RFID tag rate	25 samples/s, 20 receivers, analysed within 0.5 s	Unit 1 BOOK p.23 ↗
HML overhead reduction vs RBML	43%	Unit 1 BOOK p.37 ↗
Failure points: queue path / stream processing	6 (Fig 3.12) / 7 (Fig 4.15)	Unit 2 BOOK p.48 ↗ BOOK p.73 ↗
State-machine replication cost	$k + 1$ times resources	Unit 2 BOOK p.75 ↗
Delivery semantics	3 (at-most / at-least / exactly-once)	Unit 2 BOOK p.47 ↗
Spark Streaming batch interval	< 0.5 s	Unit 2 BOOK p.66 ↗
Gas turbine data	~ 1 TB/hour	Unit 2 BOOK p.63 ↗
Fitbit window	length 30 min, slide 5 s	Unit 3 BOOK p.82 ↗
Reservoir sampling probability	k/n (16th item, $k=15 \Rightarrow 15/16$)	Unit 3 BOOK p.87 ↗
HyperLogLog: 1 billion items	2% error in 1.5 KB; precision 6 bits $\Rightarrow$ 64 registers	Unit 3 BOOK p.90 ↗
Count-Min: 8×128 array	$\approx 1.5\%$ error with 99.6% probability	Unit 3 BOOK p.92 ↗
Bloom filter space	$m/8$ bytes	Unit 3 BOOK p.93 ↗
Memory speeds	SSD seek 100,000 ns; DRAM 100 ns; L1 0.5 ns	Unit 3 BOOK p.100 ↗
Three long-term write options	message-by-message / batch from analysis / via queue + batch loader	Unit 3 BOOK p.97 ↗
Kafka Connect added in	Kafka 0.9	Unit 5 BOOK p.327 ↗
Kafka Streams introduced	0.10.0	Unit 5 BOOK p.596 ↗
Connect REST port	8083	Unit 5 BOOK p.342 ↗
Stream-table external lookup cost	5–15 ms per lookup; DB $\sim 10\text{K}$ ev/s vs 100K–500K ev/s	Unit 5 BOOK p.619 ↗
Stock example window	5 s, advance 1 s	Unit 5 BOOK p.633 ↗
Kafka Streams tasks	= number of input partitions	Unit 5 BOOK p.644 ↗

3 · Mnemonics (all in one place)

C-M-A-I-D(+L)	Collection → Message queue → Analysis → In-memory store → Data access (+ Long-term storage)
R-R-A-P-O-S	Request/Response, Request/Acknowledge, Pub/sub, One-way, Stream
RBML = front door, SBML = back door	Receiver logs on receipt; sender logs before sending; HML = both, async
μs, ms, minutes	Hard (none), Soft (low), Near (high tolerance)
"CCUNI"	Chronological, Continuous, Unique, Non-homogeneous, Imperfect (streaming data)
I-N-P-C-O-M-D	7 failure points: Incoming stream, Network, Processor, Connection-to-output, Output, Manager, Driver
S-S-F-S	Spark = small batches; Storm = single tuple; Flink = stream-first; Samza = stage-wise (YARN + Kafka)
"One Cat Really Dislikes"	One-pass, Concept drift, Resource limits, Domain constraints
Reservoir · HLL · Count-Min · Bloom	Sampling · Distinct · Frequency · Membership
RRW-WWW	Read-through, Refresh-ahead Write-through, Write-around, Write-back
T-R-H-D-T-S-F-C	Pipeline: Timeliness, Reliability, High throughput, Data formats, Transformations, Security, Failure handling, Coupling
Trigger = When, Eviction = Out	Trigger fires processing; eviction removes data
Sliding overlaps, Tumbling drains	Sliding: length + interval; Tumbling: window full ⇒ empty

4 · Conflicts & traps at a glance

⚠ ONE-GLANCE TRAP LIST (details in each unit)

1. **Soft vs near examples** — book puts stock quotes under soft, slides put stock trading under near. Quote the Table 1.1 latencies, use slide examples (Unit 1 §1.1).
2. **Checkpointing** — book: bad fit for streaming; CIE scheme: checkpoint + logging together. Write both (Unit 1 §6).
3. **Which logging?** power-failure at the node ⇒ RBML; crash of the receiver with senders able to resend ⇒ SBML (Unit 1).
4. **6 vs 7 failure points** — queue path 6, stream processing 7 (Unit 2).
5. **Exactly-once in Kafka** — 2017 book says not provided; 2021 Kafka book says yes via idempotence + transactions (Unit 2).
6. **Fig 4.11 caption** says at-least-once but shows at-most-once (Unit 2).
7. **Sliding vs hopping** — same thing in different books (Unit 3/5).
8. **Table 5.3 / text** — Flink "only" native tumbling support, Storm can implement easily (Unit 3).
9. **Stream time/event time** (Streaming Data) vs event/log-append/processing time (Kafka) — don't mix (Unit 3/5).
10. **Range-assignor question (Unit 4, CIE-1 Nov'25 & SEE Feb'26 8a)** — official solution gives C1:p0-2, C2:p3-5; with C3: 2 each; the "issue" it states (uneven load) does not really arise with 6 partitions/3 consumers; the true issue is a consumer beyond the partition count sits idle (e.g. 7th consumer) and Range can be unbalanced across several topics. See Unit 4 notes; write the official assignment + mention these.
11. **Paper file names** — folder "2021" holds Dec'24/Jan'25 papers; folder "2022" holds Nov'25–Feb'26 papers.

5 · 80 rapid-fire Q&As (likely MCQs / Part A)

Unit 1 (1–20)

1. Hard real-time example?	Pacemaker / anti-lock brakes.
2. Which real-time class has seconds–minutes latency?	Near.
3. Streaming data system is ___ real-time.	Non-hard.
4. Name the five tiers of the blueprint.	Collection, message queuing, analysis, in-memory store, data access.
5. Optional tier in the blueprint?	Long-term storage.
6. Security is added how?	Overlaid: authentication + authorization at tiers.
7. Vertical scaling means?	Add resources to the same server.
8. Horizontal scaling means?	Add more servers.
9. Scaling goal for streaming tiers?	Horizontal.
10. Pattern with acknowledgement token?	Request/acknowledge.
11. Fire-and-forget pattern?	One-way.
12. Pattern where the service becomes the client?	Stream.
13. Half-async means?	Only one side (client or service) is asynchronous.
14. Clickstream collection uses which pattern?	Stream.
15. Why are request/response services easy to scale?	Stateless: any instance can serve any client.
16. Rule for buffering layer in stream scaling?	No business logic on messages read from the stream.
17. Checkpointing stores?	Global snapshot of whole system state.
18. Data node loses messages after power failure — technique?	RBML.
19. SBML deletes a logged message when?	After the next tier acknowledges it.
20. HML advantage?	Same safety as RBML+SBML with lower overhead (async, one storage).

Unit 2 (21–40)

21. Three components of message queuing?	Producer, broker, consumer.
22. Main purpose of queue tier?	Decouple collection from analysis tier.
23. Problem if consumers are slower than producers without a queue?	Back-pressure $\Rightarrow$ data lost.
24. Durable messaging supports?	Offline consumers, replay, disaster recovery.
25. At-most-once means?	May be lost, never re-read.
26. At-least-once means?	Never lost, may be duplicated.
27. Exactly-once workaround techniques?	No blind retry; store last-message metadata.
28. Kafka metadata to store for exactly-once?	Offset.
29. Two data-quality bonuses?	Message auditing, duplicate detection.
30. Broker crash mitigation?	Ack after disk write, replication, little in memory.
31. In-flight data?	Data in motion, never at rest.
32. Continuous query model?	Query registered once; data flows through it; results pushed.
33. DBMS query model?	One-time pull on static data.
34. Storm master node?	Nimbus (supervisors are workers).
35. Flink master/worker?	JobManager / TaskManager.
36. Samza depends on?	YARN and Kafka.
37. Tuple-at-a-time vs stage-wise?	Storm vs Samza.
38. Idempotent job needed for?	At-least-once delivery.
39. Streaming manager fails — what happens?	Processors run headless.
40. Rollback recovery uses?	Checkpoint + log replay (lower overhead, slower recovery).

Unit 3 (41–58)

41. Stream time?	Time event enters the system.
42. Event time?	Time event occurred at source.
43. Difference between them called?	Time skew.
44. Trigger policy?	Rule for when to process the window.
45. Eviction policy?	Rule for removing data from the window.
46. Sliding window parameters?	Window length (eviction) + sliding interval (trigger).
47. Tumbling window overlap?	None.
48. Two tumbling types?	Count-based, temporal.
49. Spark Streaming time support?	Stream time only.
50. Concept drift?	Statistical properties of data change over time.
51. Load shedding?	Dropping tuples that can't be processed in time.
52. Reservoir sampling inclusion probability?	k/n .
53. Algorithm for distinct counts?	HyperLogLog.
54. Algorithm for frequency?	Count-Min Sketch.
55. Bloom filter errors?	False positives yes, false negatives no.
56. Batch loader?	Reads batches from queue tier, writes to long-term store.
57. Cache that acknowledges before store write?	Write-back.
58. IMDB design philosophy?	Memory-first, disk for durability.

Unit 4 (59–66) — repeated from past papers; full notes in Unit 4 PDF

59. Three ways of sending messages with a Kafka producer?	Fire-and-forget, synchronous, asynchronous (callback).
60. At-least-once producer settings (SEE Feb'26 1.9)?	acks = all, retries > 0 (large), enable.idempotence = false (true adds duplicate protection) — check Unit 4 notes for the exact wording.
61. Role of MirrorMaker?	Copies (mirrors) data between Kafka clusters. BOOK p.369 ↗
62. Consumer group purpose?	Share partitions among consumers for scaling; one partition → one consumer in a group.
63. Eager vs cooperative rebalance?	Stop-all-and-reassign vs move only affected partitions (CIE-1 Nov'25 A5).
64. Range assignor, 6 partitions, 2 consumers?	C1: P0–P2, C2: P3–P5.
65. Why Kafka over other pub/sub systems?	Durable log, scalable, replicated, high throughput, ecosystem (Connect/Streams).
66. Kafka ecosystem inputs/outputs?	Applications create data → Kafka → metrics, reports, loops back.

Unit 5 (67–80)

67. Kafka's main value in a pipeline?	Reliable buffer decoupling producers/consumers.
68. ETL?	Transform inside the pipeline.
69. ELT?	Load raw, transform at target.
70. Use Kafka Connect when?	Datastore code can't be changed; connector exists.
71. Two connector types?	Source and sink.
72. Who stores Connect offsets?	Workers, in internal Kafka topics.
73. RegexRouter?	Change destination topic via regex + replacement.
74. TimestampRouter?	Change topic based on message timestamp.
75. Better than JDBC polling for databases?	CDC (Debezium).
76. Local state store in Kafka Streams?	RocksDB + changelog topic.
77. Windowed join is which?	Stream–stream.
78. Single-event pattern also called?	Map/filter; stateless.
79. Multiphase processing needed when?	Result needs all data (top-10) ⇒ repartition to a single-partition topic.
80. Name two Kafka Streams examples.	Word count, stock-market statistics (also click-stream enrichment).

6 · Model-QP-only topics (outside the current syllabus — read if time permits)

The Model QP (course code 21AI72) puts **Chapter 7 "Making the data available"** in its Unit IV: 7a "elaborate any two protocols used to send data to the client", 8a "different ways to integrate filtering for each protocol", 1.7 "two drawbacks of data sync". No CIE/SEE has asked it, and the current Unit IV syllabus is Kafka only, so low probability. The Unit 4 notes do not cover this; summary below (Streaming Data Ch.7, printed pp.113–131).

Communication pattern BOOK p.114 ↗	Idea	Drawbacks
Data Sync BOOK p.115 ↗	Client gets full data set once, then only changes	Large data set / bandwidth; may not fit device; version conflicts; need merge policy Model QP 1.7
RMI/RPC BOOK p.116 ↗	API calls a method on the client when data arrives	Failure detection hard; may overwhelm client
Simple messaging BOOK p.117 ↗	Client asks for latest data, API replies	Chatty; no new-data alert; large payload after being offline
Publish-subscribe BOOK p.118 ↗	Client subscribes to a channel; API pushes on change	More complex protocol; API tracks clients

Table 7.1 — protocols [BOOK p.129 ↗](#)

Protocol	Message frequency	Direction	Latency	Efficiency	Fault tolerance
Webhooks BOOK p.119 ↗	Low	Uni (server→client, HTTP POST to registered callback)	Average	Low	None
HTTP long polling BOOK p.121 ↗	Average	Bi	Average	Average	None
Server-sent events BOOK p.123 ↗	High	Uni; single persistent HTTP connection; push-proxy mode saves mobile power	Low	High	None by default, partial
WebSockets BOOK p.126 ↗	High	Bi (HTTP handshake then TCP, full duplex)	Low	High	None by default, can be completely implemented

Filtering [BOOK p.129 ↗](#): **where** — analysis tier (aggregations), streaming API tier (raw streams), client; **type** — **static** (decided in advance, like an SQL view) vs **dynamic** (client-driven at run time, like an arbitrary query). Per protocol (Table 7.2 [BOOK p.131 ↗](#)): webhooks — capture the query at endpoint registration; long polling — encode filter in the URL; SSE — filter in the EventSource URL/query parameters; WebSockets — most flexible, client sends a message with the filter after connecting.

Part B — Answer Outlines

A 6–8 point skeleton for every past or likely long question, with the diagram/table to draw. Marks shown are as printed in the paper. Use with the unit notes (§ numbers point to them).

How to use: write the numbered points as headings/first lines, expand each with 1–2 sentences, draw the named figure with labels. **IMPORTANT** = asked in a paper. **DRAW:** = diagram to include. **Marks guide:** 16 marks Part B question = typically 2 sub-parts; spend ~1.2 min per mark. Paper names: SEE Feb'26 (*SPA SEE 2022.pdf*), Model QP, CIE-1 Dec'24, CIE-2 Jan'25, CIE-1 Nov'25, CIE-2 Dec'25, CIE-3 Jan'26.

UNIT 1 — Q2 (compulsory)

1.1 Streaming pattern for collecting data, with example **IMPORTANT** SEE Feb'26 2a (8M) ·

CIE-1 Dec'24 3a (5M)

1. Define: stream pattern = collection tier **connects to a stream source and pulls data in**; one request → continuous responses (Unit 1 §4.5).
2. **DRAW:** Fig 2.9 bottom (stream source ⇌ service, "Response (continuous)") next to request/response optional.
3. Roles flipped: service becomes the client; vs request/response where an external client pushes.
4. Characteristics: asynchronous, multiple chunks, non-blocking.
5. Example: Meetup RSVP JSON stream over one long-lived HTTP connection / connect to Twitter for sentiment analysis.
6. "Ingest a stream and produce another stream": analysis output is a new stream.
7. Scaling: direct persistent connection leaves nodes idle ⇒ buffering layer, no business logic (Figs 2.11, 2.12).
8. When to use: live updates, large datasets in chunks, video streaming.

1.2 Publish/subscribe pattern with diagram **IMPORTANT** SEE Feb'26 2b (8M) · CIE-1 Dec'24 2a

(6M) · CIE-1 Nov'25 PartA

1. Definition: producers publish to a broker by topic; broker delivers to all subscribed consumers; asynchronous decoupling.
2. **DRAW:** Fig 2.7 (4 producers → Messages → Broker with Topic A/B/C and Subscriptions → Consumers).
3. 6 flow steps (publish, sort by topic, subscribe, broker routes, deliver to all subscribers, async).
4. Component table: publisher, subscriber, topic, broker, message, subscription.
5. Push vs pull delivery by product; producer need not subscribe.
6. Benefits: decoupling, scalability from 4 cars to all cars; broker can add persistence/reliability.
7. Example: Fig 2.8 traffic topic — cars publish "current traffic", a car subscribes.
8. Drawbacks: broker is critical; delivery guarantees depend on product.

1.3 Streaming data architecture for a scenario **IMPORTANT** Model QP 2a (10M) · CIE-1 Dec'24

1a (10M vehicle tracking) · CIE-1 Nov'25 1a (10M IoT)

1. State the blueprint (Fig 1.5) — **DRAW**: five tiers + long-term storage + clouds for clients, labelled with scenario devices.
2. Map each tier to the scenario (Unit 1 §8 table: collection, queue, analysis, in-memory, data access, storage).
3. Interaction pattern justification (stream / one-way / pub-sub; request/ack where a token is needed).
4. Scaling model: horizontal, stateless collection nodes behind a load balancer, buffering layer.
5. Security: authentication + authorization at each tier, TLS in transit (Fig 1.6).
6. Fault tolerance: logging (RBML/SBML/HML), replicated durable queue, checkpoints.
7. Example flow of one message end-to-end (sensor → alert).

1.4 Importance of scaling **IMPORTANT** Model QP 2b (6M) · CIE-1 Dec'24 A3

1. Define scaling; two types.
2. **DRAW/TABLE**: vertical vs horizontal (restaurant vs highway).
3. Vertical limit = machine capacity.
4. Horizontal: add nodes, elastic/auto-scaling, goal for every tier.
5. Stateless patterns scale easily (load balancer, Fig 2.10).
6. Stream pattern needs buffering layer (Figs 2.11–2.12).
7. Benefits: throughput, availability, cost.

1.5 Fault tolerance in the collection tier: checkpointing, RBML, SBML, HML

IMPORTANT CIE-1 Dec'24 3b (5M) · CIE-1 Nov'25 1a · Model QP 1.2

1. Why: nodes fail; client data not reproducible; goal = no loss, recover as if no crash.
2. Checkpointing: global snapshot; possible loss after last snapshot; hard across layers.
3. Logging idea: replayable messages ⇒ consistent state without a snapshot.
4. **DRAW**: Fig 2.15 + 2.16 (RBML happy path + recovery).
5. SBML: **DRAW** Fig 2.17 (log before send, ACK, delete).
6. Comparison table (what logged / protects / recovery / cost).
7. HML: single storage, asynchronous writes, 43% less overhead.
8. Jointly: restore checkpoint, replay log after it (CIE scheme).

1.6 Real-time classes; overlap with streaming systems **IMPORTANT** CIE-1 Nov'25 2a (5M) ·

CIE-1 Dec'24 A1 · Model QP 1.1

1. Table 1.1 (hard/soft/near: examples, latency, tolerance).
2. Overlap: all need timely processing; soft/near blurry.
3. Hard: strict, failure = disaster.
4. Soft: deadlines preferred (video streaming); near: minimal lag (stock trading).
5. Streaming data system: non-hard real-time computation + clients consume when needed (Fig 1.4).
6. Conclusion: streaming relaxes strict deadlines (see trap box).

1.7 Compare interaction patterns **IMPORTANT** CIE-1 Nov'25 2b (5M)

1. List five patterns.
2. Request/response + half/full-async.
3. Request/acknowledge + token example.
4. Pub/sub + topic example.
5. One-way (lossy, telemetry).
6. Stream (continuous pull).
7. **TABLE:** response, sync/async, use, reliability (Unit 1 §4.6).

1.8 Sender-based vs receiver-based logging **IMPORTANT** CIE-1 Dec'24 3b (5M)

1. Definitions.
2. **DRAW:** Fig 2.14 with "we don't want to lose this" on both sides.
3. RBML: raw input, before processing, protects the node.
4. SBML: processed output, before sending, protects against next tier failure; needs ACK/delete.
5. Recovery steps for each.
6. Cost/overhead; HML compromise.

UNIT 2 — Q3 / Q4

2.1 Need for message queuing tier with example **IMPORTANT** Model QP 3a (10M) · CIE-2

Dec'25 1a (6M) · CIE-1 Dec'24 2b (4M)

1. Architecture without the queue (Fig 3.2) — tiers tightly coupled.
2. Decoupling of tiers/components.
3. Buffering surges / back-pressure (**DRAW** Fig 3.6 A→B→C).
4. Offline/slow consumers; durable storage.
5. Delivery guarantees, ordering, flow control.
6. Fault tolerance (data centre link cut).
7. Example: San Diego collection / Amsterdam analysis; or traffic app.

2.2 Producer, broker, consumer with a queuing layer **IMPORTANT** SEE Feb'26 4b (6M) · CIE-

2 Dec'25 2a (10M)

1. **DRAW:** Fig 3.5 (blueprint with producer → broker(queue) → consumer).
2. Producer sends message to broker.
3. Broker places it in a queue (abstraction over multiple queues).
4. Consumer reads from the broker.
5. Isolation: producer faster than consumer, durable messaging, offline consumers.
6. Delivery semantics + acknowledgements.
7. (10M) failure points of Fig 3.12 + security of Fig 3.14.

2.3 Build a message-queueing model for the traffic-replay product **IMPORTANT**

Model QP 3b (6M) · CIE-1 Dec'24 5b (4M logistics tracking)

1. Problem: transient queue discards consumed messages (Fig 3.10) — no history.

2. Choose a **durable** message queue (disk/JDBC/file).
3. **DRAW:** Fig 3.11 — real-time consumer (zero delay) + offline consumer $\Rightarrow$ historical data store.
4. Retain messages by city/day (topic or key); replay by re-reading.
5. Sizing/retention for week/month replay.
6. Security and replication of durable store.

2.4 Message delivery semantics for that scenario **IMPORTANT** Model QP 4b (6M) · SEE Feb'26

3a(iv) · CIE-1 Dec'24 A5

1. Define three semantics.
2. Pick exactly-once (history must be complete and not double counted) or at-least-once + dedup.
3. **DRAW:** Fig 3.13 — producer (no blind retry) $\rightarrow$ broker $\rightarrow$ consumer + metadata in persistent storage.
4. Technique 1: verify with broker instead of retrying.
5. Technique 2: store last message ID/offset.
6. Bonuses: auditing, duplicate detection.
7. Trade-off table (Table 3.1: complexity vs guarantee).

2.5 Seven points of failure in stream processing, with example **IMPORTANT** Model QP

4a (10M) · CIE-2 Jan'25 3a (10M) · SEE Feb'26 8b (10M) · CIE-1 Dec'24 4a · CIE-1 Nov'25 3a

1. **DRAW:** Fig 4.15 numbered 1–7.
2. Points 1–2: incoming stream and its network — handle gracefully.
3. Point 3: stream processor — manager restarts/moves it.
4. Points 4–5: output connection and destination — control back-pressure.
5. Points 6–7: streaming manager (headless) and application driver.
6. Consolidate: data loss + loss of resource management.
7. Techniques: replication + coordination; state-machine ($k+1$) vs rollback recovery.
8. Real example: fraud detection / ad billing (exactly-once needed). For "communication failures": checkpoint + log replay, idempotent jobs, acknowledgements. For "broker crash": replication/ISR, ack after write, consumers resume from committed offset, leader election.

2.6 Fault-tolerance challenges in the queuing tier: broker crash, inter-broker network, drive failure **IMPORTANT** SEE Feb'26 3b (8M)

1. **DRAW:** Fig 3.16 (3 brokers with storage).
2. Broker crash: in-memory messages at risk $\Rightarrow$ producer waits for disk-write ack.
3. Replicate to > 1 broker; hold little data in memory (performance trade).
4. Network disrupted: replicated data safe; broker rejoins and re-syncs; mid-send message risk.
5. Questions: new replica? delay before "down"? never restored?
6. Drive failure: replicas of lost data? un-flushed data lost? recover broker?
7. Impact on reliability: loss / duplicates $\Rightarrow$ choose delivery semantics.

2.7 Scenario: Coke vending machines **IMPORTANT** SEE Feb'26 3a (8M)

1. Impact of interruption: lost revenue/offers, wrong stock recommendations.
2. Days of data tolerable: depends on inventory handling/locale; small.
3. Historical data: yes, for reporting and inventory trends.
4. Delivery semantics: at-least-once (justify: duplicate offer harmless, missed offer costs sale; at-most-once arguable).
5. Architecture: machines → collection (one-way/pub-sub) → durable queue → analysis (geo + stock) → push/tweet.
6. Draw small blueprint.

2.8 Apache Samza architecture for a professional networking platform

IMPORTANT SEE Feb'26 4a (10M) · CIE-2 Jan'25 A1 (Samza diagram) · CIE-3 Jan'26 A1

1. Samza = stage-wise framework on YARN + Kafka.
2. **DRAW:** Unit 2 §7.2 figure + Fig 4.10 (Yarn client → resource manager → task runners ↔ Kafka).
3. Kafka input topics partitioned (events/sec, scalability).
4. YARN allocates containers; one task per partition.
5. Local state + Kafka changelog (state consistency, low latency).
6. Output topic to personalisation service (job chaining).
7. Failure: YARN restart + changelog replay; Kafka replication.
8. Semantics: at-least-once + idempotent updates.

2.9 Distributed stream-processing architecture + key components **IMPORTANT** CIE-3

Jan'26 1a (10M: need 2 + diagram 2 + explanation 6) · Model QP 1.4

1. Need: single machine cannot handle volume/velocity (1 TB/hour turbine).
2. **DRAW:** Fig 4.4.
3. Application driver.
4. Streaming manager.
5. Stream processors.
6. Data sources / output store.
7. Map to Spark/Storm/Flink/Samza (one line each + Fig 4.7–4.10).
8. Example: turbine monitoring (Fig 4.5).

2.10 Stream-processing frameworks — explain any two with diagram **IMPORTANT**

CIE-3 Jan'26 4 (10M) · CIE-2 Jan'25 2a · SEE Feb'26 1.4

1. List: Spark Streaming, Storm, Flink, Samza.
2. Pick two (suggest Storm + Flink, or Spark + Storm).
3. **DRAW:** Fig 4.8 / Fig 4.9 (or 4.7).
4. Components and roles (Nimbus/supervisors, JobManager/TaskManagers).
5. Processing model (tuple-at-a-time vs stream-first dataflow; micro-batch).
6. Unit of work: topology vs job.
7. Windowing/delivery support (Tables 5.2–5.3).

8. When to use.

2.11 DBMS vs streaming systems (query management) **IMPORTANT** CIE-2 Jan'25 1a (6M) ·

CIE-3 Jan'26 A2

1. **DRAW:** Fig 4.3 (query to data vs data through query).
2. One-time vs continuous query.
3. Pull vs push of results.
4. Changing data during downtime.
5. Query state after crash.
6. Active/passive inversion; Table 4.1.

2.12 State management; state-machine vs rollback recovery **IMPORTANT** CIE-3 Jan'26

2a (4M), 3b (4M)

1. State needed for aggregations/joins/late events.
2. Continuum (Fig 4.13) in-memory → replicated persistent store.
3. Ad impression/click join example (Fig 4.14).
4. State-machine: replicate job, same ordered input, $k+1$ resources, quick failover.
5. Rollback: checkpoint + log replay, lower overhead, slower recovery.
6. Table with use cases (intrusion detection vs moderate latency).

UNIT 3 — Q5 / Q6

3.1 Window type for treadmill energy/temperature **IMPORTANT** SEE Feb'26 5a (10M)

1. Requirements: independent, non-overlapping, resets each minute.
2. **Temporal tumbling window, 1 minute** (trigger = eviction = 1 min).
3. **DRAW:** timeline with windows [0–1), [1–2)... drained at each trigger (Unit 3 §2.5 figure).
4. Metric 1: sum power consumption per window, global.
5. Keep full messages so future split by gym location is possible.
6. Metric 2: filter temperature > threshold; alert; window resets.
7. Why not sliding (overlap double-counts); stream vs event time.
8. Framework: Flink built-in (Table 5.3).

3.2 Fitbit dashboard — sliding window **IMPORTANT** Model QP 5a (10M) · CIE-2 Jan'25 4a (10M)

1. Need: average speed per age group, refreshed every 5 s over last 30 min.
2. **Sliding window: length 30 min (eviction), interval 5 s (trigger)**.
3. **DRAW:** Fig 5.3 relabelled (30 min / 5 s) with overlapping windows.
4. Group by age band (12–17, 18–24, 25–34...).
5. Stream time vs event time + time skew.
6. Framework support (Table 5.2): Flink both times + custom policies; Spark stream time only; Storm/Samza need work.
7. Justify vs tumbling (would not overlap/refresh every 5 s).

3.3 Sliding vs tumbling **IMPORTANT** SEE Feb'26 6a (6M)

1. Definitions with trigger/eviction.
2. **DRAW:** Fig 5.3 and Fig 5.5 side by side.
3. Overlap vs non-overlap.
4. Policies (time-based vs count/time).
5. Types.
6. Use cases; framework support. Use the comparison table (Unit 3 §2.4).

3.4 Summarization techniques **IMPORTANT** SEE Feb'26 5b (6M), 6b (10M) · CIE-3 Jan'26 Q5 (10M) ·

CIE-2 Jan'25 1b

1. Why: unbounded, no memory, approximate acceptable.
2. Classify: sampling, distinct count, frequency, membership.
3. Reservoir sampling — k/n replacement (Fig 5.6).
4. HyperLogLog — hash, register index, leading zeros, harmonic mean (Fig 5.7).
5. Count-Min Sketch — hash per row, increment, min (Figs 5.8–5.9).
6. Bloom filter — set bits, AND to test, false positives only (Fig 5.10).
7. Example for each (ad network).
8. For "any two": spend ~5 marks each with diagram + steps + example + error type.

3.5 Reservoir sampling for 10 M ad views per minute **IMPORTANT** Model QP 5b (6M)

1. Problem: cannot hold in memory/disk; need random sample.
2. Fixed reservoir of k items filled with first k .
3. n -th item enters with probability k/n .
4. If chosen replaces a random reservoir element.
5. Equal probability for all items.
6. **DRAW:** Fig 5.6 with reservoir 15 and 16th item (15/16).
7. Implementable in any framework; example ad analysis.

3.6 Caching strategies **IMPORTANT** Model QP 6a (10M) · CIE-2 Jan'25 5a (10M, any three)

1. Caching system = in-memory key-value store (DRAM).
2. Read-through (Fig 6.4).
3. Refresh-ahead (Fig 6.5).
4. Write-through (Fig 6.6).
5. Write-around (Fig 6.7).
6. Write-back (Fig 6.8).
7. **TABLE** with pros/cons; products (Memcached write-around, EHCache, Hazelcast, Redis).
8. Risk: dual writes, node loss $\Rightarrow$ IMDB alternative.

3.7 Three ways to write data to long-term storage **IMPORTANT** CIE-3 Jan'26 2b (6M), 3a (6M

video analytics)

1. **DRAW:** Fig 6.2 (A, B, C).
2. A: message-by-message at stream speed.

3. B: batch in analysis tier.
4. A/B risk: store cannot keep up.
5. C: queue + batch loader (decoupling).
6. Batch loader features/tools (Gobblin, Secor).
7. Choose C for CCTV analytics (compliance, model training).

3.8 Bloom filter for membership queries **IMPORTANT** CIE-2 Jan'25 1b (4M)

1. Membership query = "has X been seen before?"
2. Bit array + hash functions.
3. Insert sets bits; **DRAW** Fig 5.10.
4. Query ANDs bits; any 0 $\Rightarrow$ never seen.
5. False positives, no false negatives.
6. Real use: blocklist check before expensive lookup.

UNIT 4 — Q7 / Q8 (full notes in the existing Unit 4 PDF)

Question	Where asked	Skeleton pointer
Commits and offsets; two commit strategies	SEE Feb'26 7a (10M)	Unit 4 notes — C6
Three ways of sending messages	SEE Feb'26 7b (6M)	Unit 4 notes — B3
Range assignor 6 partitions	SEE Feb'26 8a (6M) · CIE-1 Nov'25 5a	Unit 4 notes — C5; see trap box in Quick Revision §4
E-commerce fraud: failures, communication failures, broker crash	SEE Feb'26 8b (10M)	Use Unit 2 §2.5 + §2.6 (seven points, broker crash) and Unit 1 §1.5 (checkpoint/logging) — this "Unit 4" question is really Unit 2 content
Kafka ecosystem diagram; why Kafka; producer program; JSON consumer program	CIE-1 Nov'25, Model QP	Unit 4 notes — A6, A5, B9, C9

UNIT 5 — Q9 / Q10

5.1 Why Kafka Connect for MySQL + PostgreSQL + warehouse; features; limitations **IMPORTANT** SEE Feb'26 9a (10M)

1. Systems are existing datastores you cannot modify $\Rightarrow$ Connect with ready connectors, configuration only.
2. **DRAW:** Connect architecture (Unit 5 §3): source $\rightarrow$ workers/tasks/converters $\rightarrow$ Kafka $\rightarrow$ sink.
3. Built-in: config management, offset storage, parallelism, error handling, data types, REST API.
4. Scalability: tasks.max, distributed workers.
5. Fault tolerance: worker failure $\Rightarrow$ reassignment; offsets/configs in Kafka topics; dead-letter queue.

6. CDC (Debezium) for accurate change capture.
7. Limitation 1: complex business logic / joins (SMTs stateless) $\Rightarrow$ custom consumer or Kafka Streams.
8. Limitation 2: no connector / need fine control or app-embedded events $\Rightarrow$ custom producer/consumer.

5.2 Criteria for choosing Connect vs custom producers/consumers **IMPORTANT** [SEE](#)

Feb'26 10b (8M) · CIE-1 Dec'24 5a

1. **TABLE:** can you modify the source code? $\Rightarrow$ clients; datastore you can't change $\Rightarrow$ Connect.
2. Effort: configuration vs coding.
3. Built-in features: offsets, parallelism, error handling, REST.
4. Data formats via converters.
5. Transformation needs: SMT vs Streams.
6. Operational overhead of a Connect cluster.
7. Latency/control needs.
8. Example: MySQL $\rightarrow$ Kafka via Connect; order events from your own app via producer.

5.3 Kafka pipeline for logs from microservices **IMPORTANT** [SEE Feb'26 9b \(6M\)](#)

1. **DRAW:** Unit 5 §3.2 diagram.
2. Services $\rightarrow$ source connector/shipper $\rightarrow$ app-logs topic.
3. Kafka as buffer, retention, replication.
4. Kafka Streams filter/enrich.
5. Sink connectors $\rightarrow$ Elasticsearch, S3.
6. SMT (TimestampRouter, MaskField); dead-letter queue.
7. Security and monitoring.

5.4 Considerations for a Kafka-based fraud-detection pipeline **IMPORTANT** [CIE-2](#)

Dec'25 Q3 (10M) · Model QP 9a

1. Timeliness (ms decisions).
2. Reliability / delivery guarantees.
3. Throughput and scalability.
4. Data formats and schema evolution.
5. ETL vs ELT.
6. Security / access control.
7. Failure handling (dead letters, replay).
8. Coupling and agility. **TABLE** per consideration with fraud example.

5.5 Table–table and streaming joins, with example **IMPORTANT** [CIE-2 Dec'25 Q4 \(10M\) ·](#)

Model QP 10b

1. Stream–table: enrichment from local table copy via CDC (Fig 14-7).
2. Table–table: non-windowed, current state of both, equi-join (same key and partitioning) or foreign-key join.
3. Streaming (windowed) join: match same key within time window (Fig 14-8).

4. **DRAW:** Fig 14-7 and Fig 14-8.
5. Examples: click enriched with profile; search ↔ click within 1 s.
6. Requirements: same partition count/key; same task; RocksDB window state.
7. Join types: inner/left/outer where applicable.

5.6 Processing with local state **IMPORTANT** SEE Feb'26 10a (8M) · Model QP 1.10

1. Definition (given statement).
2. **DRAW:** Fig 14-4 (trades topic partitions → processors with local state → aggregated topic).
3. Example: min and average stock price per day (store min, sum, count).
4. Partition by key so each instance holds its subset.
5. Issues: memory, persistence, rebalancing.
6. Kafka Streams: RocksDB + changelog topic (log compaction).
7. Local vs external state comparison.

5.7 Single-event processing pattern — order-confirmation e-mail **IMPORTANT** Model

QP 10a

1. Pattern: process each event in isolation (map/filter).
2. **DRAW:** Fig 14-3 adapted: orders topic → processor → confirmation-email action.
3. Order event (order ID, customer ID, products, total) consumed.
4. Validate/enrich/map to e-mail message (stateless).
5. Send e-mail; optionally produce to "confirmations" topic.
6. No state ⇒ easy recovery/scaling; another instance can take over.
7. Implementable with a plain consumer or Kafka Streams.

5.8 Kafka Connect — connectors, types, SMTs **IMPORTANT** Model QP 9b · CIE-2 Dec'25 A4, A5

1. Connector = plug-in deciding number of tasks and splitting work.
2. Two types: source / sink.
3. Tasks do the data movement; workers run connectors/tasks.
4. Workers handle REST config, offset commits, failure reassignment.
5. Converters (JSON/Avro/Protobuf).
6. SMT list; RegexRouter and TimestampRouter definitions.
7. **DRAW:** Connect architecture figure.

5.9 Kafka Streams architecture / word count / stock stats / click-stream

IMPORTANT syllabus · SEE Feb'26 1.10

1. Library, no external cluster; topology = DAG of source/processor/sink nodes.
2. **DRAW:** Fig 14-10 (word count topology).
3. Tasks = partitions (Figs 14-11/14-12); repartition topic (Fig 14-13).
4. Word count: flatMapValues → map → filter → groupByKey → count.
5. Stock stats: 5 s window advancing 1 s, aggregate min/count/avg.
6. Click stream: stream-table leftJoin + stream-stream windowed join.
7. Failures: changelog rebuild, standby replicas, offsets.